



VS Code (全称 *Visual Studio Code*) 是一款由微软推出的免费、开源、跨平台的代码编辑器。

VS Code 支持 Windows、macOS 和 Linux，拥有强大的功能和灵活的扩展性。

VS Code 首次发布时间为 2015 年 4 月 29 日。

在 2019 年的 Stack Overflow 组织的开发者调查中，Visual Studio Code 被认为是最受开发者欢迎的开发环境。

许可证

VS Code 的源代码在 GitHub 上以 MIT 许可证发布，这意味着源代码是开源的，用户可以自由使用、复制、修改和分发源代码，只要遵循 MIT 许可证的条款。

VS Code 产品本身使用的是微软的软件许可证条款。这些条款规定了用户可以如何安装和使用 VS Code，包括用于开发和测试应用程序、在内部企业网络中部署、演示应用程序等。

发行版

- 2015 年 4 月 29 日，在 Build 2015 大会上公布开发计划并发布预览版本。
- 2015 年 11 月 18 日，开源并在 GitHub 上宣布支持扩展功能。
- 2016 年 4 月 14 日，发布正式版。
- ...
- 2024 年 11 月，版本号 1.96，可以免费使用 GitHub Copilot。
- ...

功能特点

Visual Studio Code (VS Code) 是一个功能强大的编辑器，它默认支持 JavaScript、TypeScript、CSS 和 HTML 等编程语言，并且可以通过安装扩展来支持 Python、C/C++、Java 和 Go 等更多语言。

VS Code 提供了丰富的编辑功能，包括语法高亮、括号补全、代码折叠和代码片段，对于某些语言还提供了 IntelliSense 智能感知功能来辅助开发。

此外，VS Code 还具备调试 Node.js 程序的能力。

VS Code 基于 Electron 框架构建，这使得它能够跨平台运行，并允许用户在不同操作系统上保持一致的开发体验。

VS Code 支持多目录同时打开，并能够将这些信息保存在工作区中，以使用户可以轻松地在不同项目间切换和复用设置。

作为一个跨平台的编辑器，VS Code 还允许用户根据需要更改文件的代码页、换行符和编程语言，以适应不同的开发环境和需求。

相关链接

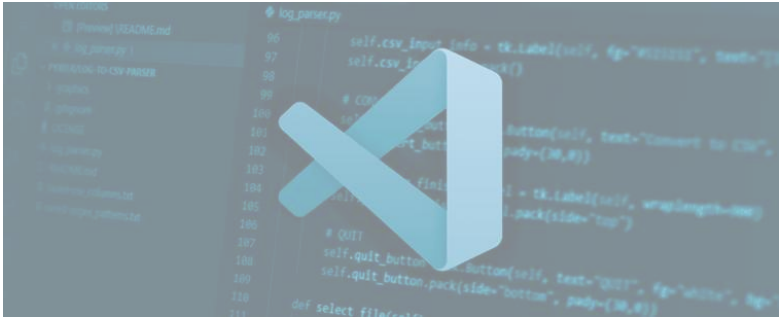
- VS Code 下载地址: <https://code.visualstudio.com/>
- VS Code 开源地址: <https://github.com/microsoft/vscode>
- VS Code 扩展: <https://marketplace.visualstudio.com/vscode>

VSCode 简介

Visual Studio Code (简称 VS Code) 是由微软开发的一个现代化、轻量级的代码编辑器，它支持几十种主流编程语言 (如 JavaScript、Python、C++、Java、Go 等)，并通过扩展提供更广泛的语言支持。

VSCode 可以在 Windows、macOS 和 Linux 操作系统上运行，为开发者提供了一个统一的开发环境。

VSCode 因其强大的功能和灵活性，已经成为全球开发者广泛使用的代码编辑器之一。



特点

- 轻量与高性能 - VS Code 的设计初衷是既轻量又强大，启动速度快、内存占用低，同时支持各种现代开发需求。
- 多语言支持 - 支持几十种主流编程语言（如 JavaScript、Python、C++、Java、Go 等），并通过扩展提供更广泛的语言支持。
- 强大的扩展系统 - 通过扩展市场，用户可以安装上千种插件（如代码美化工具、调试插件、Git 集成工具），自由定制开发环境。
- 丰富的内置功能 - 内置 Git 控制、调试工具、终端等，帮助开发者提升效率。
- 跨平台一致性 - 在不同操作系统上，提供一致的用户体验和功能支持。

为什么选择 VS Code？

- 免费开源：无需付费，没有功能限制，完全开源（基于 MIT 协议）。
- 灵活性：无论是前端开发、后端开发，还是 DevOps、数据科学，VS Code 都能胜任。
- 强大的社区支持：有庞大的开发者社区，持续维护和开发插件与功能。
- 高度可定制化：几乎所有功能都能通过配置文件调整，满足个性化需求。

适用场景

- 前端开发：支持 HTML/CSS/JavaScript、TypeScript，以及 React、Vue、Angular 等框架。
- 后端开发：支持 Node.js、Python、Java 等后端开发语言。
- 数据科学：可安装 Jupyter Notebook 插件，进行数据分析和可视化。
- DevOps：支持 YAML 配置、Docker、Kubernetes 等工具集成。
- 其他场景：例如文档编辑（Markdown）、静态网站生成、代码复审等。

对比其他编辑器的优势

VS Code 与其他主流编辑器对比：

编辑器	优势	劣势
VS Code	功能强大、扩展丰富、轻量灵活	需要一定配置时间
Atom	简洁、界面友好	性能较慢，社区逐渐萎缩
Sublime	速度快、占用资源低	付费软件，功能不够全面
IntelliJ	针对特定语言（如 Java）优化强	占用资源高，功能复杂

版本类型

以下是 Visual Studio Code 的主要版本类型及其特点对比：

版本名称	下载渠道	许可协议	主要特点	适合人群
官方版	微软官网	微软专有许可协议	- 官方支持 - 带有微软品牌和服务 - 内置遥测功能	需要稳定功能的普通用户和开发者
开源版（Code-OSS）	GitHub 源码	MIT 许可协议	- 完全开源 - 无微软品牌和遥测功能	开源爱好者或对隐私有要求的用户

VSCode 教程

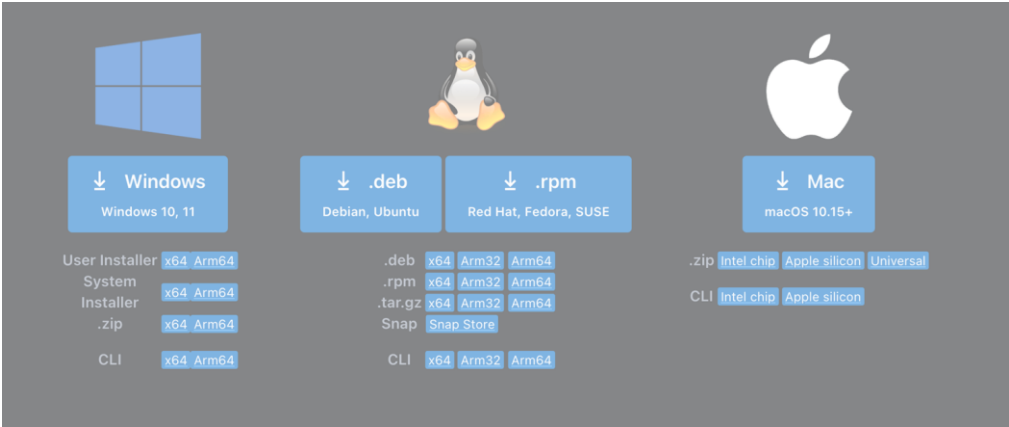
			- 需要手动构建	
VSCodium	VSCodium 官网	MIT 许可协议	- 基于 Code-OSS 构建 - 去掉微软遥测功能 - 即装即用	需要无遥测的开源版本的用户
Insiders 版	VS Code Insiders	微软专有许可协议	- 最新特性预览 - 每日构建，可能存在不稳定性	喜欢尝鲜的开发者，测试版功能用户
社区构建版	各开源社区提供	各社区指定协议	- 根据 Code-OSS 修改 - 添加社区特色功能	需要特定功能或自定义版本的用户

VSCode Windows 安装

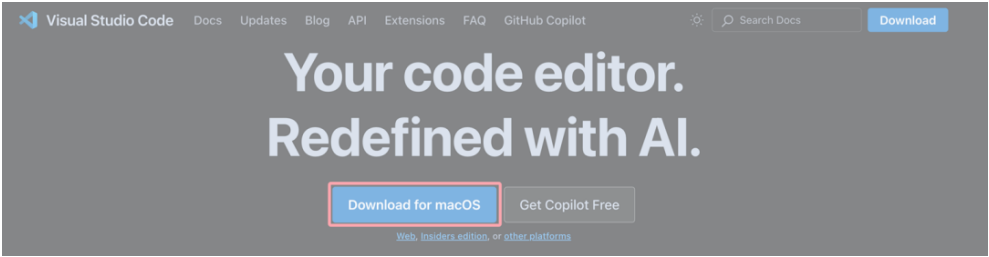
VSCode 是微软开发的跨平台免费源代码编辑器，支持 Windows、macOS 和 Linux。
在安装 VS Code 之前，请确保您的设备满足以下最低要求：

操作系统	最低要求
Windows	Windows 7 64 位或更高版本
macOS	macOS 10.11 El Capitan 或更高版本
Linux	Ubuntu 16.04+, Debian 9+, Fedora 30+, CentOS 7+

VS Code 官方网站下载页面：<https://code.visualstudio.com/Download>。

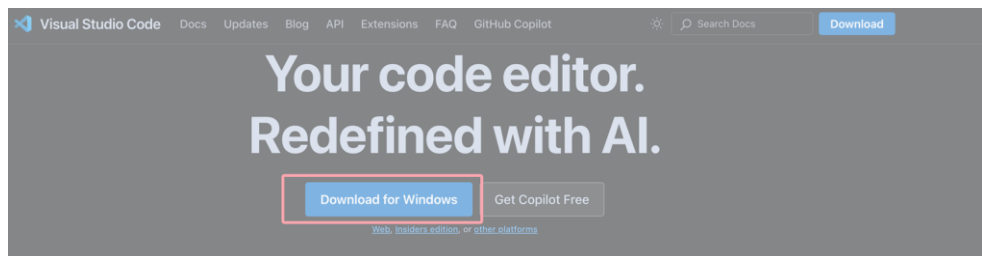


Windows 系统注意下载正确的版本，如果系统账户是 Administrator，需要下载 System Installer 版本，如果系统账户是其它自定义账户，需要下载 User Installer 版本。
默认情况下访问 VS Code 官网 <https://code.visualstudio.com/>，页面会根据你的系统自动匹配安装包，比如我是 macOS，就会出现 Download for macOS 按钮：

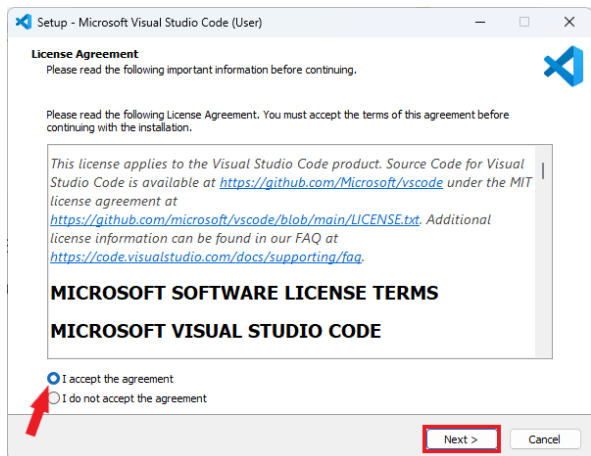


在 Windows 上安装

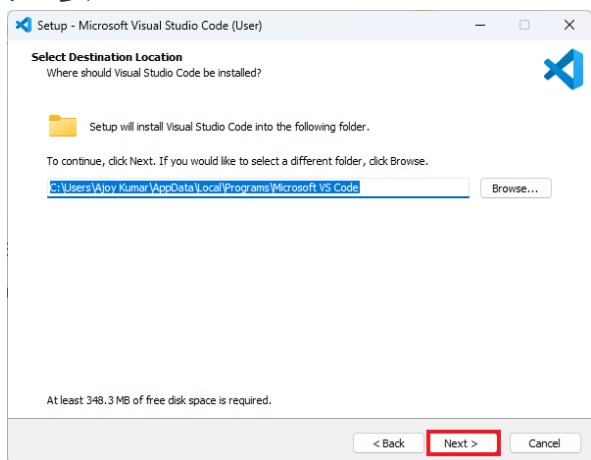
访问 VS Code 官网 <https://code.visualstudio.com/>，点击 "Download for Windows" 下载适配的安装包，默认会下载稳定版（Stable）。



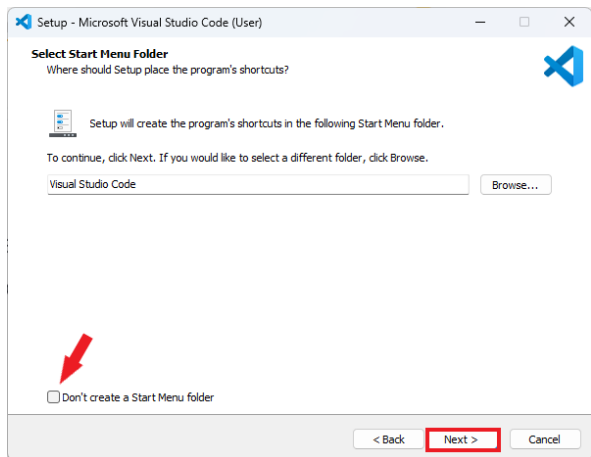
下载的是一个类似 `VSCodeUserSetup-{version}.exe` 安装程序，双击下载的 `.exe` 文件。安装程序打开后，会要求你接受 Visual Studio Code 的条款和条件，点击 "I accept the agreement (我接受协议)"，然后点击 "Next (下一步)"。



选择安装位置，默认情况下 VS Code 会安装在以下目录 `C:\Users\{Username}\AppData\Local\Programs\Microsoft VS Code`，Username 为你的用户名，没特别要求按默认的来，点击 "Next (下一步)"。



接下来是设置一些开始菜单的目录，按默认就好了，点击 "Next (下一步)"。

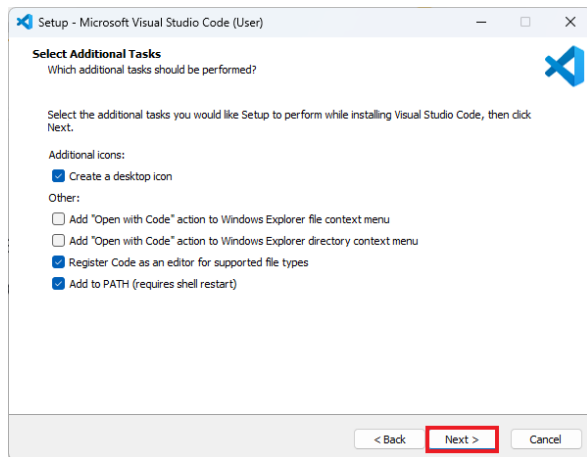


接下来，可以勾选以下选项（推荐）：

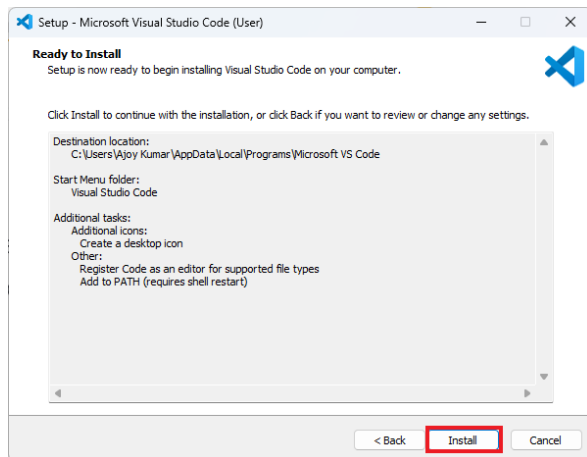
- 在桌面创建快捷方式。

VSCode 教程

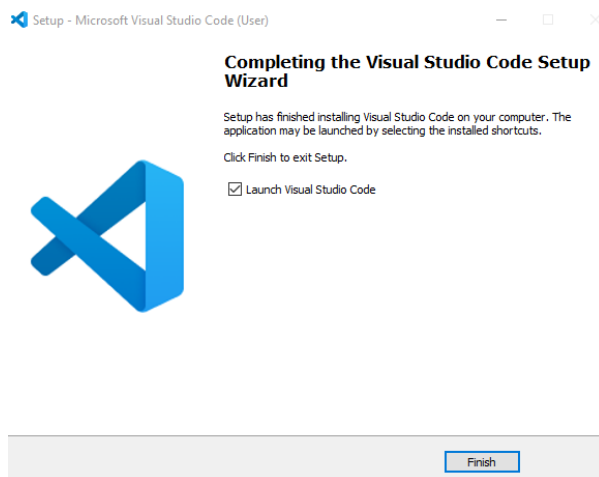
- 将 VS Code 添加到右键菜单中（方便直接用 VS Code 打开文件）。
- 将 VS Code 添加到 PATH 环境变量（方便在终端中运行 code 命令）。



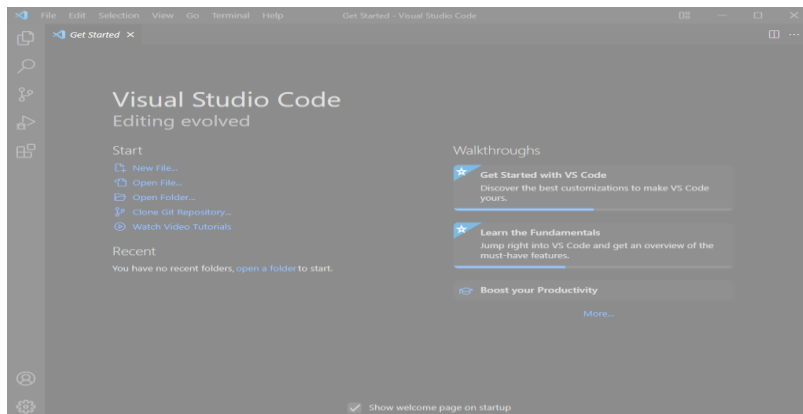
点击 "Install (安装)" 按钮，等待完成后启动 VS Code。



点击 "Finish (完成)" 按钮完成安装：

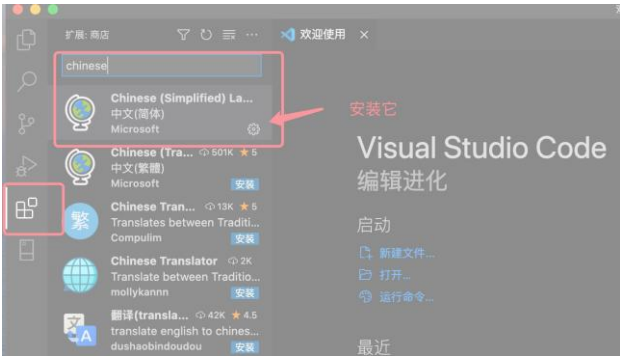


启动 VS Code ，界面如下所示：



安装汉化包

VScode 安装汉化包很简单，打开 VScode，点击左侧安装扩展图标，在搜索框输入 **Chinese**：



然后点击第一个搜索出来选项【Chinese (Simplified) (简体中文)】的 Install 按钮就可以：



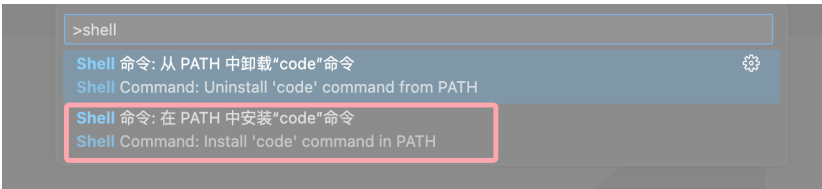
安装完成后，重启 VSCode，界面显示的就是中文了。

VSCode 的 code 命令

启用 VSCode 的 code 命令 非常简单，先打开命令面板：

- macOS 系统快捷键：⌘⇧P
- Windows/Linux 快捷键：Ctrl + Shift + P

搜索安装 >shell command:



然后选择 **Shell Command: Install 'code' command in PATH** 即可为系统 PATH 路径添加了 **code** 命令的引用。

Vscode Code 命令详细参考：[VSCode code 命令](#)

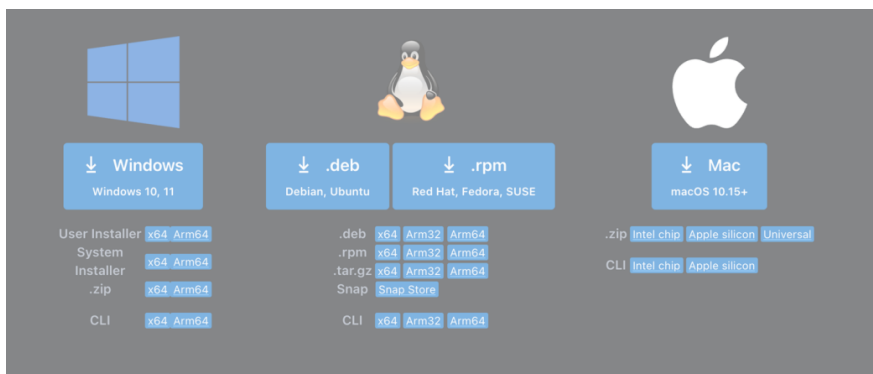
VSCode macOS 安装

VSCode 是微软开发的跨平台免费源代码编辑器，支持 Windows、macOS 和 Linux。
在安装 VS Code 之前，请确保您的设备满足以下最低要求：

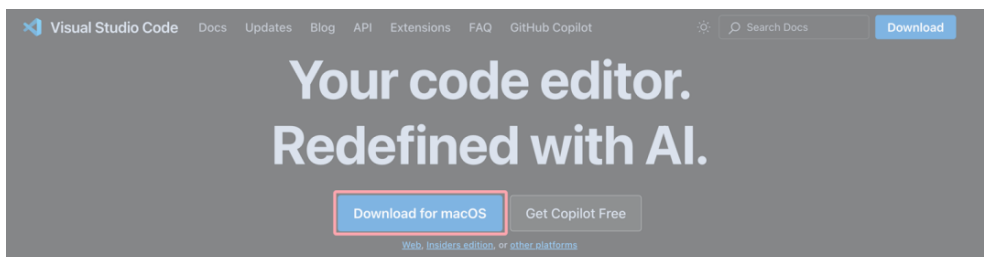
操作系统	最低要求
Windows	Windows 7 64 位或更高版本
macOS	macOS 10.11 El Capitan 或更高版本
Linux	Ubuntu 16.04+, Debian 9+, Fedora 30+, CentOS 7+

VS Code 官方网站下载页面：<https://code.visualstudio.com/Download>。

VSCode 教程

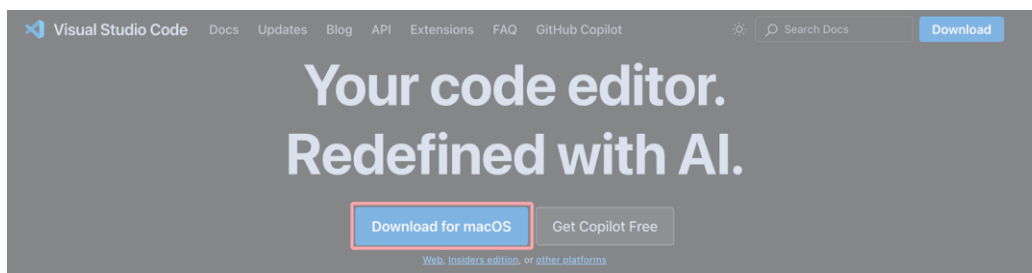


默认情况下访问 VS Code 官网 <https://code.visualstudio.com/>，页面会根据你的系统自动匹配安装包，比如我是 macOS，就会出现 **Download for macOS** 按钮：

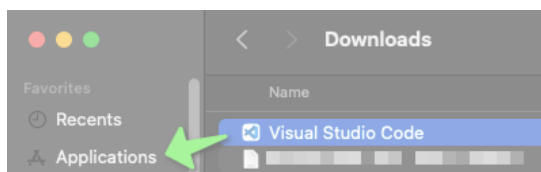


在 macOS 上安装

访问 VS Code 官网 <https://code.visualstudio.com/>，点击 "Download for macOS"，会下载一个 VSCode-darwin-universal.zip 的压缩包，解压后是一个 Visual Studio Code。



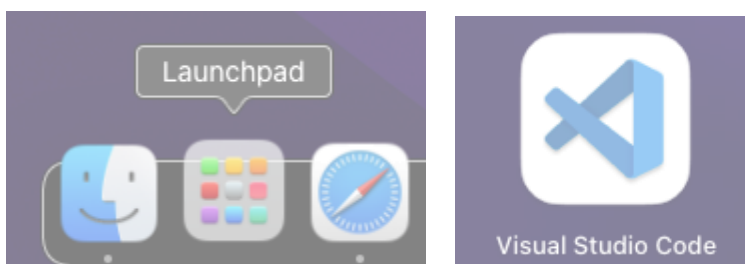
将 Visual Studio Code.app 拖到 "Applications (应用程序)" 文件夹中，使其在 macOS Launchpad (启动台) 中可用。



按下 **Command**  按键，然后按下空格键，在搜索框查找 'Visual'，点击 Visual Studio Code 按钮即可打开：

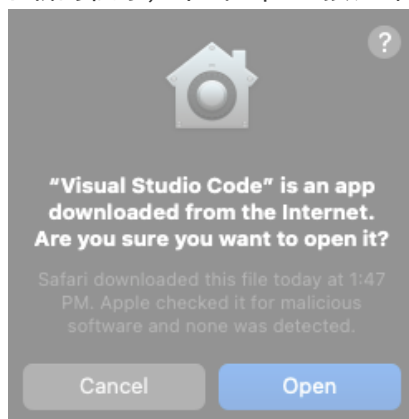


也可以在启动台查找 Visual Studio Code 图标，找到 Visual Studio Code 图标后，单击它：

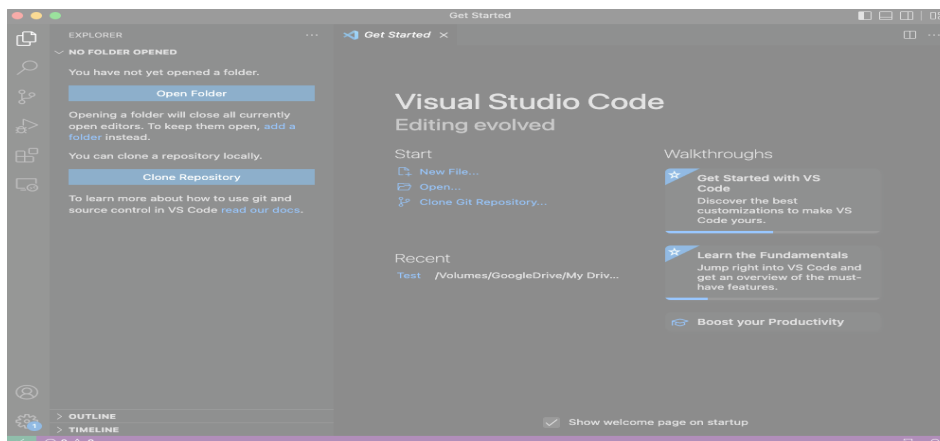


VSCode 教程

Mac OS 将提示是否确认打开从互联网下载的程序，单击 Open 按钮即可：



启动 Visual Studio Code，界面如下：

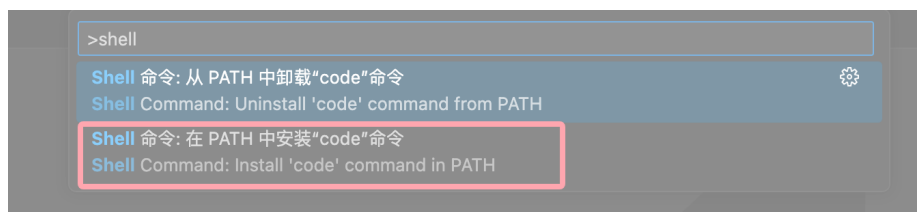


VSCode 的 code 命令

启用 VSCode 的 code 命令 非常简单，先打开命令面板：

- macOS 系统快捷键：`⇧⌘P`
- Windows/Linux 快捷键：`Ctrl + Shift + P`

搜索安装 `>shell command`：



然后选择 `Shell Command: Install 'code' command in PATH` 即可为系统 PATH 路径添加了 `code` 命令的引用。

VSCode Linux 安装

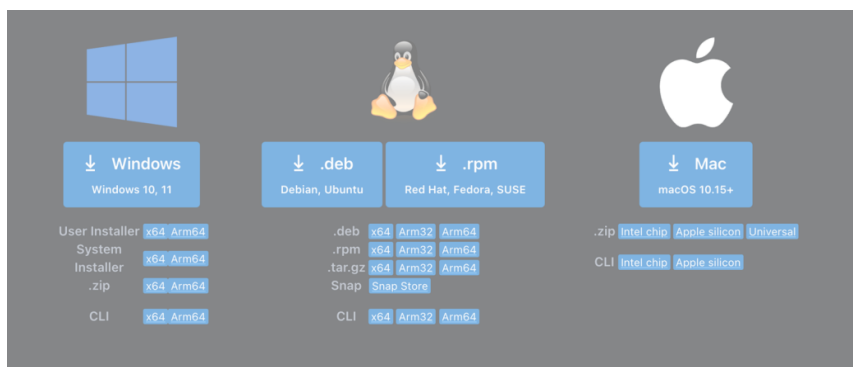
VSCode 是微软开发的跨平台免费源代码编辑器，支持 Windows、macOS 和 Linux。

在安装 VS Code 之前，请确保您的设备满足以下最低要求：

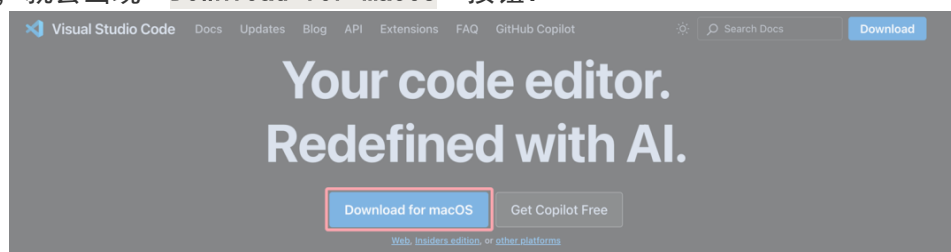
操作系统	最低要求
Windows	Windows 7 64 位或更高版本
macOS	macOS 10.11 El Capitan 或更高版本
Linux	Ubuntu 16.04+, Debian 9+, Fedora 30+, CentOS 7+

VS Code 官方网站下载页面：<https://code.visualstudio.com/Download>。

VSCode 教程

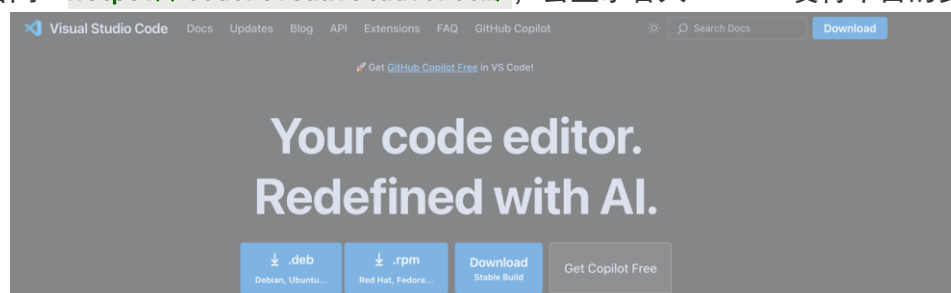


默认情况下访问 VS Code 官网 <https://code.visualstudio.com/>，页面会根据你的系统自动匹配安装包，比如我是 macOS，就会出现 **Download for macOS** 按钮：



在 Linux 上安装

访问 VS Code 官网 <https://code.visualstudio.com/>，会显示各大 Linux 发行平台的安装包。



使用包管理器安装（推荐）

对于基于 Debian 的系统（如 Ubuntu）：

```
sudo apt update
sudo apt install software-properties-common apt-transport-https
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor > microsoft.gpg
sudo install -o root -g root -m 644 microsoft.gpg /usr/share/keyrings/
sudo sh -c 'echo "deb [arch=amd64 signed-by=/usr/share/keyrings/microsoft.gpg]
https://packages.microsoft.com/repos/vscode stable main" > /etc/apt/sources.list.d/vscode.list'
sudo apt update
sudo apt install code
```

对于基于 Red Hat 的系统（如 Fedora）：

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
sudo sh -c 'echo -e "[code]\nname=Visual Studio
Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenabled=1\nngpgcheck=1\nngpgkey=htt
ps://packages.microsoft.com/keys/microsoft.asc" > /etc/yum.repos.d/vscode.repo'
sudo dnf check-update
sudo dnf install code
```

直接下载二进制包安装

从 VS Code 官网 下载适配的 .deb 或 .rpm 文件。

对于 .deb 文件：

```
sudo dpkg -i code*.deb
sudo apt-get install -f
```

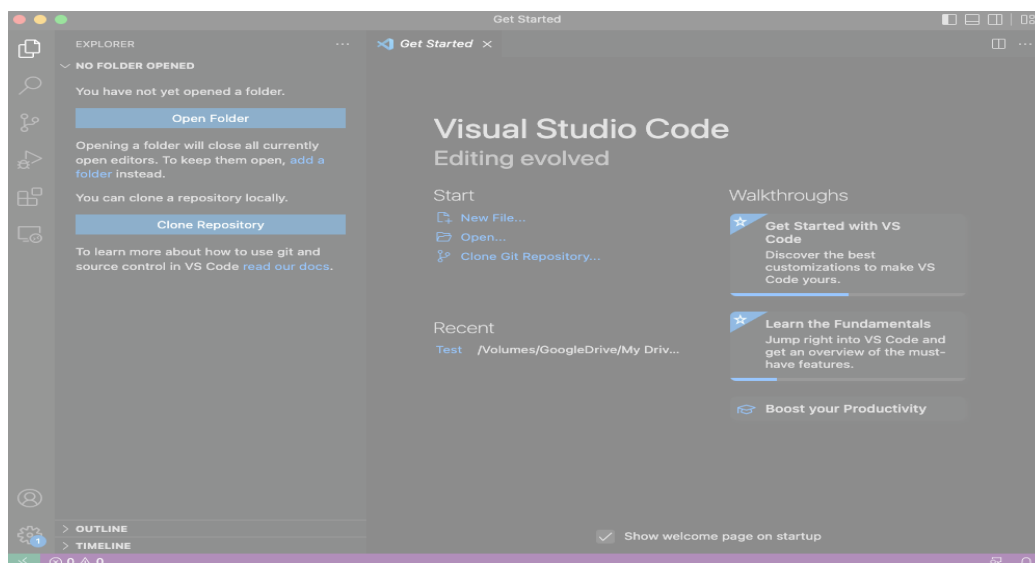
VSCode 教程

对于 .rpm 文件：

```
sudo rpm -i code*.rpm
```

运行 VS Code

在终端输入 `code` 启动 VS Code，或者通过系统菜单打开。

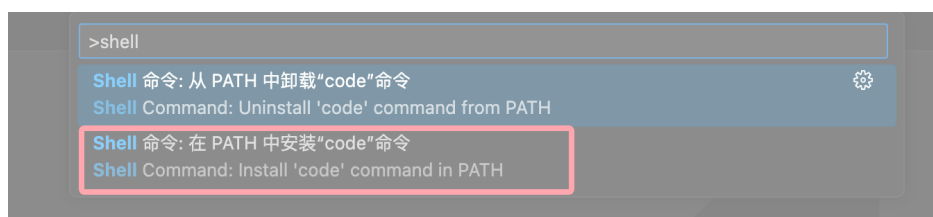


VSCode 的 code 命令

启用 VSCode 的 `code` 命令非常简单，先打开命令面板：

- macOS 系统快捷键：`⇧⌘P`
- Windows/Linux 快捷键：`Ctrl + Shift + P`

搜索安装 `>shell command`：



然后选择 `Shell Command: Install 'code' command in PATH` 即可为系统 PATH 路径添加了 `code` 命令的引用。

VS Code AI 编程

VS Code 包含了很多 AI 编程的扩展，本文将介绍如何在 VS Code 中安装 **TONGYI Lingma**（通义灵码）扩展。

TONGYI Lingma（通义灵码）是阿里云推出的一款智能编程助手，基于通义大模型开发，旨在帮助开发者提高编程效率。

通义灵码支持多种编程语言，能够提供代码补全、代码解释、代码优化、智能问答等功能。

官方说明参见：<https://lingma.aliyun.com/>。

1、安装

打开 Visual Studio Code 扩展窗口，搜索 **TONGYI Lingma**，找到通义灵码后单击安装。

安装完成后，请重启 Visual Studio Code。

登录并开启智能编码之旅

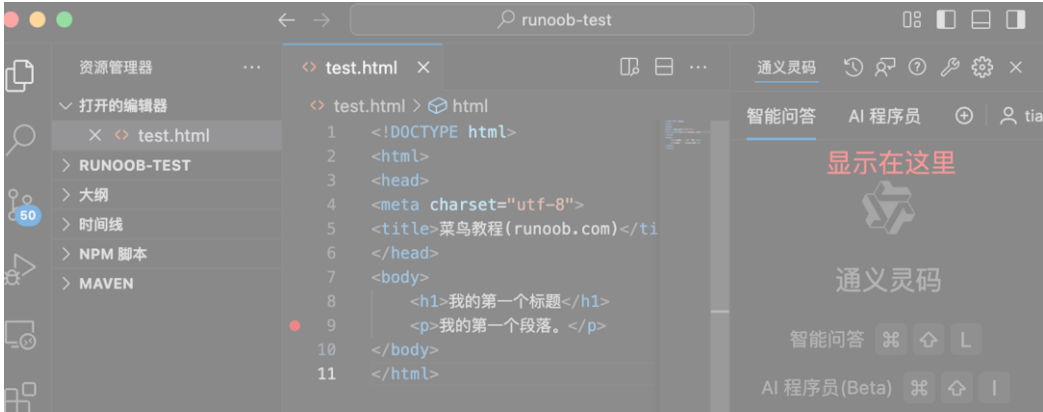
重启 Visual Studio Code 后，单击侧边导航的通义灵码，在通义灵码助手的窗口单击登录按钮。

提示：如果安装后在侧边导航上找不到通义灵码入口，可鼠标聚焦在侧边导航后右键查看，勾选通义灵码后即可将插件入口配置在侧边导航上。

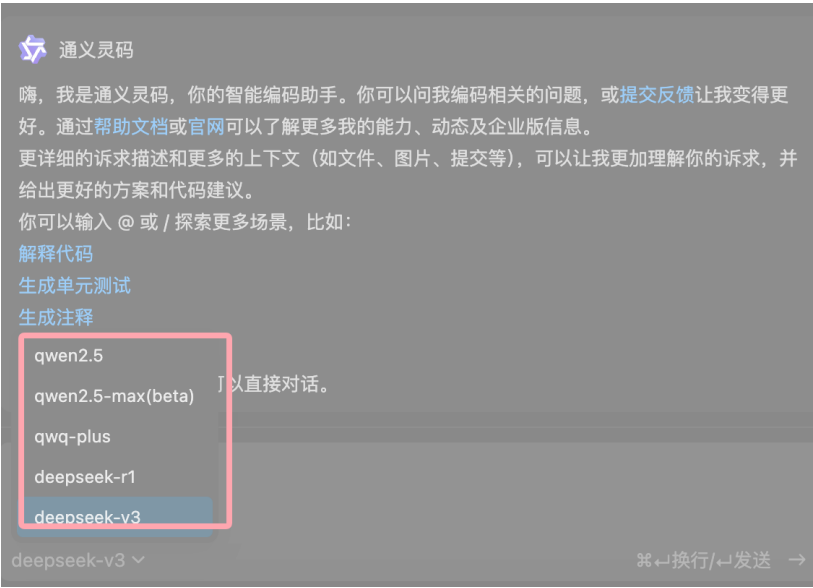
单击登录后，将前往登录页面，完成登录后可进入 IDE 客户端开始使用。

登录相关具体操作，可参考：[登录通义灵码插件端](#)。

我们还可以按住扩展图标按钮，直接拖动到右侧，把 AI 编程功能放到右边，如下所示：

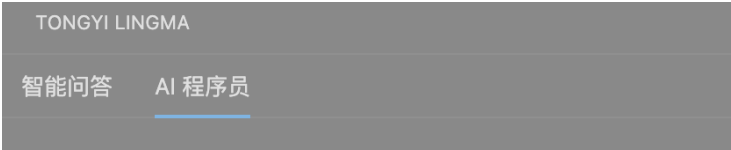


通义灵码支持多种模型的选择，目前已经支持 Qwen2.5、DeepSeek-V3 和 R1 系列模型：
在 VSCode 输入框里选择模型，即可轻松切换模型。



2、功能展示

通义灵码主要有两个角色：**智能问答**与 **AI 程序员**。



角色	功能描述
智能助手	基本的对话功能，可以提问，优化代码、生成单元测试等。
AI 程序员	具备多文件代码修改（Multi-file Edit）和工具使用（Tool-use）的能力，可以与开发者协同完成编码任务，如需求实现、问题解决、单元测试用例生成、批量代码修改等。

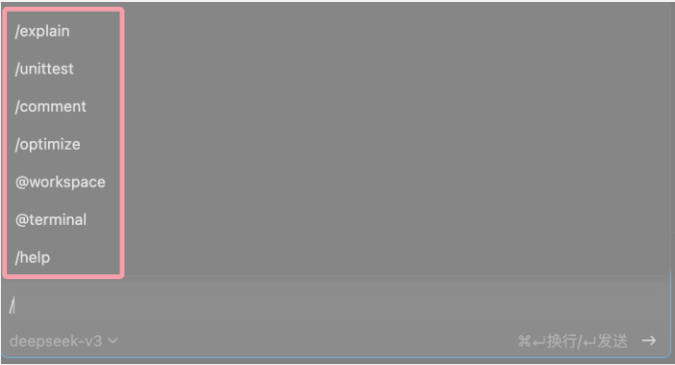
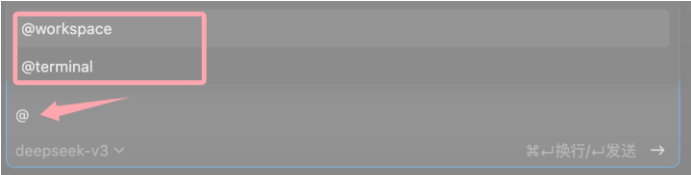
VSCode 教程

- 1、新建会话
- 2、查看会话历史
- 3、可以自己提问
- 4、选择指定代码问答
- 5、代码中的快捷入口
- 6、@workspace 本地工程问答
- 7、@terminal 问答
- 8、/ 查看快捷指令
- 9、AI 程序员

@ 与 /

另外，我们可以在文本框通过输入 @ 或 / 探索更多场景，比如：

- 解释代码
- 生成单元测试
- 生成注释
- 优化代码

符号	功能描述
/	命令提示符，用于输入命令或指令。例如：/explain、/unittest、/comment 等，可以执行特定的操作或功能。 
@	快捷方式提示符，用于快速访问特定的功能或命令。例如：@workspace、@terminal，可以快速打开工作区或终端。 

更多操作参考：

- 1、完整文档
- 2、智能补全使用指南
- 3、智能问答使用指南
- 4、AI 程序员使用指南
- 5、配置通义灵码说明

VSCode 界面说明

1、启动页面

在完成 VSCode 安装后，点击 VSCode 图标，启动界面如下所示：



上图展示了 Visual Studio Code (VS Code) 的欢迎界面，它是一个代码编辑器的起始页，提供了快速访问不同功能和资源的入口。

以下是图片中各个部分的说明：

活动栏 (Activity Bar)：

- 位于左侧，包含多个图标，每个图标代表 VS Code 的一个功能视图。
- 图片中用红色箭头标注了四个功能：
 - **文档管理**：可能是指资源管理器，用于管理文件和文件夹。
 - **搜索**：用于在整个工作区中搜索文件或文本。
 - **版本控制**：通常与 Git 集成，用于代码版本管理。
 - **debug 调试**：用于启动调试会话，进行代码调试。

启动 (Start)：

- 位于活动栏上方，包含常用功能的快捷方式。
- 图片中用红色方框标注了三个选项：
 - **新建文件**：创建一个新的文件。
 - **打开...**：打开一个文件或文件夹。
 - **运行命令...**：使用命令面板执行命令。

最近打开的目录 (最近)：

- 显示了最近打开的文件夹列表，方便快速切换到之前的工作目录。
- 图片中用红色方框标注了最近打开的目录，例如 "runoob-test"。

演练 (Tutorial)：

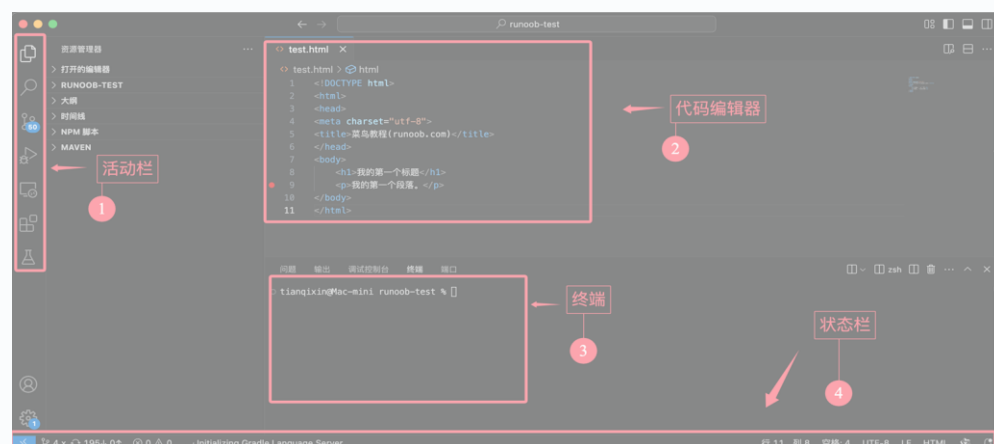
- 位于欢迎界面的右侧，提供了一些快速链接，帮助用户了解和使用 VS Code。
- 图片中用红色方框标注了三个链接：
 - **开始使用 VS Code**：提供自定义方法和使用专属 VS Code 的指导。
 - **了解基础知识**：链接到 VS Code 的基础知识，帮助用户了解必备功能。
 - **提高工作效率**：提供提高使用 VS Code 效率的建议。

vscode 文档：

- 位于演练部分的上方，可能是指向 VS Code 官方文档的链接。

2、编辑页面

打开或编辑一个代码的界面如下所示：



VSCode 教程

活动栏与侧边栏

界面左侧，通过图标切换不同的功能模块：



- **资源管理器（第一个图标）**：在图片中显示了项目文件结构，其中包含：
 - RUNOOB-TEST 文件夹：当前工作区的项目文件夹。
 - test.html 文件：当前正在编辑的 HTML 文件。
- **搜索（第二个图标）**：用于全局搜索文件内容（未激活）。
- **源代码管理（第三个图标）**：显示 Git 状态，方便管理代码版本。
- **运行和调试（第四个图标）**：可以设置断点并运行代码。
- **扩展（第五个图标）**：用于安装和管理插件。

紧邻活动栏右侧，展示当前功能模块的具体内容：

- 当前显示的是 **资源管理器**。
- 项目结构清晰展示，包括：
 - RUNOOB-TEST 文件夹中的所有文件。
 - 当前正在编辑的文件 test.html。

编辑区

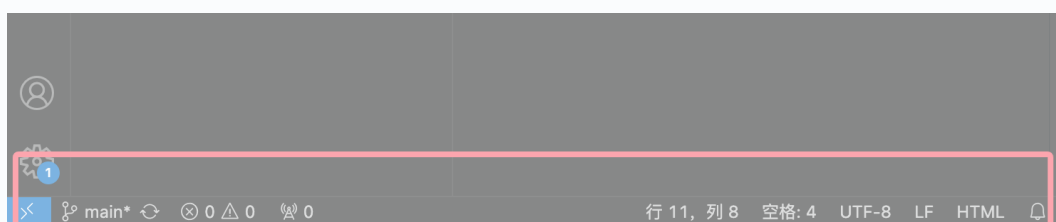
位于界面中间，用于显示和编辑文件内容：



- 编辑区域显示 test.html 文件的代码。
- 支持语法高亮（如 DOCTYPE html 和 <h1> 标签）。
- 光标定位在第 11 行（底部状态栏显示“行 11”）。

状态栏

位于底部，显示当前文件和编辑环境的信息：



- **Git 状态**：当前分支为 main（左下角）。

VSCode 教程

- 文件信息：
 - 编码格式：UTF-8。
 - 行尾序列：LF。
 - 当前文件类型：HTML。
- 终端状态：显示当前打开的终端类型为 zsh。

VS Code 活动栏

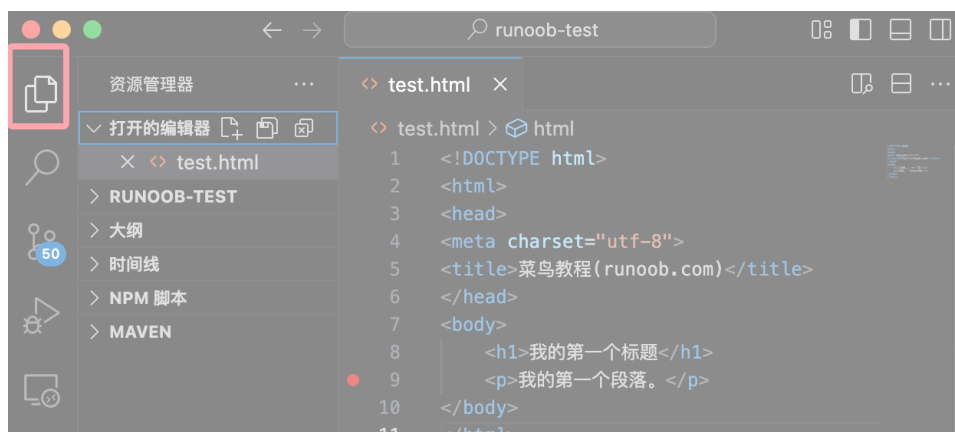
活动栏（Activity Bar）是 VS Code 左侧的一个垂直工具栏，它提供了快速访问不同功能和视图的入口。通过活动栏，用户可以轻松切换文件资源管理器、搜索、源代码管理、调试、扩展等功能。



活动栏的主要功能

1. 文件资源管理器（Explorer）

文件资源管理器是活动栏中的第一个图标，通常是一个文件夹的图标，点击后，它会显示当前工作区中的所有文件和文件夹。你可以在这里浏览、打开、创建、删除或重命名文件。



2. 搜索（Search）

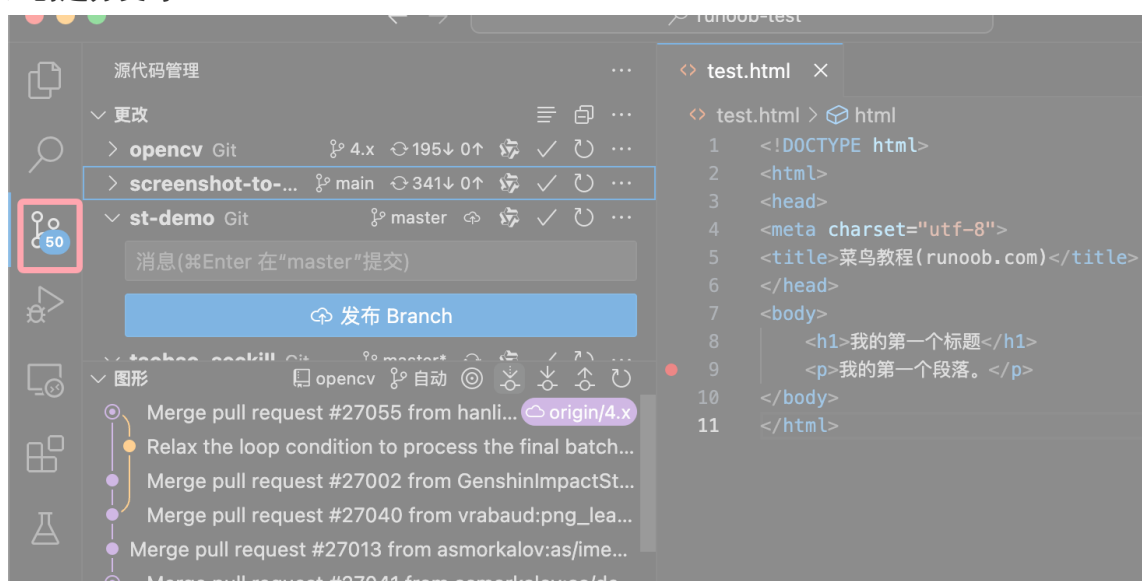
搜索功能允许你在整个项目中查找特定的文本或代码片段，点击搜索图标（放大镜）后，你可以输入关键词，VS Code 会在项目中匹配并显示所有相关结果。



可以在输入框中设置文件类型以及要排除的文件。

3. 源代码管理 (Source Control)

源代码管理功能集成了 Git，允许你在 VS Code 中直接进行版本控制操作，我们可以查看文件的更改状态、提交代码、创建分支等。



4. 调试 (Debug)

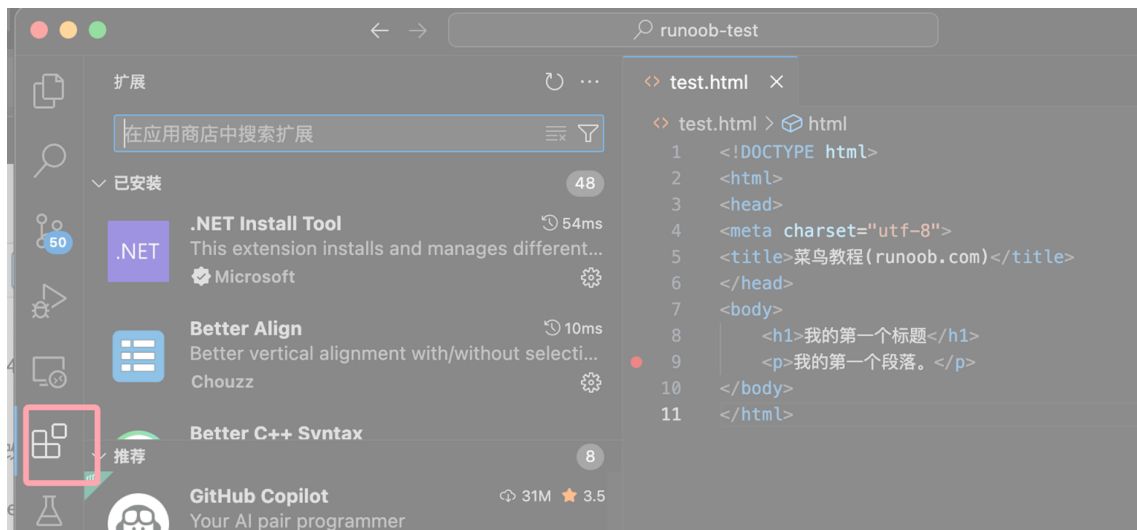
调试功能是 VS Code 中非常强大的工具之一，我们可以设置断点、逐步执行代码、查看变量值等，帮助你快速定位和修复代码中的问题。



5. 扩展 (Extensions)

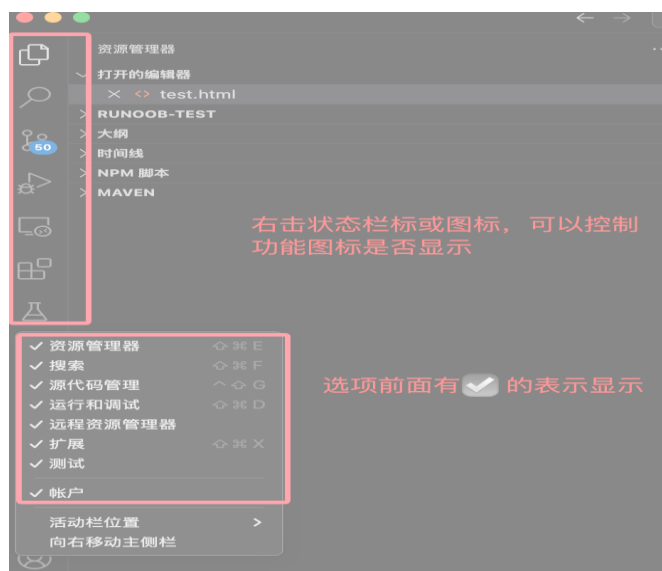
扩展功能允许你安装和管理 VS Code 的插件。

通过扩展，我们可以为 VS Code 添加新的语言支持、主题、工具等功能，极大地扩展了编辑器的功能。

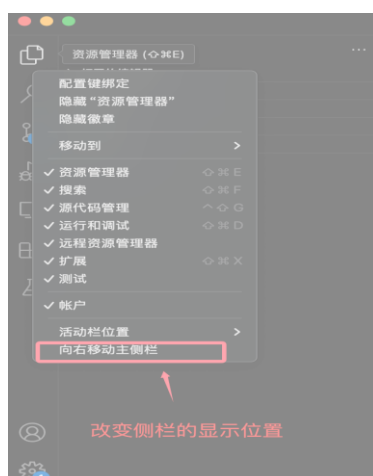


自定义活动栏

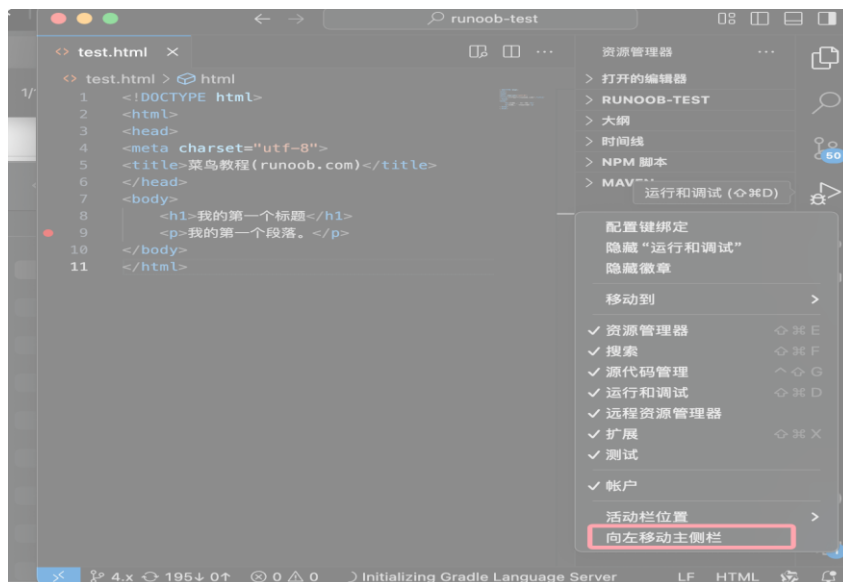
VS Code 允许用户自定义活动栏的显示内容，可以右击状态栏，控制功能图标是否显示，打勾☑的表示显示，反之，不显示：



也可以设置状态栏显示在右侧：



不习惯右侧右击活动栏，可以调回来：



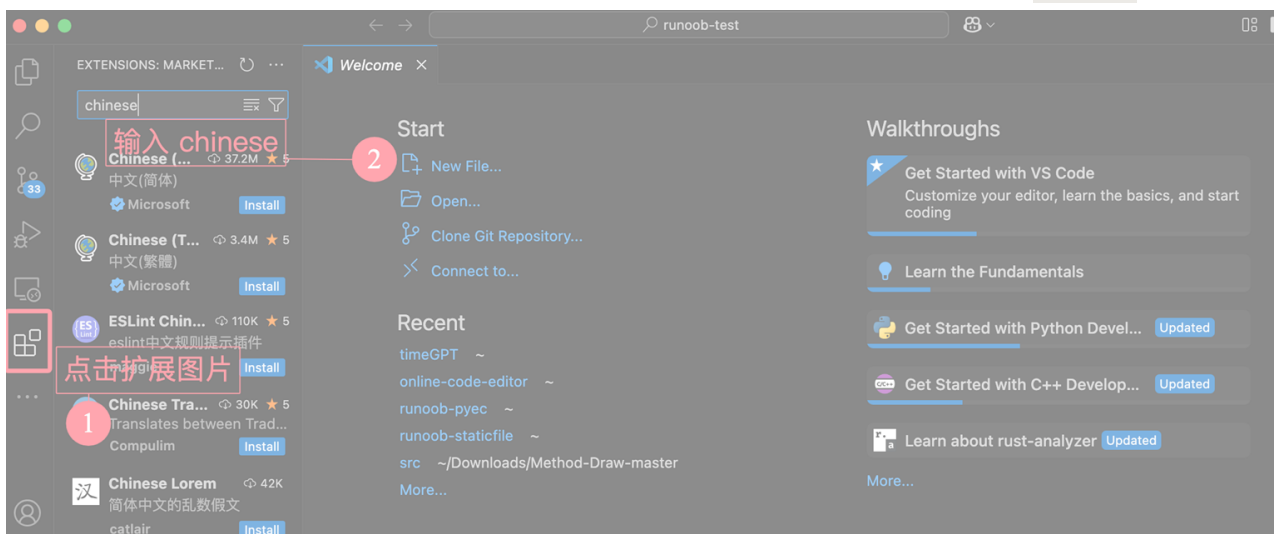
VSCode 中文设置

默认 VS Code 的界面上英文的，我们可以通过 VS Code 的扩展市场安装中文包。

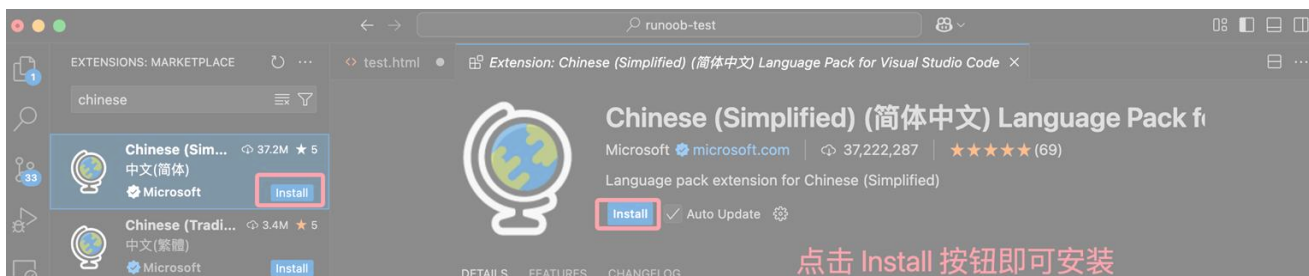
VS Code 有一个内置的扩展市场，提供了数千个由社区和微软官方提供的扩展。

接下来我们就在扩展市场中查找中文包，并安装。

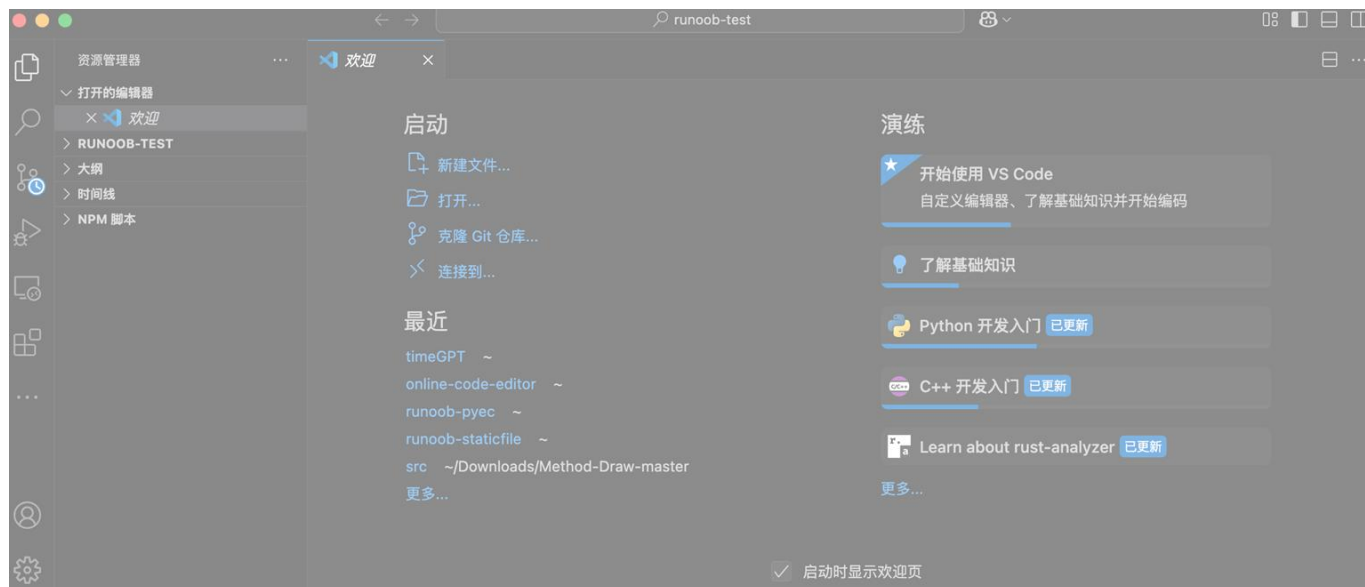
VScode 安装汉化包很简单，打开 VScode，点击左侧安装扩展图标，在搜索框输入 **Chinese**：



然后点击第一个搜索出来选项【Chinese (Simplified) (简体中文)】的 Install 按钮就可以：



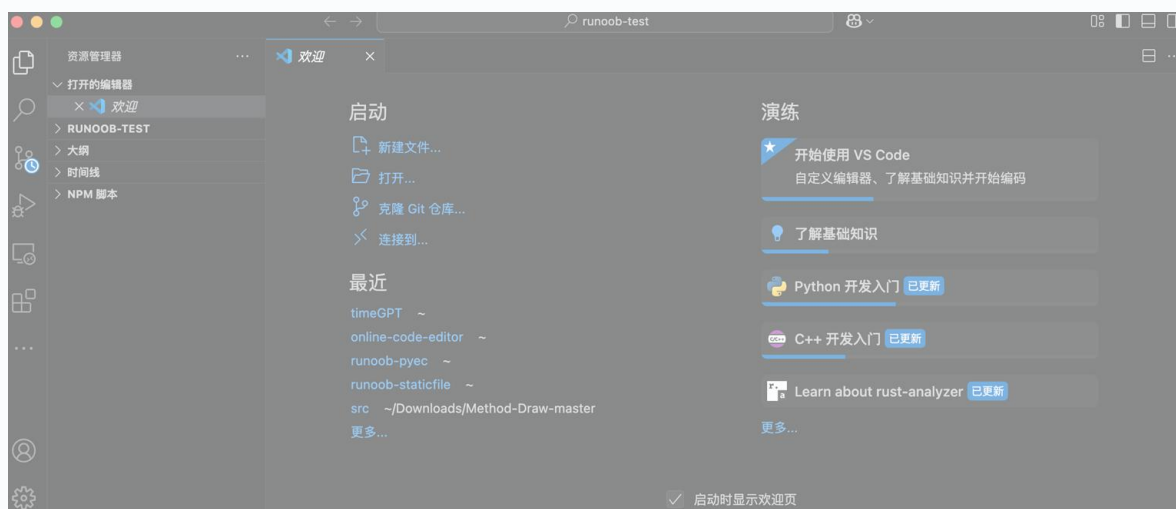
安装完成后，重启 VSCode，界面显示的就是中文了。



VSCode 打开目录

我们可以在 VS Code 中打开一个代码文件，也可以在在 VS Code 中打开一个目录（文件夹）是一个简单的过程，操作都很简单。

首先，我们打开已经安装好的 VS Code，在 VS Code 首次打开时，你会看到欢迎页面，上面有不同的操作来开始使用。



可以 **新建文件...** 或 **打开...** 选项：

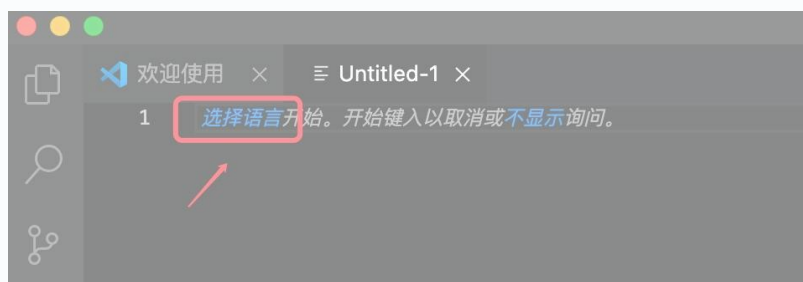


创建一个 Python 代码文件

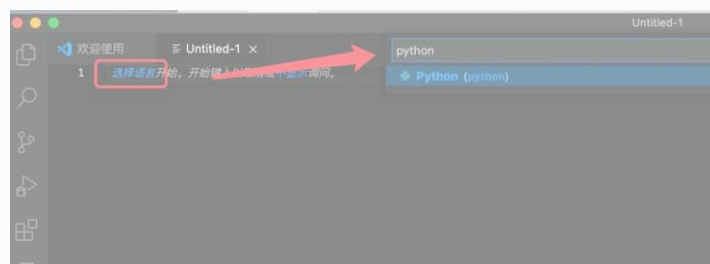
打开 VSCode，然后点击新建文件：



点击选择语言：



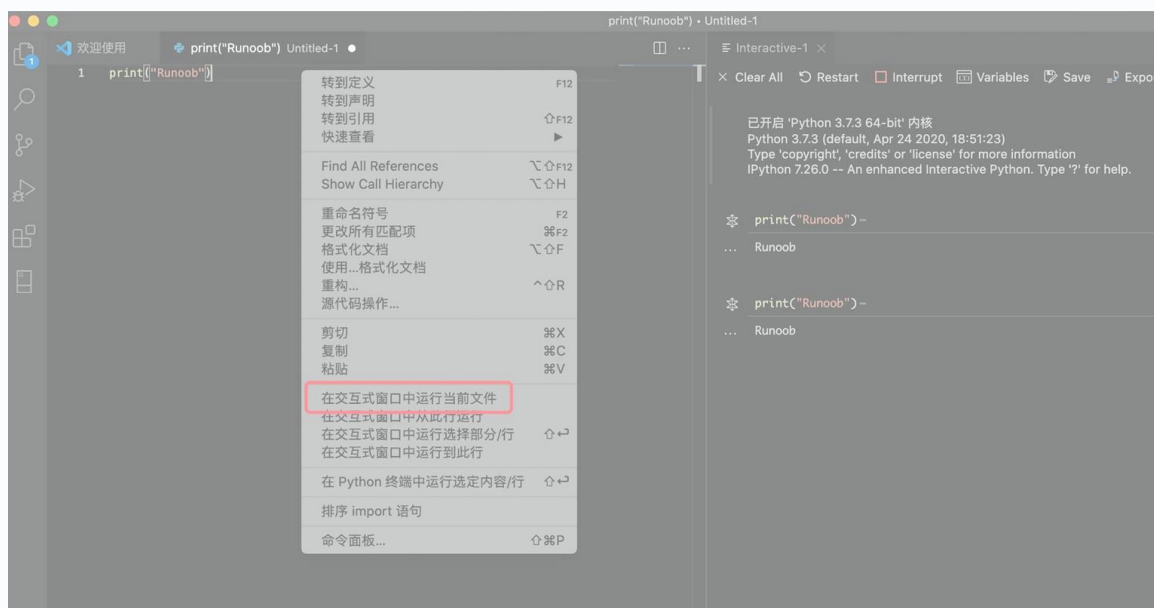
在搜索框输入 Python，选中 Python 选项：



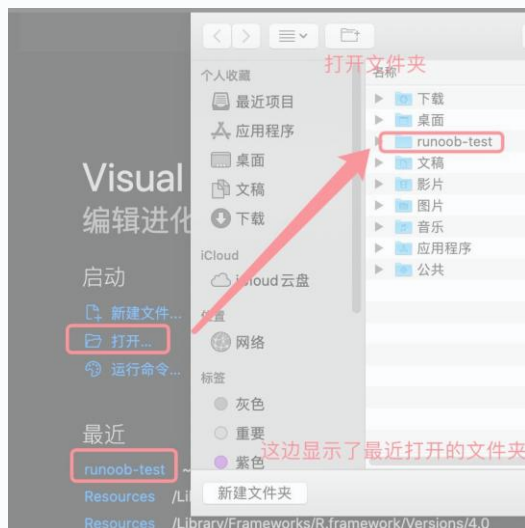
输入代码：

```
print("Runoob")
```

右击鼠标，选择在交互式窗口运行文件，如果有提示需要安装扩展，直接点安装即可（没有安装会一直显示在连接 Python 内核）：



另外，我们也可以打开一个已存在的文件或目录（文件夹），比如我们打开一个 runoob-test，你也可以自己创建一个：



然后我们创建一个 `test.py` 文件，点击下面新建文件图标，输入文件名 `test.py`：



注：runoob-test 里面包含了一个 `.vscode` 文件夹，是一些配置信息，可以先不用管。

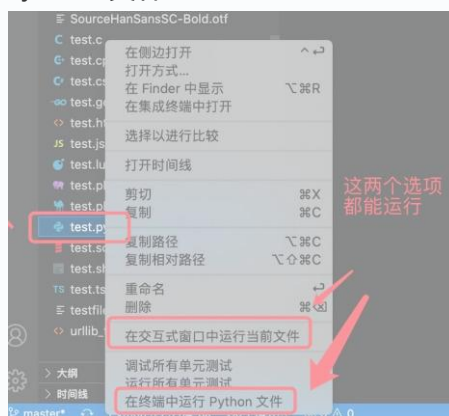
在 `test.py` 输入以下代码：

```
print("Runoob")
```

点击右上角绿色图标，即可运行：



可以右击文件，选择"在终端中运行 Python 文件"：

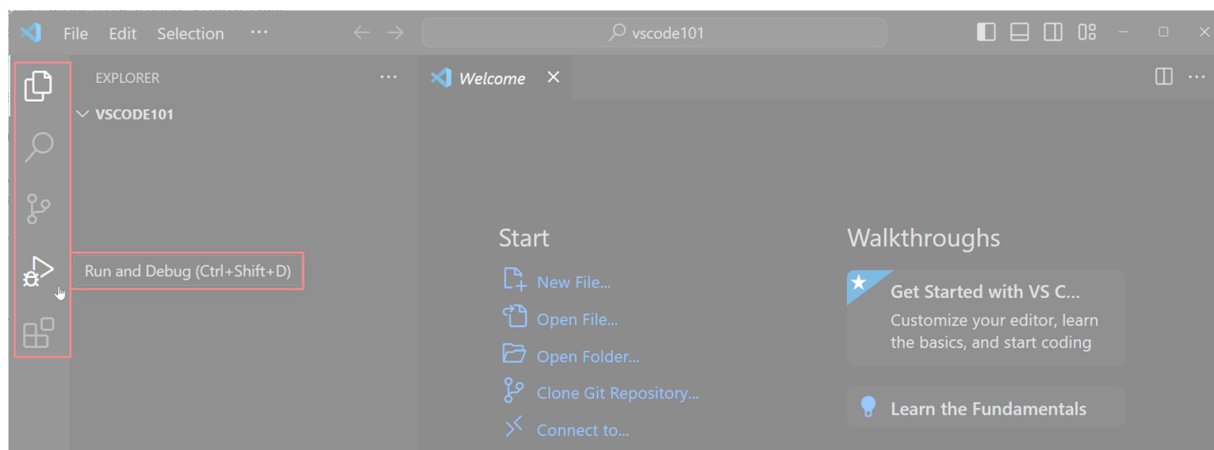


当然也可以在代码窗口上右击鼠标，选择"在终端中运行 Python 文件"。

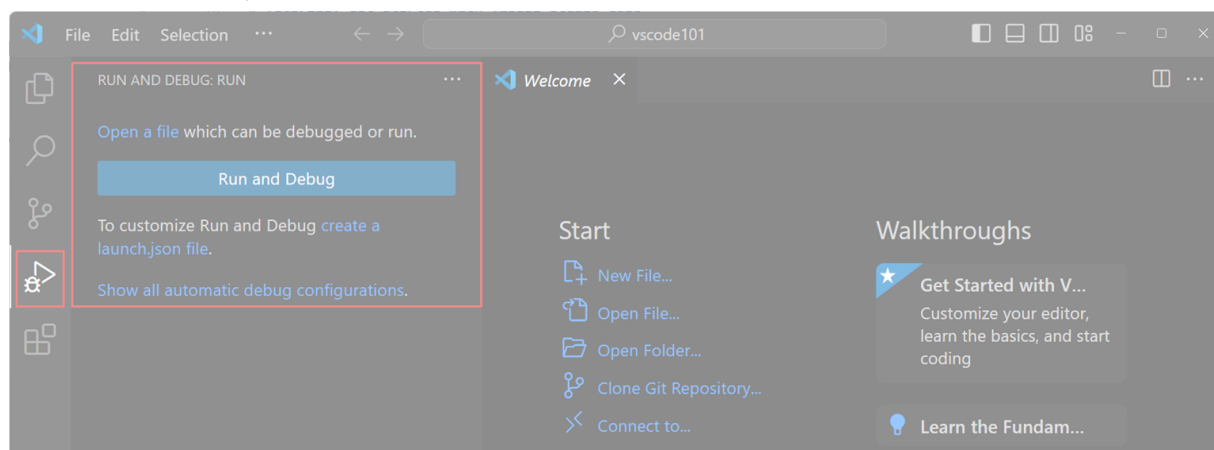
VSCode 新建文件

活动栏（Activity Bar）用于切换不同的视图模块，例如资源管理器、搜索、源代码管理、运行和调试等。将鼠标悬停在活动栏的图标上，可以看到每个视图的名称和对应的快捷键提示。

VSCode 教程

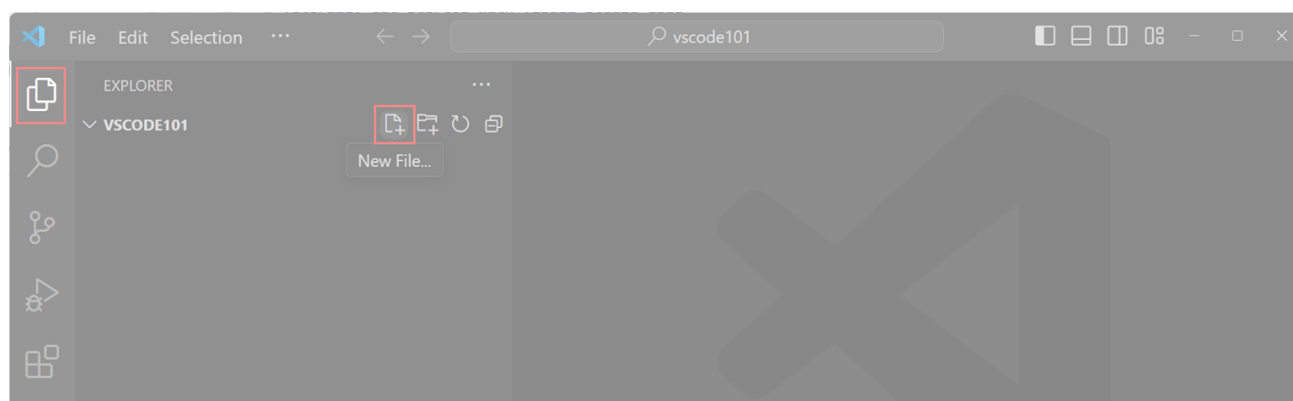


提示：再次点击视图图标，或者按下快捷键，可以打开或关闭该视图。
当你选择活动栏中的某个视图时，主侧边栏会打开并显示该视图的具体信息。
选择"运行和调试"视图后，主侧边栏将显示配置调试会话的选项：



使用编辑器查看和编辑文件

在活动栏中选择"资源管理器（Explorer）"视图。
点击"新建文件（New File...）"按钮，在工作区中新建一个文件。



输入文件名为 index.html，然后按回车键确认。



VSCode 教程

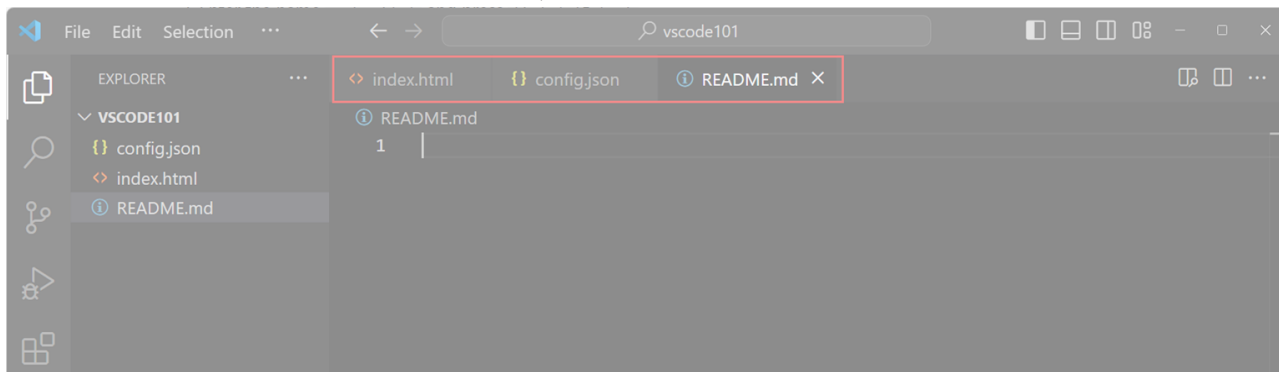
新建文件会自动添加到工作区中，并在窗口的主编辑区域打开。

在 index.html 文件中开始输入 HTML 代码，当你开始输入时，会自动弹出代码建议（IntelliSense）。

- 使用 **↑** 和 **↓** 键浏览建议。
- 按 **Tab** 键插入选中的建议。

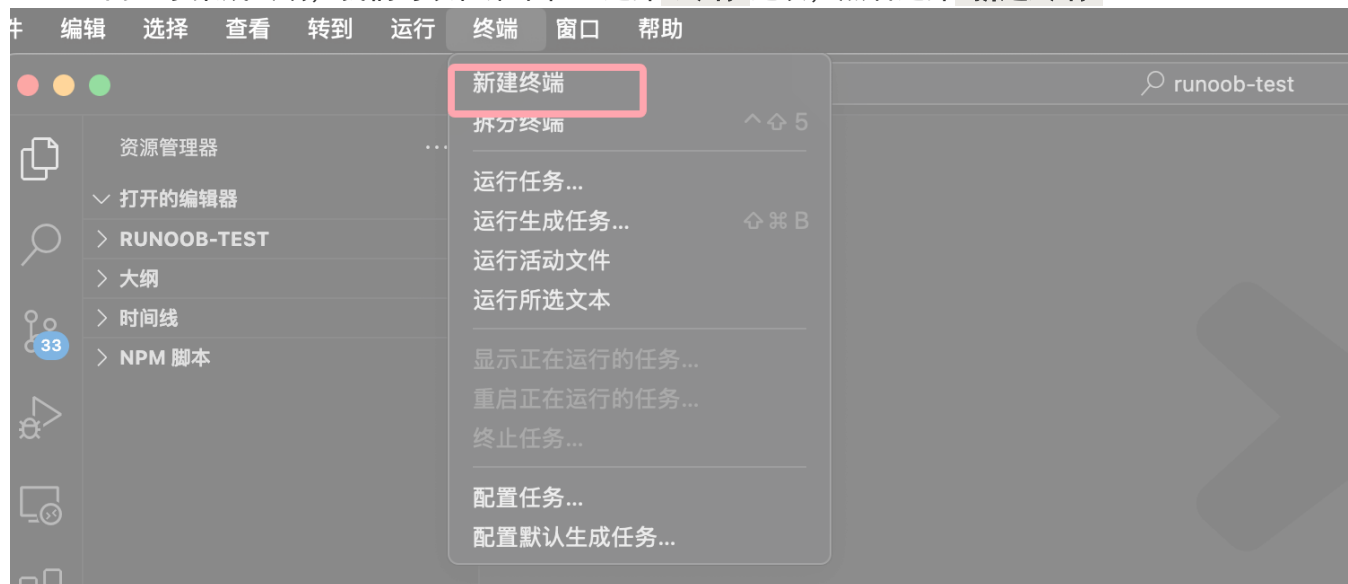
我们添加更多文件到工作区，每个文件会在一个新的编辑器标签页中打开。

VS Code 支持垂直或水平排列多个编辑器窗口，以便同时查看多个文件。



VSCode 集成终端

VS Code 内置了集成终端，我们可以在菜单栏上选择“终端”选项，然后选择“新建终端”：



也可以选择目录下的文件，右击，在下拉菜单选择“在集成终端中打开”：



终端快捷键：

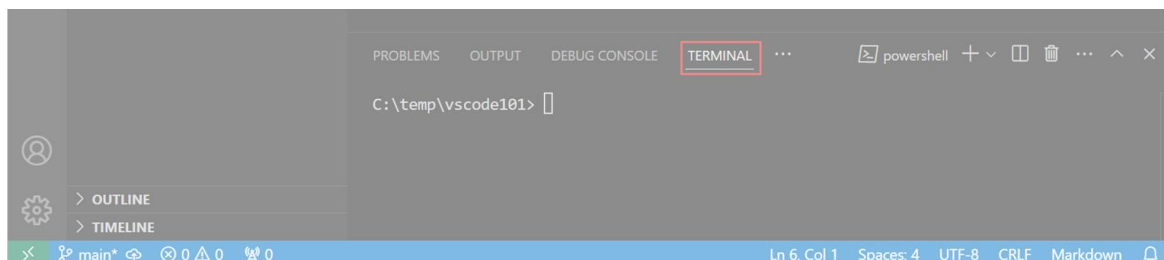
功能

Windows/Linux

VSCode 教程

显示集成终端	Ctrl + `
新建终端	Ctrl+Shift+``
切换终端	Ctrl+PageUp/PageDown
关闭终端	Ctrl+Shift+W

根据操作系统的不同，我们可以选择不同的终端环境，例如 PowerShell、命令提示符（Command Prompt）、或 Bash。

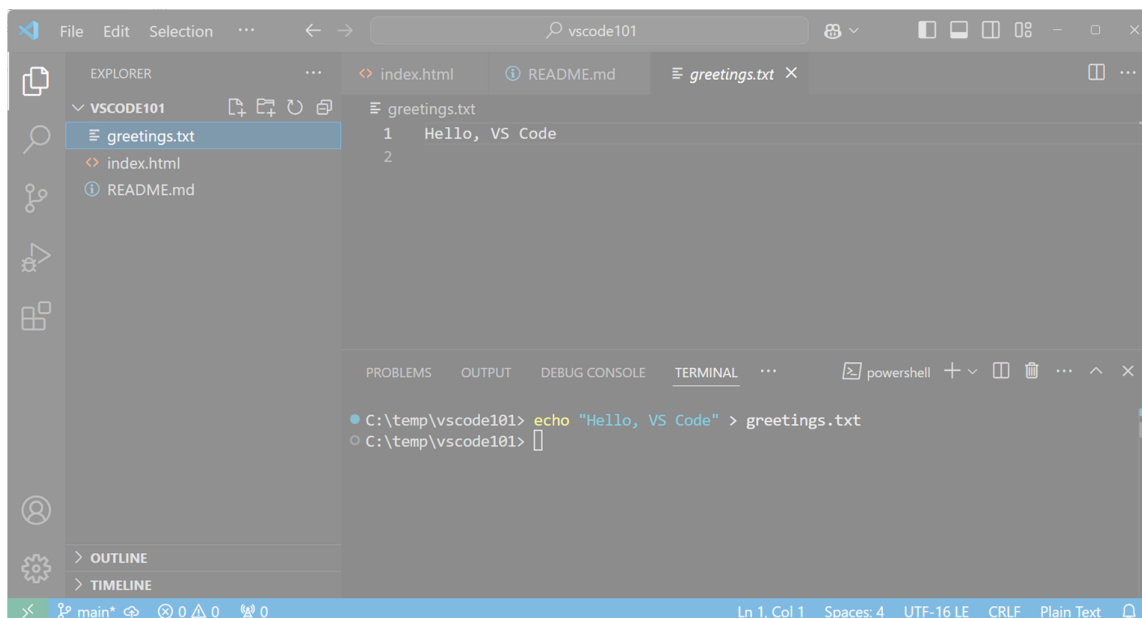


在终端中输入以下命令，创建一个新的文件 `greetings.txt`：

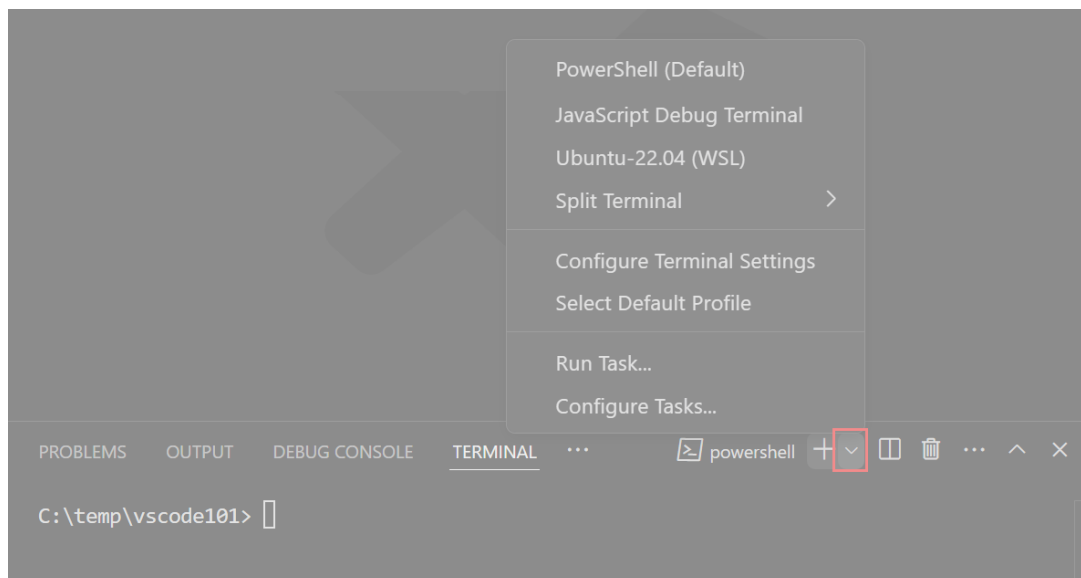
```
echo "Hello, VS Code" > greetings.txt
```

工作区的默认文件夹是当前项目的根目录。

创建文件后，“资源管理器（Explorer）”会自动显示新文件。



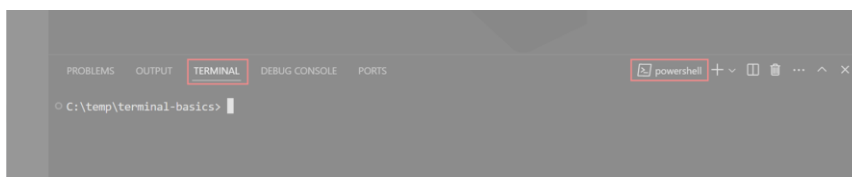
我们可以同时打开多个终端，点击终端右上角的“启动配置（Launch Profile）”下拉菜单，查看可用的终端环境并选择。



VSCode 教程

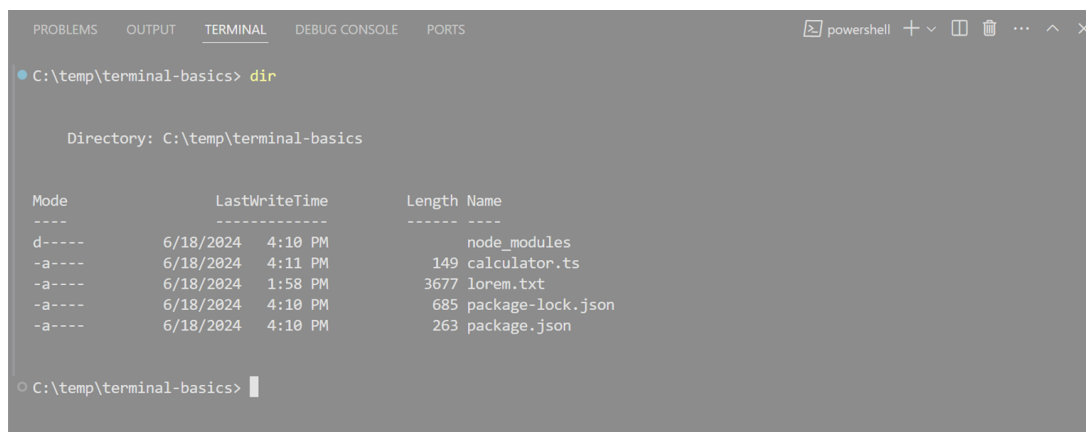
使用终端运行命令

终端会打开一个默认的 Shell，如 Bash、PowerShell 或 Zsh，终端的工作目录会从工作区文件夹的根目录开始。



输入类似 `ls` 或 `dir` 这样的基本命令来列出当前目录下的文件。

终端会直接显示命令的输出内容：



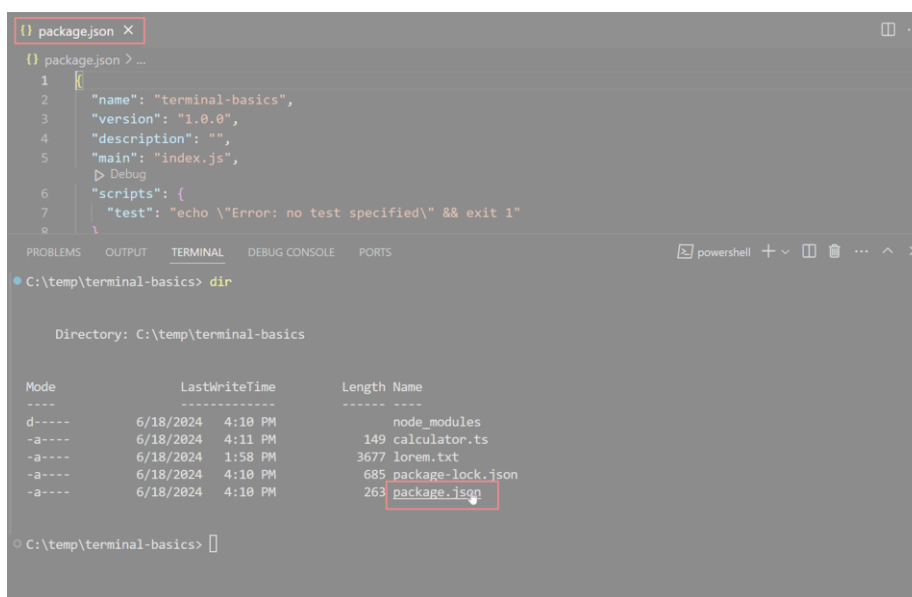
与命令输出交互

VS Code 中的终端还提供了与命令输出交互的功能，命令通常会输出文件路径或 URL，你可能希望直接打开或跳转到这些链接。

例如，编译器或代码检查工具可能会返回一个包含文件路径和行号的错误信息。你不需要手动去搜索该文件，只需在终端输出中选择该链接即可直接在编辑器中打开文件。

让我们看看如何与终端中的命令输出进行交互：

1. 打开你之前运行过 `ls` 或 `dir` 命令的终端。
2. 在终端中，按住 `Ctrl/Cmd` 键，将鼠标悬停在文件名上，然后选择该链接。



注意，当你将鼠标悬停在输出中的文本上时，它会变成一个链接。

当你选择文件名时，VS Code 会在编辑器中打开选定的文件。

终端输出中的所有文本都是可以点击的，如果你选择终端中的超链接，它会在默认浏览器中打开链接。

创建一个包含可用 Shell 命令的 Command.txt 文件

在 PowerShell 中：

VSCode 教程

```
Get-Command | Out-File -FilePath .\Command.txt
```

在 Bash / Zsh 中：

```
ls -l /usr/bin > Command.txt
```

输入以下命令以搜索 Command.txt 文件中的命令：

在 PowerShell 中：

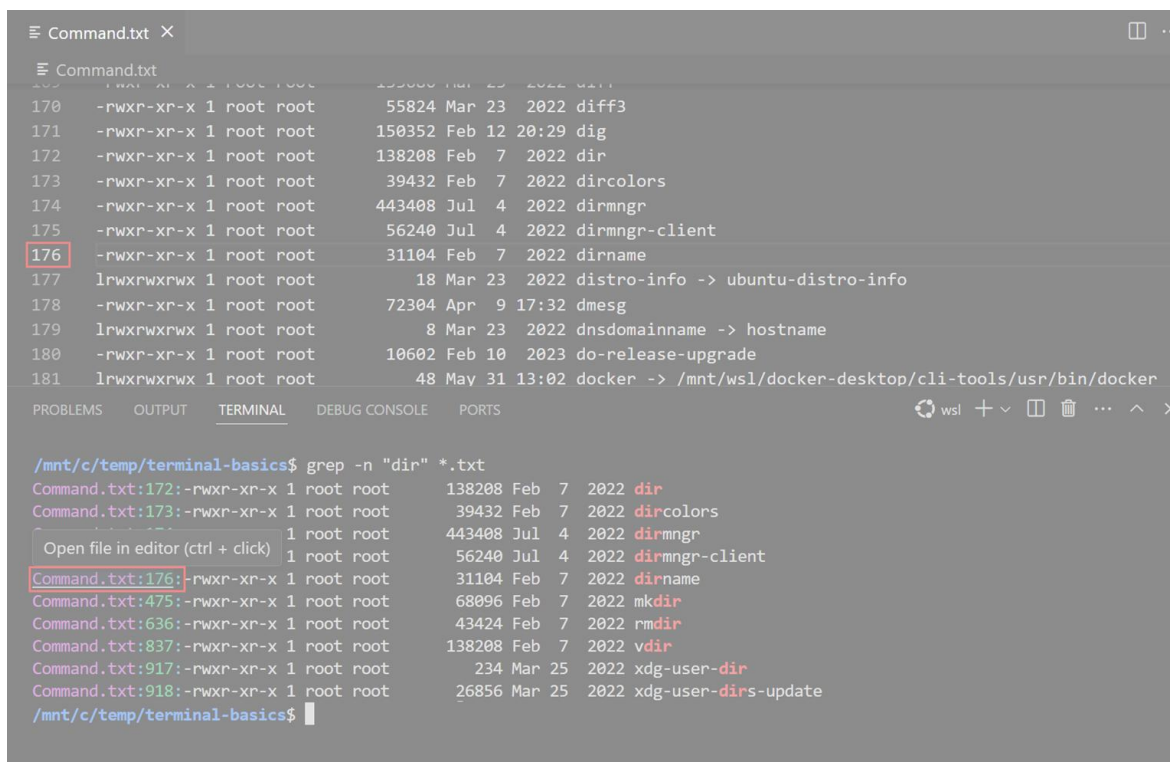
```
Get-ChildItem *.txt | Select-String "dir"
```

在 Bash / Zsh 中：

```
grep -n "dir" *.txt
```

注意，命令输出中会包含文件名和找到搜索结果的行号，终端会将这些文本识别为链接。

选择其中一个链接，可以在编辑器中打开该文件，并跳转到文件中的特定行。



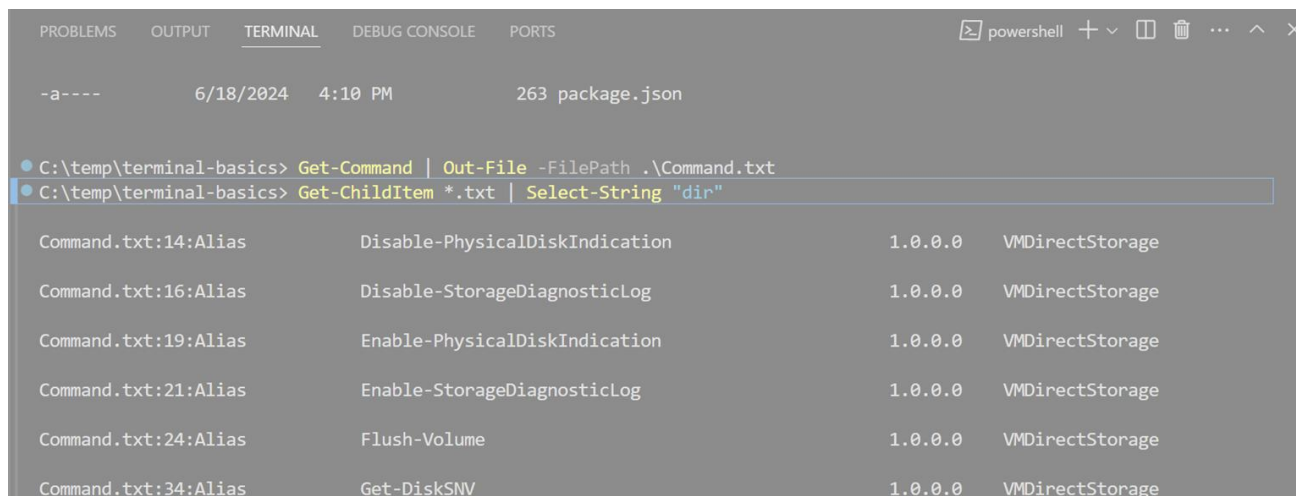
```
Command.txt 170 -rwxr-xr-x 1 root root 55824 Mar 23 2022 diff3
Command.txt 171 -rwxr-xr-x 1 root root 150352 Feb 12 20:29 dig
Command.txt 172 -rwxr-xr-x 1 root root 138208 Feb 7 2022 dir
Command.txt 173 -rwxr-xr-x 1 root root 39432 Feb 7 2022 dircolors
Command.txt 174 -rwxr-xr-x 1 root root 443408 Jul 4 2022 dirmngr
Command.txt 175 -rwxr-xr-x 1 root root 56240 Jul 4 2022 dirmngr-client
Command.txt:176:-rwxr-xr-x 1 root root 31104 Feb 7 2022 dirname
Command.txt 177 lrwxrwxrwx 1 root root 18 Mar 23 2022 distro-info -> ubuntu-distro-info
Command.txt 178 -rwxr-xr-x 1 root root 72304 Apr 9 17:32 dmesg
Command.txt 179 lrwxrwxrwx 1 root root 8 Mar 23 2022 dnsdomainname -> hostname
Command.txt 180 -rwxr-xr-x 1 root root 10602 Feb 10 2023 do-release-upgrade
Command.txt 181 lrwxrwxrwx 1 root root 48 May 31 13:02 docker -> /mnt/wsl/docker-desktop/cli-tools/usr/bin/docker

/mnt/c/temp/terminal-basics$ grep -n "dir" *.txt
Command.txt:172:-rwxr-xr-x 1 root root 138208 Feb 7 2022 dir
Command.txt:173:-rwxr-xr-x 1 root root 39432 Feb 7 2022 dircolors
Command.txt:174:-rwxr-xr-x 1 root root 443408 Jul 4 2022 dirmngr
Command.txt:175:-rwxr-xr-x 1 root root 56240 Jul 4 2022 dirmngr-client
Command.txt:176:-rwxr-xr-x 1 root root 31104 Feb 7 2022 dirname
Command.txt:177:-rwxr-xr-x 1 root root 68096 Feb 7 2022 mkdir
Command.txt:636:-rwxr-xr-x 1 root root 43424 Feb 7 2022 rmdir
Command.txt:837:-rwxr-xr-x 1 root root 138208 Feb 7 2022 vdir
Command.txt:917:-rwxr-xr-x 1 root root 234 Mar 25 2022 xdg-user-dir
Command.txt:918:-rwxr-xr-x 1 root root 26856 Mar 25 2022 xdg-user-dirs-update
/mnt/c/temp/terminal-basics$
```

浏览历史命令

在使用终端时，我们可能需要查看之前的命令及其输出，或者可能想重新运行某个命令。

我们可以使用快捷键快速浏览历史命令：按下 **Ctrl + ↑** 或 **⌘ + ↑** 快捷键，向上滚动至终端历史记录中的上一条命令。



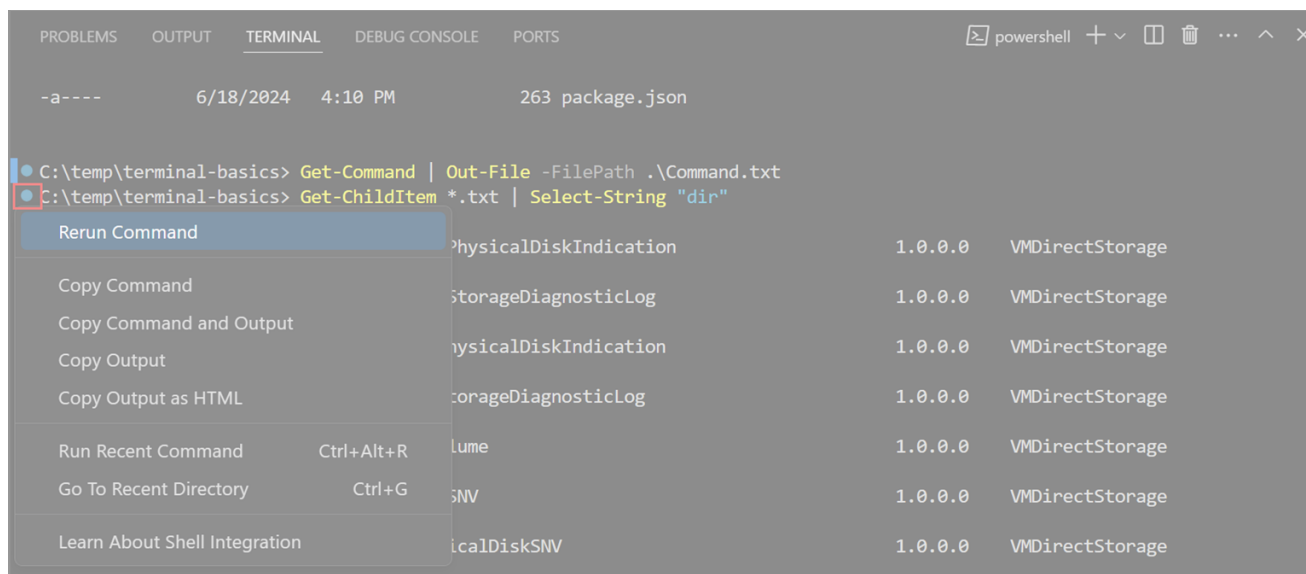
```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
-a---- 6/18/2024 4:10 PM 263 package.json

C:\temp\terminal-basics> Get-Command | Out-File -FilePath .\Command.txt
C:\temp\terminal-basics> Get-ChildItem *.txt | Select-String "dir"

Command.txt:14:Alias Disable-PhysicalDiskIndication 1.0.0.0 VMDirectStorage
Command.txt:16:Alias Disable-StorageDiagnosticLog 1.0.0.0 VMDirectStorage
Command.txt:19:Alias Enable-PhysicalDiskIndication 1.0.0.0 VMDirectStorage
Command.txt:21:Alias Enable-StorageDiagnosticLog 1.0.0.0 VMDirectStorage
Command.txt:24:Alias Flush-Volume 1.0.0.0 VMDirectStorage
Command.txt:34:Alias Get-DiskSNV 1.0.0.0 VMDirectStorage
```

如果你多次按下 **Ctrl + ↑**，终端会继续向上滚动历史记录。你也可以使用 **Ctrl + ↓** 快捷键向下浏览历史记录。

最好我们会看到一个圆形图标出现在某个已运行命令旁边，选择该圆形图标，然后选择“重新运行命令”来重新执行该命令。

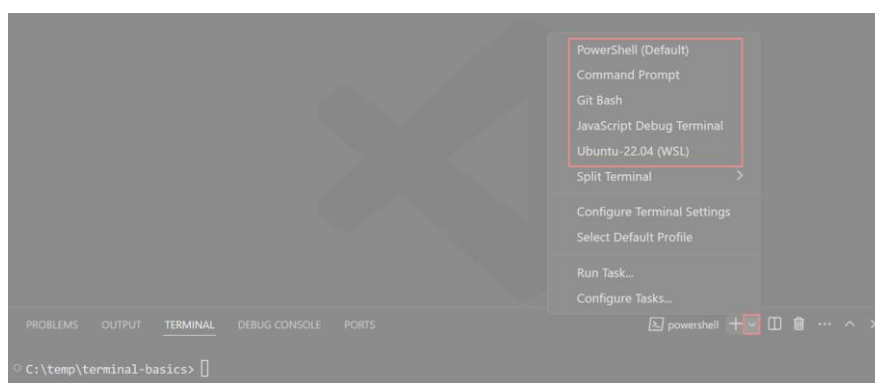


不同 Shell 中运行命令

终端支持同时打开多个终端。

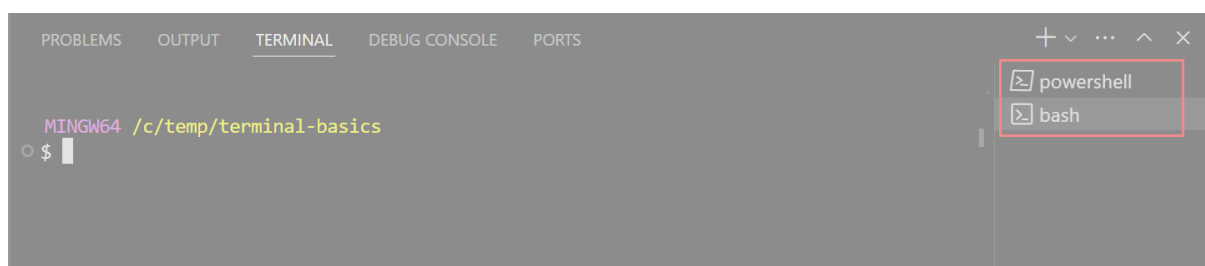
例如，我们可以将一个终端用于运行 Git 命令，另一个终端用于运行构建脚本。

在不同 Shell 中添加新终端：选择终端面板中的  图标以打开终端下拉菜单，然后从可用的 Shell 中选择一个。



注意： 可用的 Shell 取决于你电脑上安装的 Shell。

新终端会以你选择的 Shell 打开，你可以像之前一样输入命令。

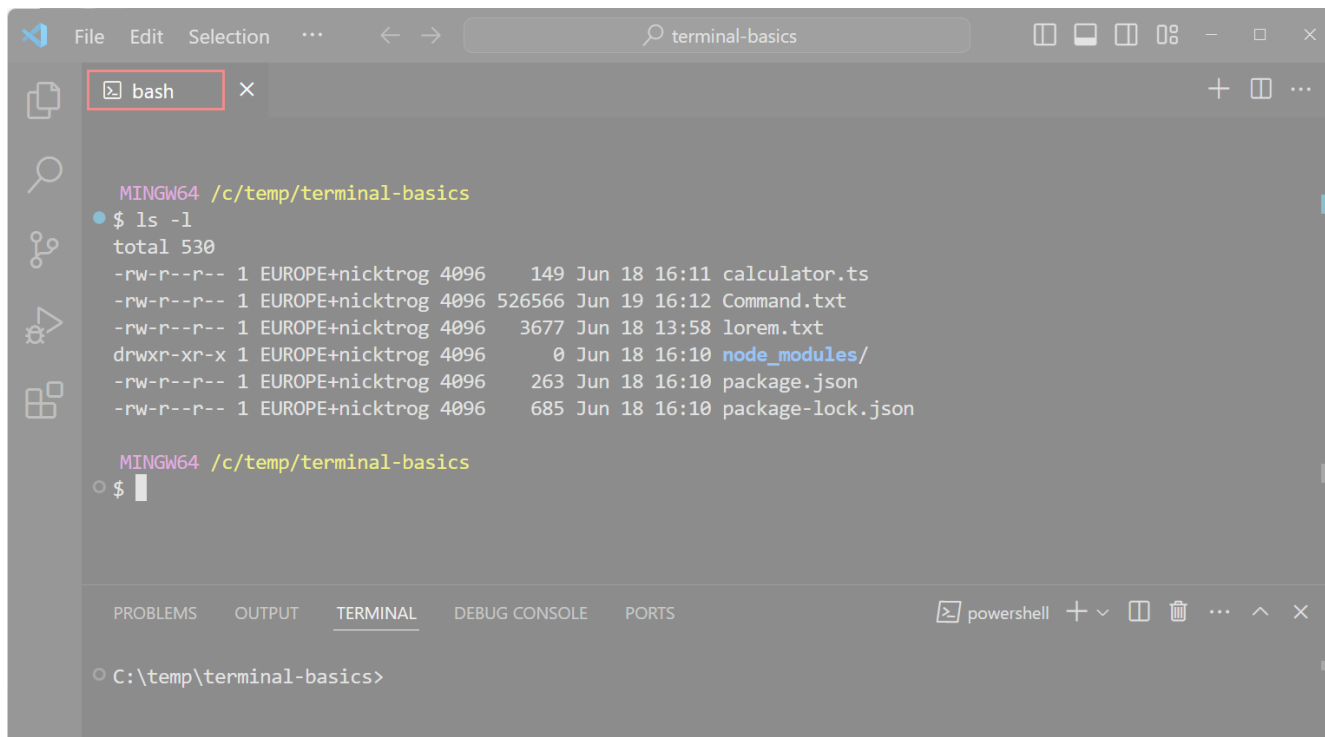


提示： 你也可以选择  图标来为默认 Shell 创建新终端，使用快捷键 `^⏏` (Ctrl + Shift + `)，或者从菜单栏选择 **终端** > **新终端** 来创建新终端。

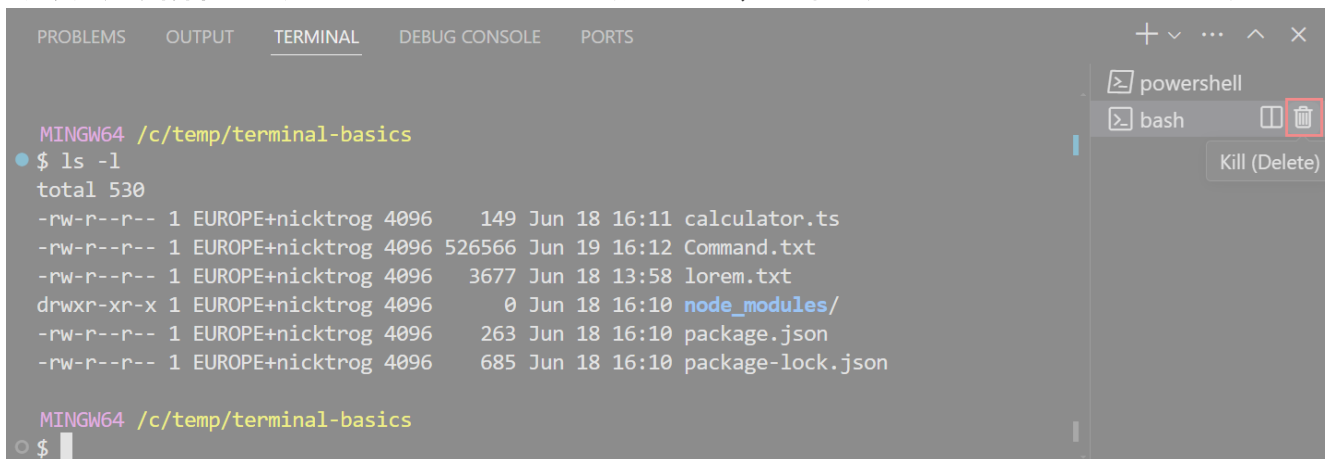
我们还可以将终端从终端列表拖动到编辑器区域。

例如，我们将终端标签拖出 VS Code 窗口，使其成为浮动窗口。

VSCode 教程



将终端移动到编辑器区域：当你将鼠标悬停在终端列表上时，选择垃圾桶图标来关闭已打开的终端。



VSCode 命令面板

VSCode (Visual Studio Code) 的命令面板是一个非常强大的工具，它允许用户快速访问和执行 VSCode 中的各种命令和功能。

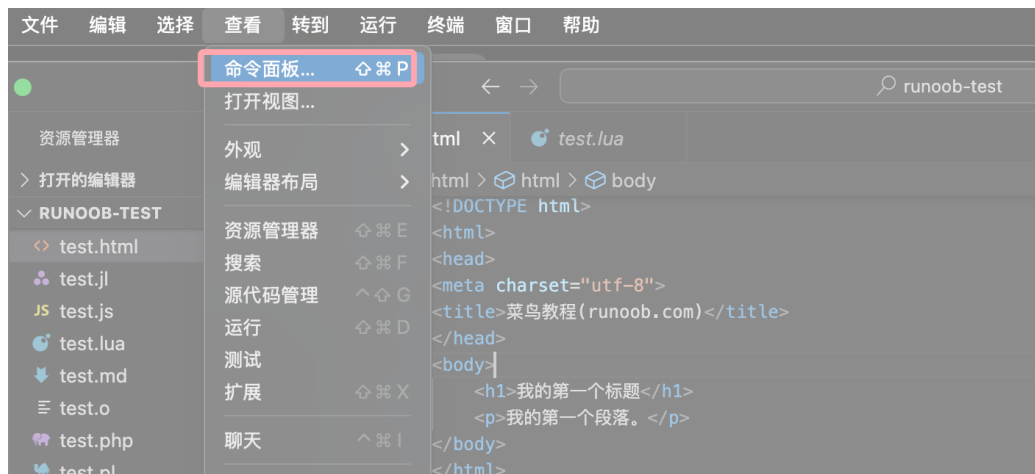
我们可以通过命令面板执行 VSCode 内置的命令，比如打开文件、搜索符号、重命名符号、保存文件等。

VSCode 使用以下快捷打开命令面板 (Command Palette)：

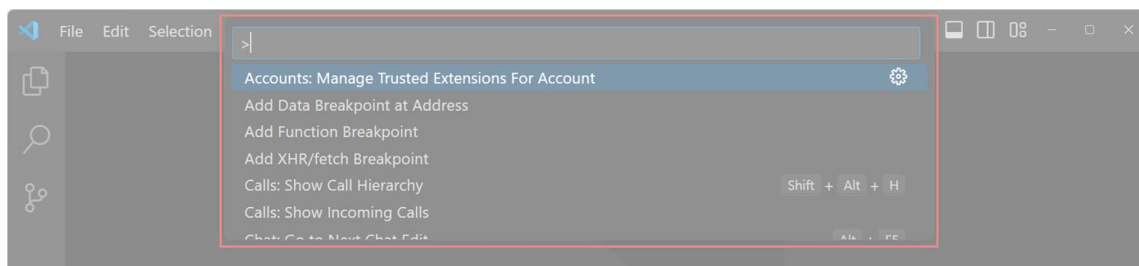
- macOS 系统快捷键：⌘⇧P
- Windows/Linux 快捷键：Ctrl + Shift + P

也可以通过菜单栏选择 **查看 (View) > 命令面板 (Command Palette...)** 打开。

VSCode 教程



命令面板几乎涵盖了 VS Code 中的所有命令，包括你安装的扩展所添加的命令。



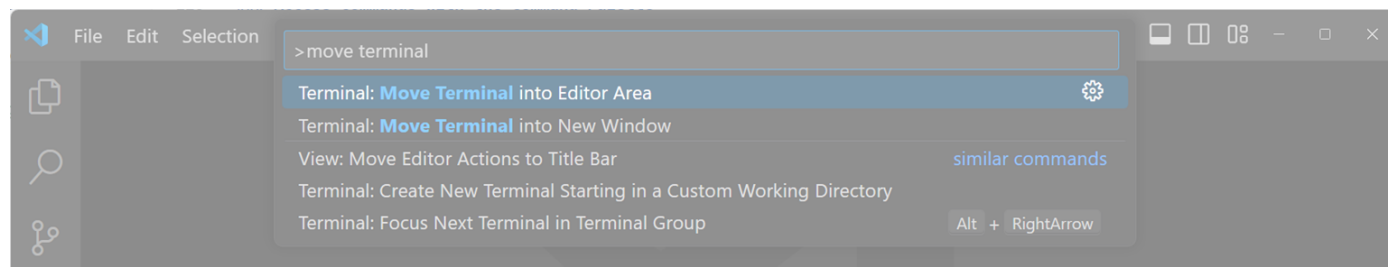
注意：命令面板会显示带有默认快捷键的命令，如果某个命令有快捷键，你可以直接使用该快捷键运行命令，而无需通过命令面板操作。

命令面板的操作模式

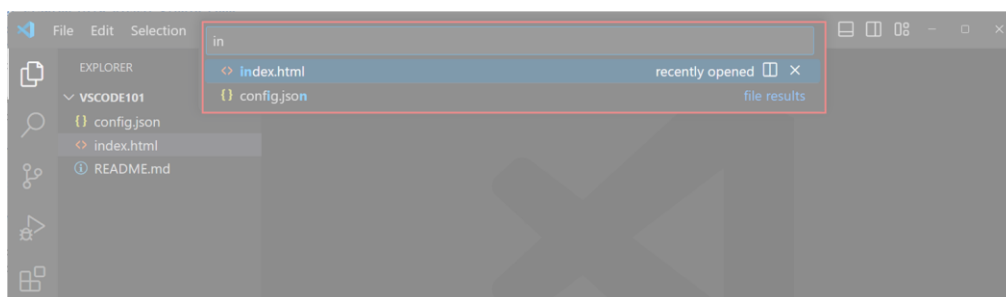
命令面板支持多种操作模式，根据输入内容呈现不同功能：

输入 **>** 符号后：开始输入以过滤命令列表。

例如，输入 **move terminal**，可以找到将终端移动到新窗口的相关命令。



删除 **>** 符号后：输入内容将搜索工作区中的文件。



按下 **Ctrl + Shift + P** (Windows/Linux/macOS 快捷键：Ctrl + Shift + P) 快捷键，可以直接打开命令面板并开始搜索文件。

VS Code 设置

VS Code (Visual Studio Code) 是一个功能丰富的代码编辑器，其设置功能允许用户自定义编辑器的行为和外观，以适应个人的工作习惯和偏好。

VSCode 教程

配置 VS Code 的两种方式：

- 使用设置编辑器（Settings Editor） 修改设置。
- 直接编辑 settings.json 文件。

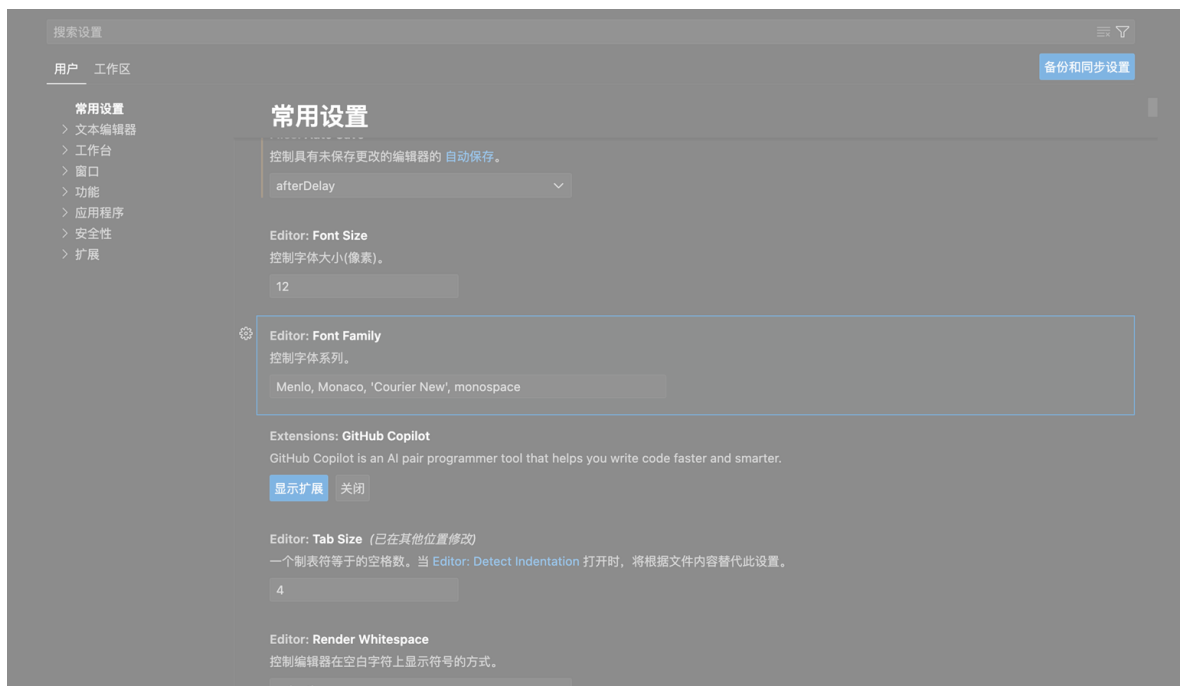
使用快捷键打开设置页面：

- macOS 快捷键：⌘ + ,
- Windows/Linux 快捷键：Ctrl + ,

也可以通过菜单选择 **Code > 首选项 > 设置** 来打开设置页面：



设置页面如下所示：



页面布局：

- **设置页面顶部：** 页面顶部的搜索框可快速筛选设置，输入关键字即可查看相关配置项。
- **左侧导航栏：** 分类显示不同的设置选项，例如常用设置、文本编辑器、工作台、窗口等，方便快速定位需要修改的项目。
- **右侧设置详情：** 展示具体的设置项和对应的值，每一项通常伴随简要说明。

常用的设置项有：

- **字体大小（Font Size）：** 设置编辑器的字体大小。
- **字体系列（Font Family）：** 指定编辑器的字体，例如 Menlo、Monaco 等。
- **制表符大小（Tab Size）：** 控制一个制表符的空格数，默认是 4。
- **自动保存（Auto Save）：** 决定文件是否自动保存及保存的触发条件。

VSCode 教程

启用自动保存

默认情况下，VS Code 不会自动保存已修改的文件。

在设置编辑器中，通过 Auto Save 下拉框选择一个值，即可更改此行为。



修改后，VS Code 会自动应用设置变化。

如果启用自动保存，当你修改工作区中的文件时，文件将自动保存。

恢复默认设置

要将某项设置恢复为默认值，将鼠标移动到设置项旁边，会显示齿轮图标，点击它然后选择重置此设置：

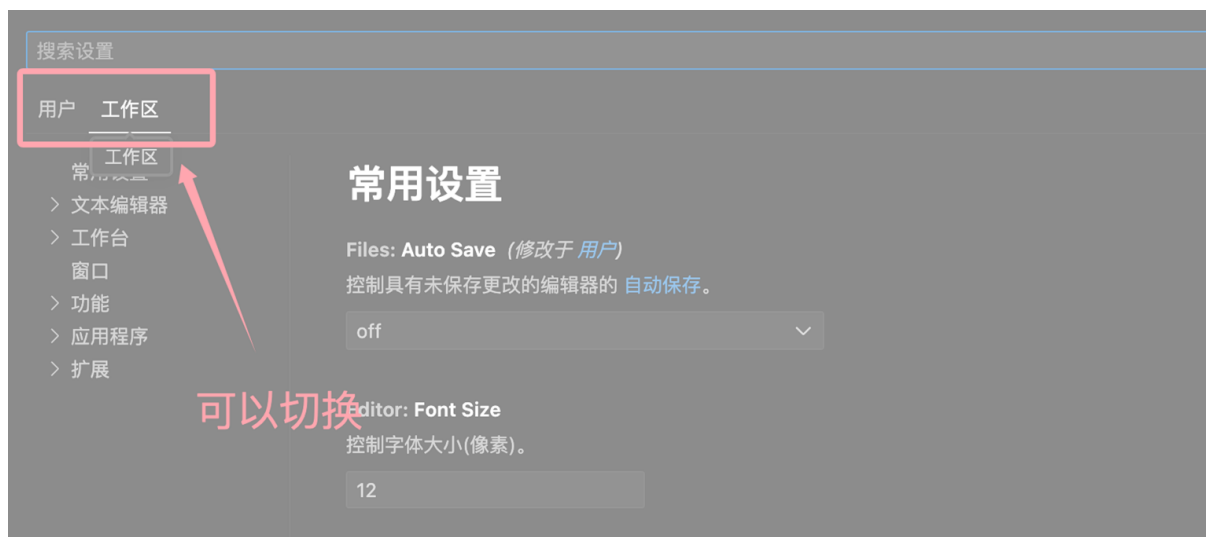


要想快速找到所有已修改的设置，可以在搜索框中输入 **@modified:**



切换用户设置与工作区设置

- 用户设置 (User settings)：对所有工作区生效。
- 工作区设置 (Workspace settings)：仅适用于当前工作区，并会覆盖用户设置。



VSCode 编写代码

VS Code 内置了对 JavaScript、TypeScript、HTML、CSS 等多种语言的支持。

在本章节中，我们将创建一个 JavaScript 代码文件，并使用 VS Code 提供的一些代码编辑功能。

VS Code 支持多种编程语言，在后面的章节中，我们还会安装 **Python 的语言扩展**，为其他语言添加支持。

1、创建 JavaScript 文件并编写代码

在资源管理器（Explorer）视图中，创建一个名为 `app.js` 的新文件，并输入以下 JavaScript 代码：

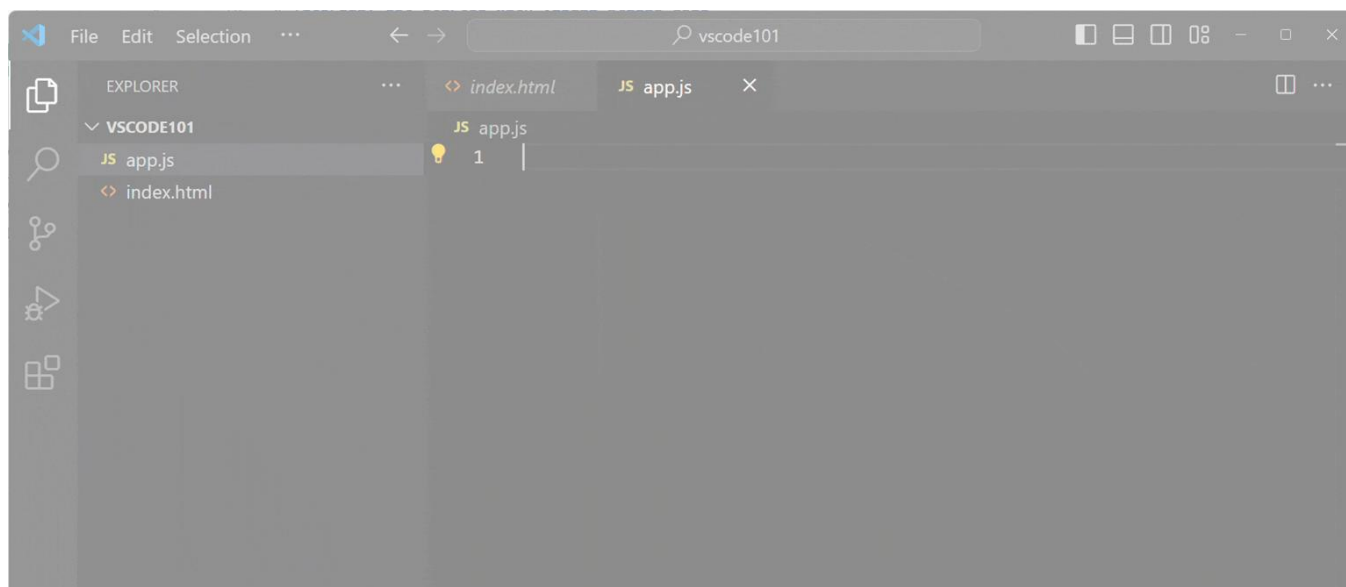
实例

```
function sayHello(name) {  
  console.log('Hello, ' + name);  
}
```

```
sayHello('VS Code');
```

代码自动补全（IntelliSense）：当您开始输入代码时，会弹出代码补全建议，使用方向键 **↑** 或 **↓**（上下键）导航建议项，按 **Tab** 键选择并插入选中的建议。

语法高亮：注意代码的格式化显示（语法高亮），这有助于区分代码的不同部分，提高可读性。



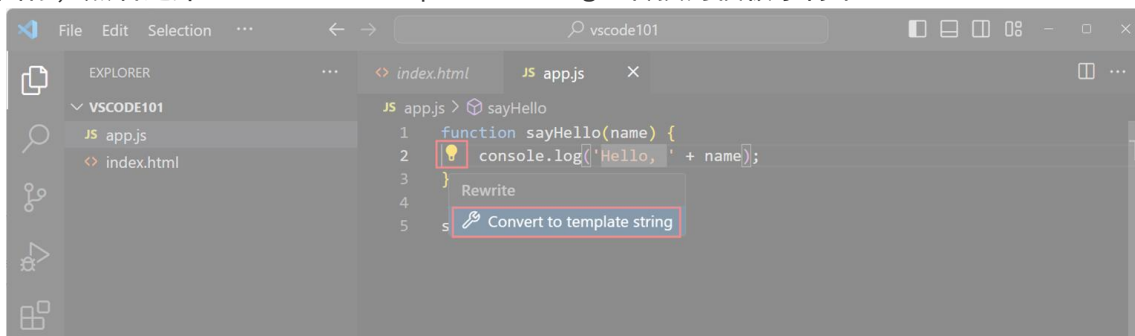
使用代码操作（Code Actions）

将光标放在字符串 `'Hello, '` 上时，您会看到一个小灯泡图标，表示可以应用代码操作（Code Action）。

您也可以使用快捷键 **^Space** 打开灯泡菜单。

VSCode 教程

点击灯泡图标，然后选择 Convert to template string（转换为模板字符串）。



代码操作为您的代码提供了快速修复建议。

在本例中，Code Action 将字符串拼接：

```
"Hello, " + name
```

转换为模板字符串：

```
`Hello, ${name}`
```

模板字符串是 JavaScript 中一种特殊的语法，可以在字符串中嵌入表达式。

转化后的代码如下：



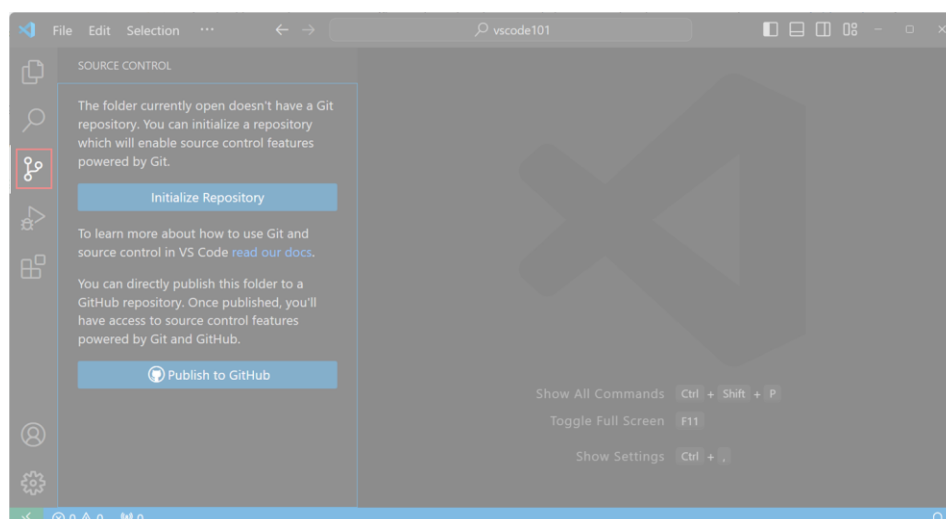
VSCode 版本控制

VSCode (Visual Studio Code) 的版本控制功能主要通过集成 Git 来实现，它提供了一套完整的界面和工具，使得开发者可以方便地进行代码的版本管理。

接下来，我们将使用内置的 Git 支持，提交之前所做的更改。

打开版本控制视图

选择活动栏 (Activity Bar) 中的"版本控制"视图，从而打开版本控制功能。

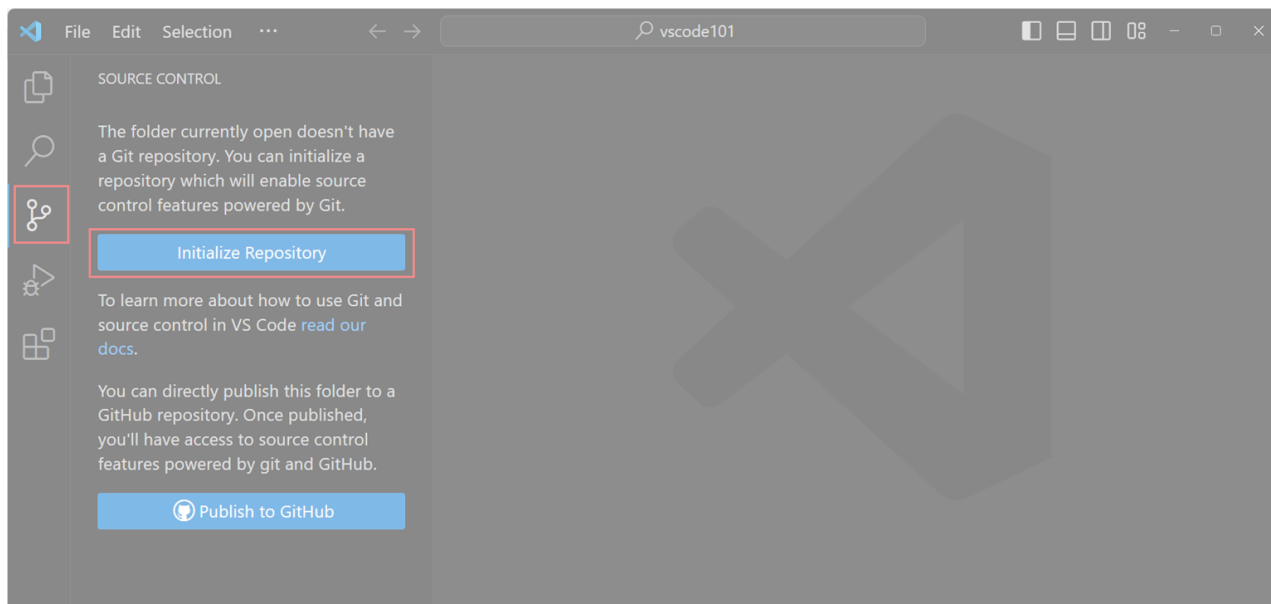


使用前要确保已在计算机上安装 Git，如果尚未安装 Git，在"版本控制"视图中会看到一个按钮，可点击进行安装。

VSCode 教程

初始化 Git 仓库

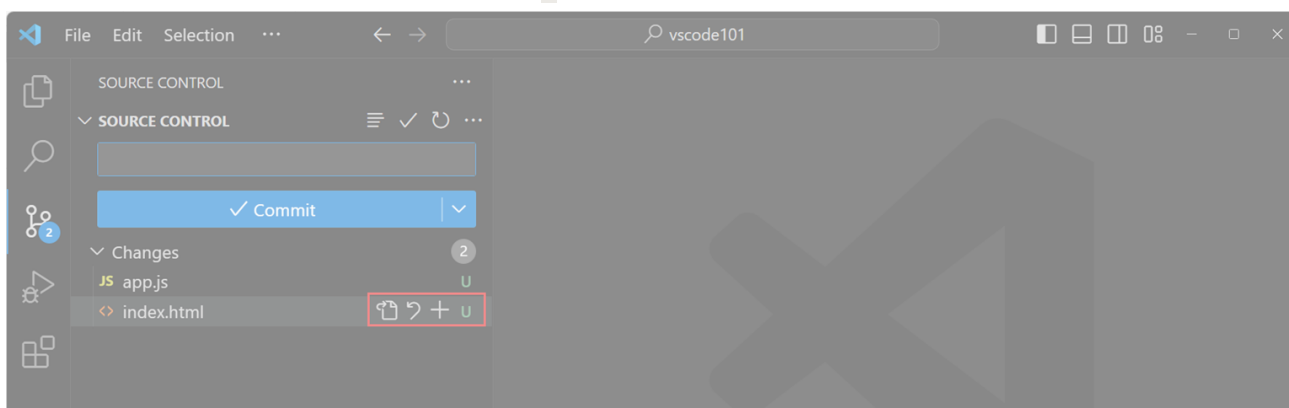
选择 "Initialize Repository"（初始化仓库） 按钮，为当前工作区创建一个新的 Git 仓库。



初始化完成后，"版本控制"视图会显示您在工作区中所做的更改。

暂存更改并提交

将鼠标悬停在某个文件上，点击文件旁边的 **+** 图标，可以暂存该文件的更改。

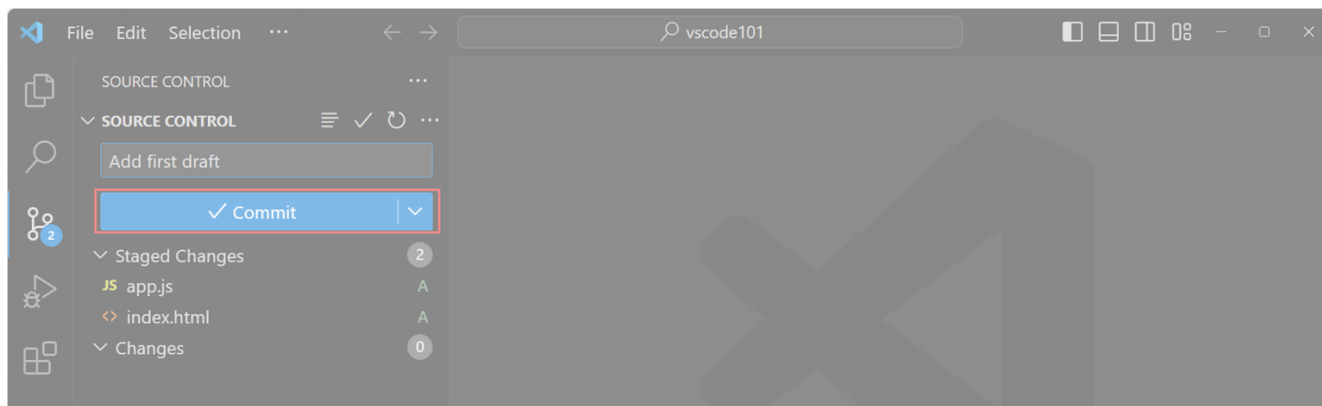


将鼠标悬停在"更改"（Changes）区域，并点击 "Stage All Changes"（暂存所有更改） 按钮，快速暂存所有文件。

提交更改

输入提交消息，例如 "Add hello function"（新增 hello 函数）。

点击 "Commit"（提交） 按钮，将更改提交到您的 Git 仓库。



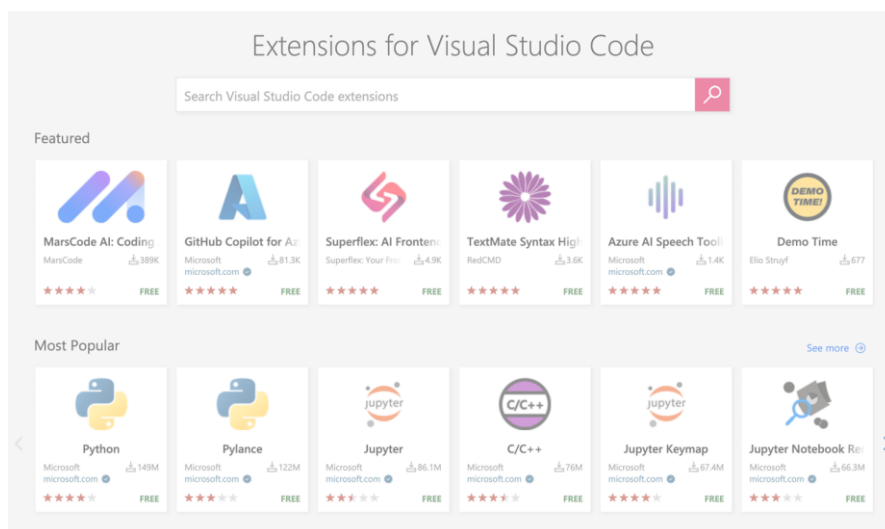
VS Code 的扩展功能和市场是其强大和灵活的核心特性之一，允许用户通过安装各种扩展来增强和定制编辑器的功能。

VS Code 拥有丰富的扩展生态系统，允许您为安装添加语言支持、调试器和工具，以适应您的特定开发工作流程。

在 Visual Studio Marketplace 中，有成千上万的扩展可供选择：

- **编程语言支持：**提供特定语言的语法高亮、代码补全、智能提示等功能。
- **调试工具：**扩展 VS Code 的调试功能，支持更多语言和环境。
- **版本控制：**提供更好的 Git 集成，如 GitLens 这样的扩展，增强代码版本控制能力。
- **主题和美化：**改变编辑器的视觉风格，包括主题、图标、字体等，提高编码环境的个性化。
- **工具集成：**集成其他开发工具和平台的功能，例如容器工具、云服务管理等。
- **辅助开发：**提供 API 文档、代码片段、快速修复建议等辅助开发功能。

VS Code 扩展市场：<https://marketplace.visualstudio.com/>

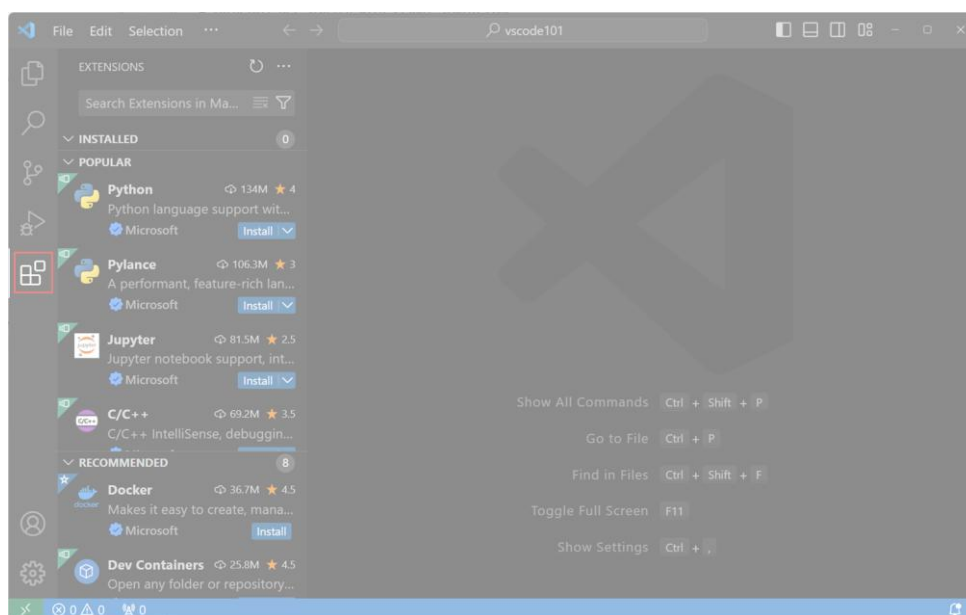


接下来，我们安装一个语言扩展，为 Python 或其他您感兴趣的编程语言添加支持。

1、打开扩展视图

选择 **活动栏 (Activity Bar)** 中的 **扩展视图 (Extensions View)** 图标。

扩展视图可以让您直接在 VS Code 中浏览和安装扩展。

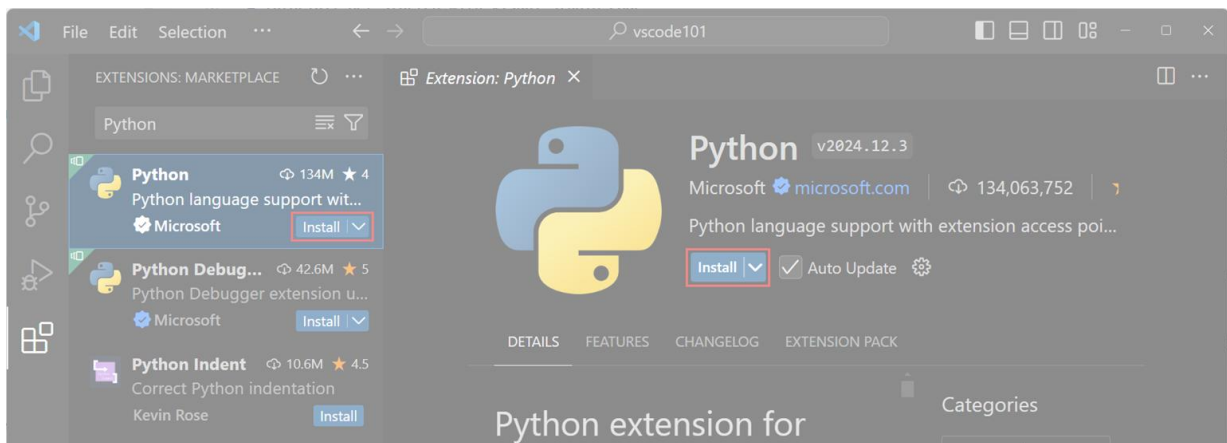


2、安装 Python 扩展

在扩展视图的搜索框中输入 Python，以浏览与 Python 相关的扩展。

选择由 Microsoft 发布的 Python 扩展，然后点击 Install（安装）按钮。

VSCode 教程



3、编写 Python 代码

在工作区中创建一个新的 Python 文件，命名为 `hello.py`。

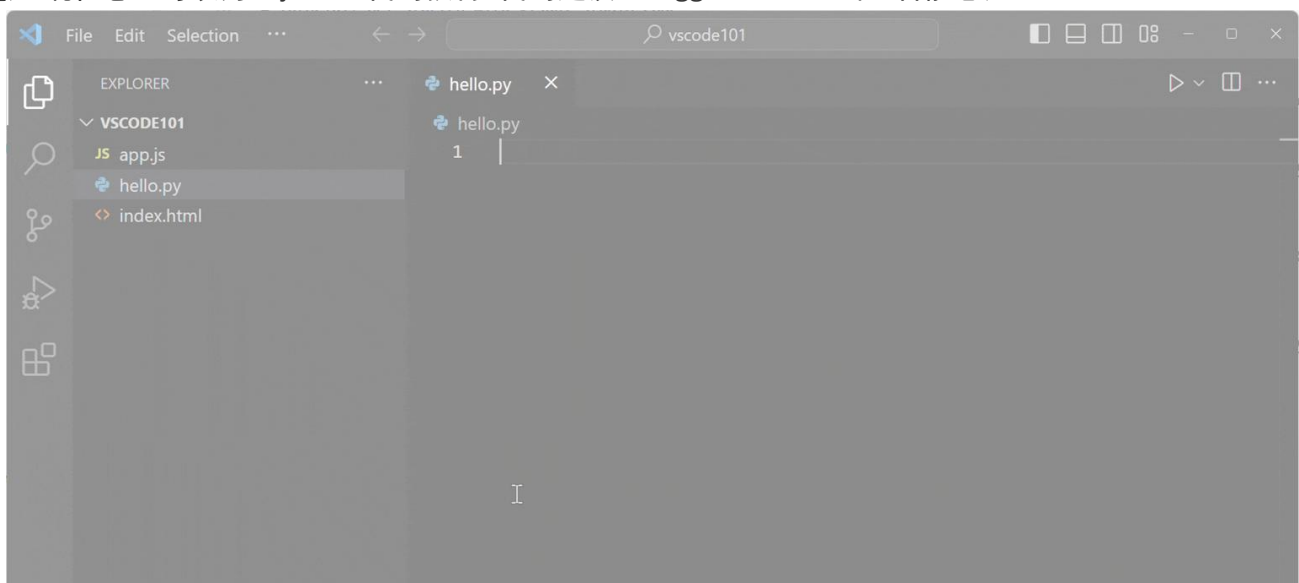
开始输入以下 Python 代码：

实例

```
def say_hello(name):  
    print("Hello, " + name)
```

```
say_hello("VS Code")
```

注意：现在您也可以为 Python 代码获得 代码建议（Suggestions）和 智能感知（IntelliSense）。



智能感知（IntelliSense）提供代码补全、语法高亮等功能，帮助您更高效地编写代码。

通过安装扩展，您可以根据需求扩展 VS Code 的功能，从而更好地支持各种编程语言！

VSCode 运行和调试代码

VS Code 内置了运行和调试 Node.js 应用程序的支持。

本章节，我们将使用上一部分安装的 **Python 扩展** 来调试一个 Python 程序。

接下来就让我们调试前面创建的 `hello.py` 程序。

调试前确保您的计算机已安装 Python 3。

如果您的计算机上未安装 Python 解释器，窗口右下角会弹出通知。

VSCode 教程

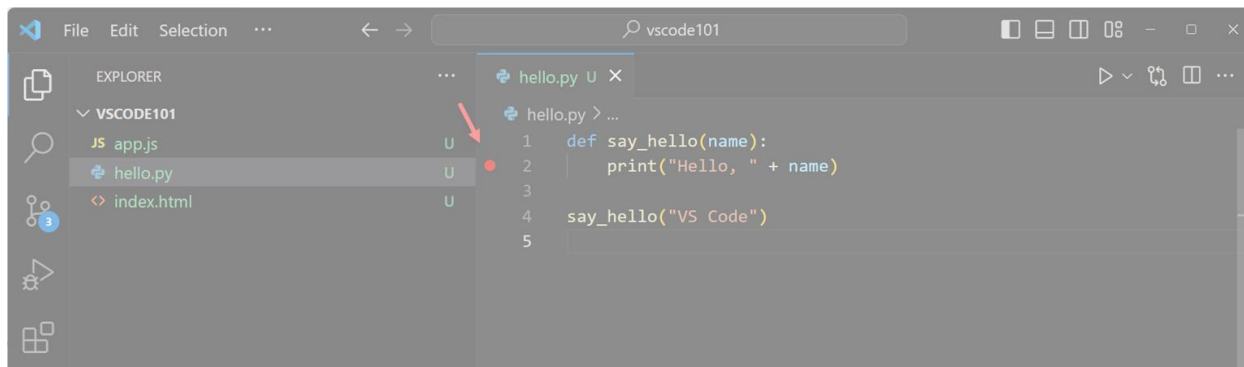
选择 Select Interpreter（选择解释器） 打开命令面板，从中选择您想使用的 Python 解释器，或安装一个新的解释器。

设置断点

在 `hello.py` 文件中，将光标放置在 `print` 行上，点击小红点（鼠标移动到红点位置会显示），或者按下 **F9** 键来设置断点。

编辑器左侧的边距中会出现一个红点，表示已设置断点。

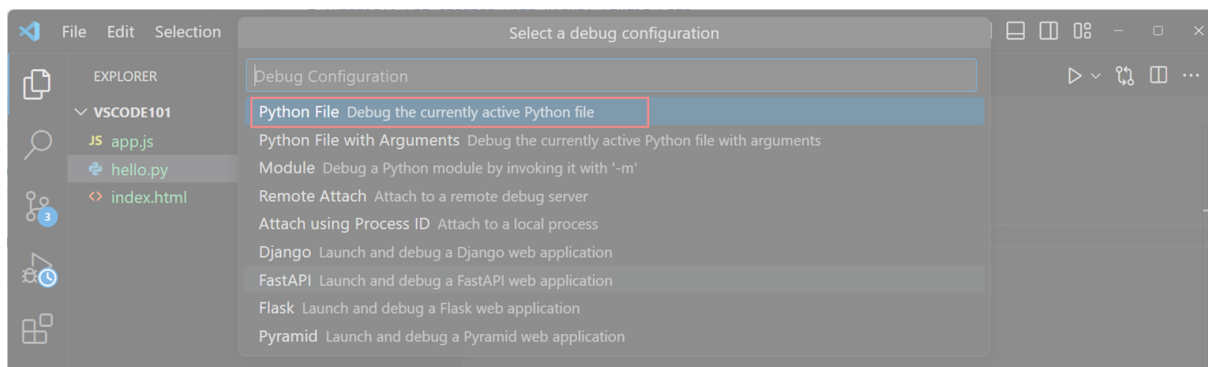
断点可以让您在程序执行到某一行代码时暂停，便于调试。



启动调试会话

按下 **F5** 启动调试会话。

系统会提示选择调试器，选择 Python 调试器。

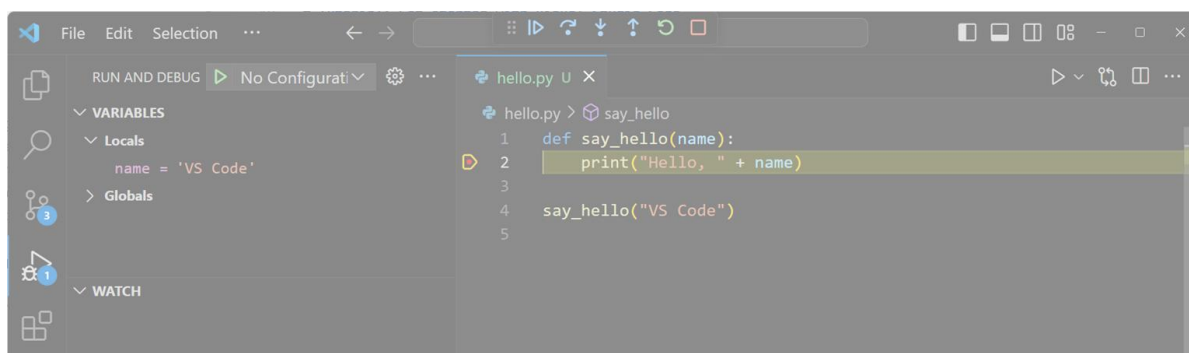


选择运行当前的 Python 文件：



调试代码

程序启动后，执行会在您设置的断点处暂停。

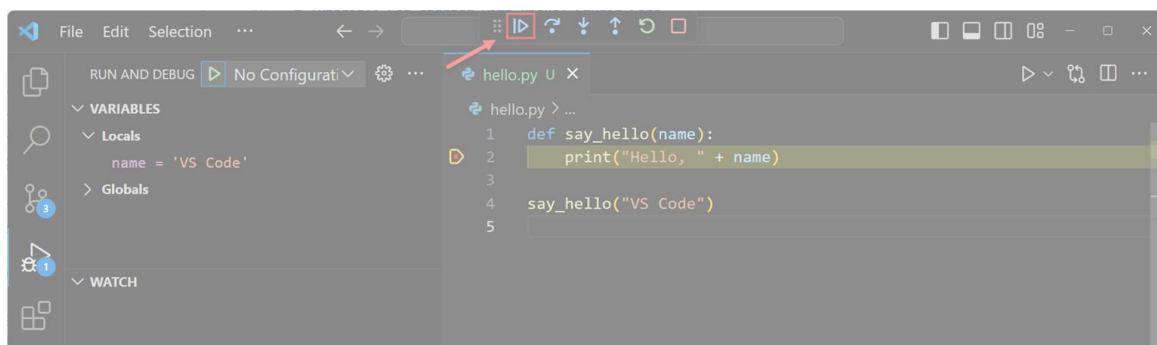


VSCode 教程

提示:将鼠标悬停在编辑器中的 `name` 变量上,即可查看其值。在调试视图的 变量视图(Variables View)中,您可以随时查看变量的值。

继续执行程序

在调试工具栏中,点击 Continue (继续) 按钮,或再次按下 F5,以继续执行程序。



VS Code 提供了许多高级的调试功能,例如监视变量(Watch Variables)、条件断点(Conditional Breakpoints)和启动配置(Launch Configurations)。

VSCode code 命令

在 Visual Studio Code 中, `code` 命令是一个命令行工具,用于快速打开 VS Code 并执行一些与代码相关的操作。

`code` 命令直接可以帮助开发者从终端或命令提示符中直接启动 VS Code 或处理特定的任务。

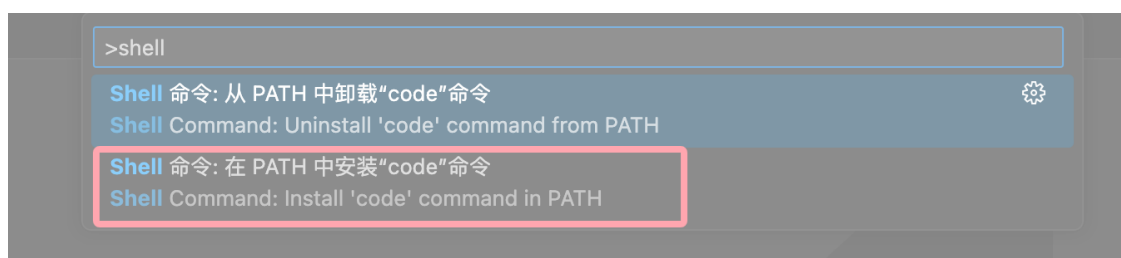
最常用的方式就是使用 `code` 命令直接从命令行中打开文件目录,此时需要先安装 `code` 命令。

Windows 系统安装 VS Code 后, `code` 命令会自动注册到系统环境变量中,如果未注册,可以手动添加 VS Code 的安装路径到 PATH 环境变量中。

启用 VSCode 的 `code` 命令 非常简单,先打开命令面板:

- macOS 系统快捷键: `⌘P` (`command + shift + p`)
- Windows/Linux 快捷键: `Ctrl + Shift + P`

搜索安装 `>shell command`:



然后选择 `在 PATH 中安装"code"命令 - Shell Command: Install 'code' command in PATH` 即可为系统 PATH 路径添加了 `code` 命令的引用。

我们可以通过命令行打开文件、安装扩展、修改显示语言,甚至查看诊断信息。


```

C:\>code --version
1.22.2
3aeede733d9a3098f7b4bdc1f66b63b0f48c1ef9
x64

C:\>code --list-extensions
DavidAnson.vscode-markdownlint
dbaeumer.vscode-eslint
eg2.tslint
msjsdiag.debugger-for-chrome
robertohuertasm.vscode-icons
streetsidesoftware.code-spell-checker

C:\>
```

以下是一些常用的命令行选项，可以通过 `code --help` 命令查看：

```

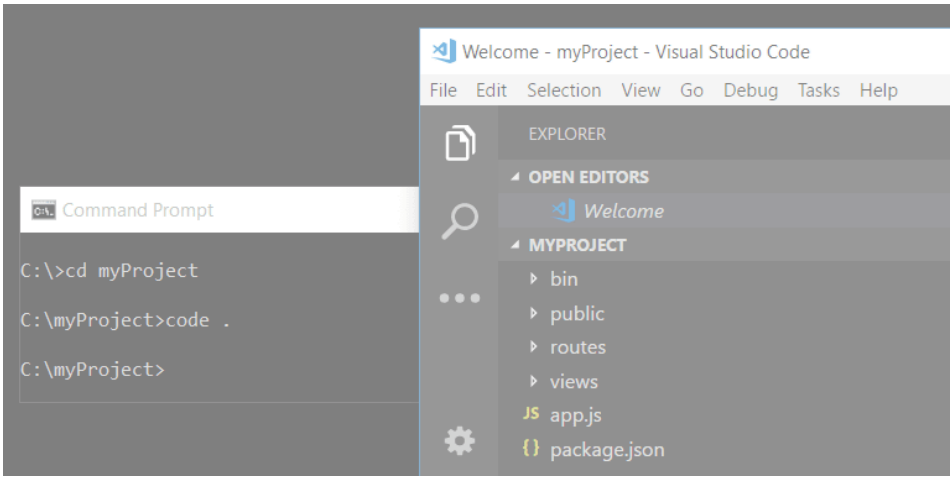
C:\>code --help
Visual Studio Code 1.39.1

Usage: code.exe [options][paths...]

To read output from another program, append '-' (e.g. 'echo Hello World | code.exe -')

Options
  -d --diff <file> <file>          Compare two files with each other.
  -a --add <folder>                Add folder(s) to the last active window.
  -g --goto <file:line[:character]> Open a file at the path on the specified
                                   line and character position.
  -n --new-window                  Force to open a new window.
  -r --reuse-window                Force to open a file or folder in an
                                   already opened window.
  -w --wait                        Wait for the files to be closed before
                                   returning.
  --locale <locale>               The locale to use (e.g. en-US or zh-TW).
  --user-data-dir <dir>           Specifies the directory that user data is
                                   kept in. Can be used to open multiple
```

我们可以在命令行中使用 `code .` 命令让文件夹在 VS Code 中打开：



命令	
<code>code <路径></code>	打开文件或文件夹
<code>code .</code>	打开当前目录作为工作区
<code>code --new-window</code>	在新窗口中打开
<code>code --diff</code>	对比两个文件的内容
<code>code --wait</code>	等待窗口关闭后再返回终端
<code>code --disable-extensions</code>	禁用所有扩展运行 VS Code

VSCode 教程

<code>code --install-extension <扩展名></code>	安装指定扩展
<code>code --list-extensions</code>	列出所有已安装的扩展
<code>code --uninstall-extension <扩展名></code>	卸载指定扩展

code 命令的常用功能

1. 打开文件或文件夹

`code <文件路径>`

例如打开 `app.js` 文件：

`code app.js`

打开文件夹（以工作区的方式）：

`code <文件夹路径>`

例如打开 `my-project` 文件夹：

`code my-project`

2. 创建新文件

如果指定的文件不存在，code 会创建该文件并打开它：

`code newfile.js`

3. 以当前目录为工作区打开 VS Code

直接运行 `code` 命令会以当前终端所在目录为工作区启动 VS Code：

`code .`

4. 对比文件

code 命令支持文件内容对比：

`code --diff <文件 1> <文件 2>`

示例：

`code --diff index.html index-backup.html`

5. 查看帮助

查看 `code` 命令的所有可用选项：

`code --help`

使用 code 命令打开 vue 项目

创建 vue 项目

打开终端或命令提示符，输入以下命令创建 vue 项目：

`npm create vue@latest`

系统会提示输入项目名称，我这边输入 `runoob-vue3-app`：

Vue.js - The Progressive JavaScript Framework

? 请输入项目名称: > runoob-vue3-app

之后会有一些选项，可以根据自己的需求选择，或者一路回车：

Vue.js - The Progressive JavaScript Framework

> 请输入项目名称: ... runoob-vue3-app

> 是否使用 TypeScript 语法? ... 否 / 是

> 是否启用 JSX 支持? ... 否 / 是

> 是否引入 Vue Router 进行单页面应用开发? ... 否 / 是

> 是否引入 Pinia 用于状态管理? ... 否 / 是

> 是否引入 Vitest 用于单元测试? ... 否 / 是

> 是否要引入一款端到端 (End to End) 测试工具? > 不需要

> 是否引入 ESLint 用于代码质量检测? > 否

VSCode 教程

正在初始化项目 `/Users/Runoob/runoob-vue3-app...`

项目初始化完成，可执行以下命令：

```
cd runoob-vue3-app
npm install
npm run dev
```

项目创建完成后，进入项目文件夹并安装依赖：

```
cd runoob-vue3-app
npm install
```

安装依赖可能也需要几分钟。

输入以下命令快速启动你的 Vue 应用：

```
npm run dev
```

```
VITE v6.0.5 ready in 578 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window
→ Vue DevTools: Press Option(⌘)+Shift(⇧)+D in App to toggle the Vue DevTools

→ press h + enter to show help
```

在浏览器中打开 `http://localhost:5173`，显示如下：



You did it!

You've successfully created a project
with Vite + Vue 3.

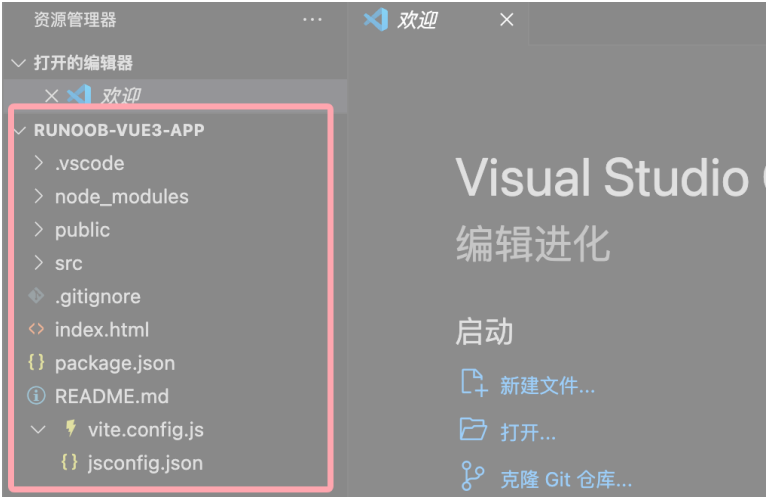
按下 `Ctrl+C` 可以停止 `vue-cli-service` 服务器。

使用 `code` 命令打开 vue 项目

从终端或命令提示符中进入我们创建的 Vue 项目文件夹 `runoob-vue3-app`，然后使用 `code` 命令让项目在 VS Code 中打开：

```
cd runoob-vue3-app
code .
```

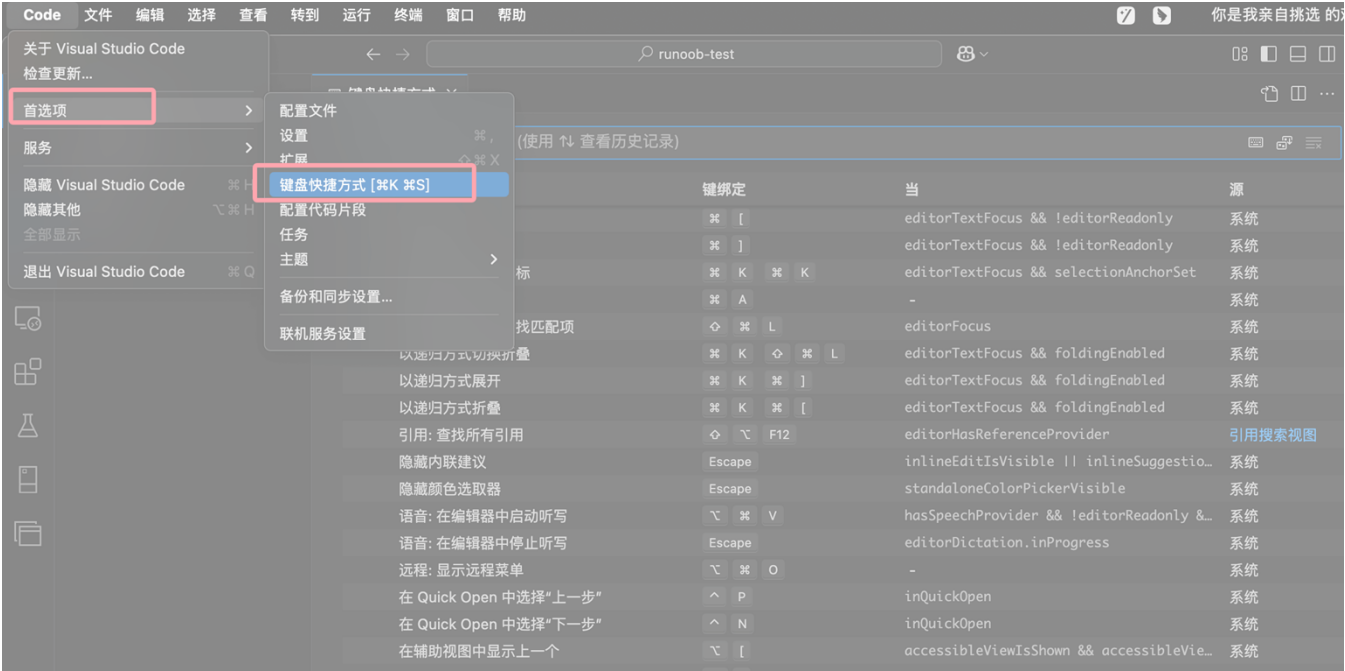
VS Code 将启动并在文件资源管理器中显示你的 Vue 项目，显示如下：



VSCode 快捷键大全

本章节是 Visual Studio Code (VS Code) 的常用快捷键大全，涵盖代码编辑、文件管理、调试等操作，便于提高开发效率。

快捷键的设置可以通过菜单栏的 **Code > 首选项 > 键盘快捷方式** 查看：



通过快捷键编辑器，您可以根据自己的需求定制键盘快捷键，提升开发效率。

说明：

- 1. macOS 的 Cmd 键对应 Windows/Linux 的 Ctrl 键。
- 2. macOS 的 Option 键对应 Windows/Linux 的 Alt 键。
- 3. 部分快捷键可能因系统或配置不同而有所差异。

1. 通用操作快捷键

功能	Windows/Linux	macOS
打开命令面板	Ctrl + Shift + P	Cmd + Shift + P
打开设置	Ctrl + ,	Cmd + ,
打开终端	Ctrl + `	Ctrl + `
新建窗口	Ctrl + Shift + N	Cmd + Shift + N
关闭窗口	Ctrl + Shift + W	Cmd + Shift + W

VSCode 教程

保存文件	Ctrl + S	Cmd + S
全部保存	Ctrl + K S	Cmd + Option + S
自动保存切换	Ctrl + Shift + P 后搜索 Auto Save	同左
快速打开，转到文件	Ctrl + P	Cmd + P
键盘快捷键设置	Ctrl + K, Ctrl + S	Cmd + K, Cmd + S

2. 文件与编辑器操作

功能	Windows/Linux	macOS
新建文件	Ctrl + N	Cmd + N
打开文件	Ctrl + O	Cmd + O
保存文件	Ctrl + S	Cmd + S
另存为	Ctrl + Shift + S	Cmd + Shift + S
关闭文件	Ctrl + W	Cmd + W
关闭所有文件	Ctrl + K, Ctrl + W	Cmd + K, Cmd + W
重新打开关闭的文件	Ctrl + Shift + T	Cmd + Shift + T
打开文件夹	Ctrl+K O	Cmd+K O
上一个文件	Ctrl+Tab	Cmd+Tab
下一个文件	Ctrl+Shift+Tab	Cmd+Shift+Tab
切换编辑器布局	Alt+Shift+数字	Cmd+Option+数字
全屏切换	F11	Ctrl+Cmd+F

3. 代码编辑快捷键

功能	Windows/Linux	
撤销	Ctrl + Z	Cmd + Z
重做	Ctrl + Y	Cmd + Y
复制	Ctrl + C	Cmd + C
剪切	Ctrl + X	Cmd + X
粘贴	Ctrl + V	Cmd + V
查找	Ctrl + F	Cmd + F
替换	Ctrl + H	Cmd + H
全选	Ctrl + A	Cmd + A
格式化代码	Shift + Alt + F	Shift + Option + F
注释行	Ctrl + /	Cmd + /
多行注释	Shift + Alt + A	Shift + Option + A
复制当前行	Alt + Shift + Down	Option + Shift + Down
删除当前行	Ctrl + Shift + K	Cmd + Shift + K
移动当前行	Alt + Up/Down	Option + Up/Down
选中当前行	Ctrl + L	Cmd + L
查找替换	Ctrl + H	Cmd + Option + F
转到行号	Ctrl + G	Cmd + G
在下方插入行	Ctrl + Enter	Cmd + Enter
在上方插入行	Ctrl + Shift + Enter	Cmd + Shift + Enter
跳转到匹配的括号	Ctrl + Shift + \	Cmd + Shift + \
缩进/取消缩进	Ctrl +] / [	Cmd +] / [
转到行首/行尾	Home / End	Cmd + ← / →
转到文件开头/结尾	Ctrl + Home / End	Cmd + ↑ / ↓

VSCode 教程

折叠/展开区域	Ctrl + Shift + [/]	Option + Cmd + [/]
切换块注释	Shift + Alt + A	Option + Shift + A
切换自动换行	Alt + Z	Option + Z

4. 多光标操作

功能	Windows/Linux	
插入光标	Alt + 点击	Option + 点击
在上方/下方插入光标	Ctrl + Alt + ↑ / ↓	Option + Cmd + ↑ / ↓
撤销上一个光标操作	Ctrl + U	Cmd + U
选择当前行	Ctrl + L	Cmd + L
选择所有匹配项	Ctrl + F2	Cmd + F2
列选择	Shift + Alt + 拖动	Shift + Option + 拖动
选中所有匹配内容	Ctrl + Shift + L	Cmd + Shift + L
选中下一个匹配	Ctrl + D	Cmd + D

5. 调试快捷键

功能	Windows/Linux	
开始调试	F5	F5
停止调试	Shift+F5	Shift+F5
步过	F10	F10
步入	F11	F11
步出	Shift+F11	Shift+F11
切换断点	F9	F9

6. 搜索和导航

功能	Windows/Linux	
全局搜索	Ctrl+Shift+F	Cmd+Shift+F
转到定义	F12	F12
转到声明	Ctrl+F12	Cmd+F12
查找引用	Shift+F12	Shift+F12
显示大纲	Ctrl+Shift+O	Cmd+Shift+O
跳转到上一个位置	Ctrl+Alt+Left	Cmd+Option+Left
跳转到下一个位置	Ctrl+Alt+Right	Cmd+Option+Right

7. 版本控制

打开版本控制视图	Ctrl+Shift+G	Cmd+Shift+G
提交代码	Ctrl+Enter	Cmd+Enter
查看变更	Ctrl+Shift+D	Cmd+Shift+D

8. 终端操作

显示集成终端	Ctrl + `	Ctrl + `
新建终端	Ctrl+Shift+``	Cmd+Shift+``
切换终端	Ctrl+PageUp/PageDown	Cmd+PageUp/PageDown
关闭终端	Ctrl+Shift+W	Cmd+Shift+W

VSCode 教程

9. 命令面板

打开命令面板	Ctrl + Shift + P	Cmd + Shift + P
打开键盘快捷键参考	Ctrl + K, Ctrl + S	Cmd + K, Cmd + S

10. 显示

切换全屏	F11	Cmd + Ctrl + F
放大/缩小	Ctrl + = / -	Cmd + = / -
切换侧边栏可见性	Ctrl + B	Cmd + B
显示资源管理器	Ctrl + Shift + E	Cmd + Shift + E
显示搜索	Ctrl + Shift + F	Cmd + Shift + F
显示源代码控制	Ctrl + Shift + G	Cmd + Shift + G
显示调试	Ctrl + Shift + D	Cmd + Shift + D
显示扩展	Ctrl + Shift + X	Cmd + Shift + X

11. 扩展操作

安装扩展	Ctrl + Shift + X	Cmd + Shift + X
扩展管理	Ctrl + Shift + P → 输入 "Extensions"	同左

12. 其他

打开 Markdown 预览	Ctrl + K, V	Cmd + K, V
禅模式	Ctrl + K, Z	Cmd + K, Z

官方提供的快捷键说明

Visual Studio Code

Keyboard shortcuts for Windows

General

Ctrl+Shift+P, F1	Show Command Palette
Ctrl+P	Quick Open, Go to File...
Ctrl+Shift+N	New window/instance
Ctrl+Shift+W	Close window/instance
Ctrl+,	User Settings
Ctrl+K Ctrl+S	Keyboard Shortcuts

Basic editing

Ctrl+X	Cut line (empty selection)
Ctrl+C	Copy line (empty selection)
Alt+ ↑ / ↓	Move line up/down
Shift+Alt+ ↑ / ↓	Copy line up/down
Ctrl+Shift+K	Delete line
Ctrl+Enter	Insert line below
Ctrl+Shift+Enter	Insert line above
Ctrl+Shift+\	Jump to matching bracket
Ctrl+] / [	Indent/outdent line
Home / End	Go to beginning/end of line
Ctrl+Home	Go to beginning of file
Ctrl+End	Go to end of file
Ctrl+↑ / ↓	Scroll line up/down
Alt+PgUp / PgDn	Scroll page up/down
Ctrl+Shift+[	Fold (collapse) region
Ctrl+Shift+]	Unfold (uncollapse) region
Ctrl+K Ctrl+[	Fold (collapse) all subregions
Ctrl+K Ctrl+]	Unfold (uncollapse) all subregions
Ctrl+K Ctrl+0	Fold (collapse) all regions
Ctrl+K Ctrl+J	Unfold (uncollapse) all regions
Ctrl+K Ctrl+C	Add line comment
Ctrl+K Ctrl+U	Remove line comment
Ctrl+/	Toggle line comment
Shift+Alt+A	Toggle block comment
Alt+Z	Toggle word wrap

Navigation

Ctrl+T	Show all Symbols
Ctrl+G	Go to Line...
Ctrl+P	Go to File...
Ctrl+Shift+O	Go to Symbol...
Ctrl+Shift+M	Show Problems panel
F8	Go to next error or warning
Shift+F8	Go to previous error or warning
Ctrl+Shift+Tab	Navigate editor group history
Alt+ ← / →	Go back / forward

Ctrl+M Toggle Tab moves focus

Search and replace

Ctrl+F	Find
Ctrl+H	Replace
F3 / Shift+F3	Find next/previous
Alt+Enter	Select all occurrences of Find match
Ctrl+D	Add selection to next Find match
Ctrl+K Ctrl+D	Move last selection to next Find match
Alt+C / R / W	Toggle case-sensitive / regex / whole word

Multi-cursor and selection

Alt+Click	Insert cursor
Ctrl+Alt+ ↑ / ↓	Insert cursor above / below
Ctrl+U	Undo last cursor operation
Shift+Alt+I	Insert cursor at end of each line selected
Ctrl+L	Select current line
Ctrl+Shift+L	Select all occurrences of current selection
Ctrl+F2	Select all occurrences of current word
Shift+Alt+→	Expand selection
Shift+Alt+←	Shrink selection
Shift+Alt+ (drag mouse)	Column (box) selection
Ctrl+Shift+Alt+ (arrow key)	Column (box) selection
Ctrl+Shift+Alt+PgUp/PgDn	Column (box) selection page up/down

Rich languages editing

Ctrl+Space, Ctrl+I	Trigger suggestion
Ctrl+Shift+Space	Trigger parameter hints
Shift+Alt+F	Format document
Ctrl+K Ctrl+F	Format selection
F12	Go to Definition
Alt+F12	Peek Definition
Ctrl+K F12	Open Definition to the side
Ctrl+.	Quick Fix
Shift+F12	Show References
F2	Rename Symbol
Ctrl+K Ctrl+X	Trim trailing whitespace
Ctrl+K M	Change file language

Editor management

Ctrl+F4, Ctrl+W	Close editor
Ctrl+K F	Close folder
Ctrl+\	Split editor
Ctrl+ 1 / 2 / 3	Focus into 1 st , 2 nd or 3 rd editor group
Ctrl+K Ctrl+ ←/→	Focus into previous/next editor group
Ctrl+Shift+PgUp / PgDn	Move editor left/right
Ctrl+K ← / →	Move active editor group

File management

Ctrl+N	New File
Ctrl+O	Open File...
Ctrl+S	Save
Ctrl+Shift+S	Save As...
Ctrl+K S	Save All
Ctrl+F4	Close
Ctrl+K Ctrl+W	Close All
Ctrl+Shift+T	Reopen closed editor
Ctrl+K Enter	Keep preview mode editor open
Ctrl+Tab	Open next
Ctrl+Shift+Tab	Open previous
Ctrl+K P	Copy path of active file
Ctrl+K R	Reveal active file in Explorer
Ctrl+K O	Show active file in new window/instance

Display

F11	Toggle full screen
Shift+Alt+0	Toggle editor layout (horizontal/vertical)
Ctrl+ = / -	Zoom in/out
Ctrl+B	Toggle Sidebar visibility
Ctrl+Shift+E	Show Explorer / Toggle focus
Ctrl+Shift+F	Show Search
Ctrl+Shift+G	Show Source Control
Ctrl+Shift+D	Show Debug
Ctrl+Shift+X	Show Extensions
Ctrl+Shift+H	Replace in files
Ctrl+Shift+J	Toggle Search details
Ctrl+Shift+U	Show Output panel
Ctrl+Shift+V	Open Markdown preview
Ctrl+K V	Open Markdown preview to the side
Ctrl+K Z	Zen Mode (Esc Esc to exit)

Debug

F9	Toggle breakpoint
F5	Start/Continue
Shift+F5	Stop
F11 / Shift+F11	Step into/out
F10	Step over
Ctrl+K Ctrl+I	Show hover

Integrated terminal

Ctrl+`	Show integrated terminal
Ctrl+Shift+`	Create new terminal
Ctrl+C	Copy selection
Ctrl+V	Paste into active terminal
Ctrl+↑ / ↓	Scroll up/down
Shift+PgUp / PgDn	Scroll page up/down
Ctrl+Home / End	Scroll to top/bottom

Other operating systems' keyboard shortcuts and additional unassigned shortcuts available at aka.ms/vscodekeybindings



Keyboard shortcuts for macOS

General

⌘⇧P, F1	Show Command Palette
⇧P	Quick Open. Go to File...
⌘⇧N	New window/instance
⇧W	Close window/instance
⇧E	User Settings
⇧K ⇧S	Keyboard Shortcuts

Basic editing

⇧X	Cut line (empty selection)
⇧C	Copy line (empty selection)
⇧↓ / ⇧↑	Move line down/up
⇧⇧↓ / ⇧⇧↑	Copy line down/up
⇧⇧K	Delete line
⇧⇧Enter / ⌘⇧Enter	Insert line below/above
⇧⇧\	Jump to matching bracket
⇧⇧ / ⇧⇧{	Indent/outdent line
Home / End	Go to beginning/end of line
⇧↑ / ⇧↓	Go to beginning/end of file
⌘⇧PgUp / ⌘⇧PgDn	Scroll line up/down
⇧⇧PgUp / ⇧⇧PgDn	Scroll page up/down
⇧⇧[/ ⇧⇧]	Fold/unfold region
⇧⇧K ⇧⇧[/ ⇧⇧K ⇧⇧]	Fold/unfold all subregions
⇧⇧K ⇧⇧O / ⇧⇧K ⇧⇧J	Fold/unfold all regions
⇧⇧K ⇧⇧C	Add line comment
⇧⇧K ⇧⇧U	Remove line comment
⇧⇧/	Toggle line comment
⇧⇧A	Toggle block comment
⇧⇧Z	Toggle word wrap

Multi-cursor and selection

⇧ + click	Insert cursor
⇧⇧↑	Insert cursor above
⇧⇧↓	Insert cursor below
⇧⇧U	Undo last cursor operation
⇧⇧l	Insert cursor at end of each line selected
⇧⇧L	Select current line
⇧⇧⇧L	Select all occurrences of current selection
⇧⇧F2	Select all occurrences of current word
⌘⇧⇧→ / ⌘⇧⇧←	Expand / shrink selection
⇧⇧ + drag mouse	Column (box) selection
⇧⇧⇧⇧↑ / ⇧⇧⇧⇧↓	Column (box) selection up/down
⇧⇧⇧⇧← / ⇧⇧⇧⇧→	Column (box) selection left/right
⇧⇧⇧⇧PgUp	Column (box) selection page up
⇧⇧⇧⇧PgDn	Column (box) selection page down

Search and replace

⇧⇧F	Find
⇧⇧⇧F	Replace
⇧⇧G / ⇧⇧⇧G	Find next/previous
⇧Enter	Select all occurrences of Find match
⇧⇧D	Add selection to next Find match
⇧⇧K ⇧⇧D	Move last selection to next Find match

Rich languages editing

⌘Space, ⇧⇧I	Trigger suggestion
⇧⇧⇧Space	Trigger parameter hints
⇧⇧⇧F	Format document
⇧⇧K ⇧⇧F	Format selection
F12	Go to Definition
⇧F12	Peek Definition
⇧⇧K F12	Open Definition to the side
⇧⇧.	Quick Fix
⇧F12	Show References
F2	Rename Symbol
⇧⇧K ⇧⇧X	Trim trailing whitespace
⇧⇧K M	Change file language

Navigation

⇧⇧T	Show all Symbols
⌘G	Go to Line...
⇧⇧P	Go to File...
⇧⇧O	Go to Symbol...
⇧⇧⇧M	Show Problems panel
F8 / ⇧F8	Go to next/previous error or warning
⌘⇧Tab	Navigate editor group history
⌘ / ⌘⇧-	Go back/forward
⌘⇧M	Toggle Tab moves focus

Editor management

⇧⇧W	Close editor
⇧⇧K F	Close folder
⇧⇧\	Split editor
⇧⇧1 / ⇧⇧2 / ⇧⇧3	Focus into 1 st , 2 nd , 3 rd editor group
⇧⇧K ⇧⇧← / ⇧⇧K ⇧⇧→	Focus into previous/next editor group
⇧⇧K ⇧⇧⇧← / ⇧⇧K ⇧⇧⇧→	Move editor left/right
⇧⇧K ⇧⇧← / ⇧⇧K ⇧⇧→	Move active editor group

File management

⇧⇧N	New File
⇧⇧O	Open File...
⇧⇧S	Save
⇧⇧⇧S	Save As...
⇧⇧⇧⇧S	Save All
⇧⇧W	Close
⇧⇧K ⇧⇧W	Close All

⇧⇧⇧T	Reopen closed editor
⇧⇧K Enter	Keep preview mode editor open
⌘Tab / ⌘⇧Tab	Open next / previous
⇧⇧K P	Copy path of active file
⇧⇧K R	Reveal active file in Finder
⇧⇧K O	Show active file in new window/instance

Display

⌘⇧F	Toggle full screen
⇧⇧⇧O	Toggle editor layout (horizontal/vertical)
⇧⇧= / ⇧⇧-	Zoom in/out
⇧⇧B	Toggle Sidebar visibility
⇧⇧⇧E	Show Explorer / Toggle focus
⇧⇧⇧F	Show Search
⌘⇧G	Show Source Control
⇧⇧⇧D	Show Debug
⇧⇧⇧X	Show Extensions
⇧⇧⇧H	Replace in files
⇧⇧⇧J	Toggle Search details
⇧⇧⇧U	Show Output panel
⇧⇧⇧V	Open Markdown preview
⇧⇧⇧V	Open Markdown preview to the side
⇧⇧K Z	Zen Mode (Esc Esc to exit)

Debug

F9	Toggle breakpoint
F5	Start/Continue
F11 / ⇧F11	Step into/ out
F10	Step over
⇧F5	Stop
⇧⇧K ⇧⇧I	Show hover

Integrated terminal

⌘~	Show integrated terminal
⌘⇧~	Create new terminal
⇧⇧C	Copy selection
⇧⇧↑ / ⇧⇧↓	Scroll up/down
PgUp / PgDn	Scroll page up/down
⇧⇧Home / End	Scroll to top/bottom

Other operating systems' keyboard shortcuts and additional unassigned shortcuts available at aka.ms/vscodekeybindings



Visual Studio Code

Keyboard shortcuts for Linux

General

Ctrl+Shift+P, F1	Show Command Palette
Ctrl+P	Quick Open, Go to File...
Ctrl+Shift+N	New window/instance
Ctrl+W	Close window/instance
Ctrl+,	User Settings
Ctrl+K Ctrl+S	Keyboard Shortcuts

Basic editing

Ctrl+X	Cut line (empty selection)
Ctrl+C	Copy line (empty selection)
Alt+ 1 / !	Move line down/up
Ctrl+Shift+K	Delete line
Ctrl+Enter / Ctrl+Shift+Enter	Insert line below/ above
Ctrl+Shift+\	Jump to matching bracket
Ctrl+] / Ctrl+[	Indent/Outdent line
Home / End	Go to beginning/end of line
Ctrl+ Home / End	Go to beginning/end of file
Ctrl+ 1 / !	Scroll line up/down
Alt+ PgUp / PgDn	Scroll page up/down
Ctrl+Shift+ [/]	Fold/unfold region
Ctrl+K Ctrl+ [/]	Fold/unfold all subregions
Ctrl+K Ctrl+0 / Ctrl+K Ctrl+J	Fold/Unfold all regions
Ctrl+K Ctrl+C	Add line comment
Ctrl+K Ctrl+U	Remove line comment
Ctrl+ /	Toggle line comment
Ctrl+Shift+A	Toggle block comment
Alt+Z	Toggle word wrap

Rich languages editing

Ctrl+Space, Ctrl+I	Trigger suggestion
Ctrl+Shift+Space	Trigger parameter hints
Ctrl+Shift+I	Format document
Ctrl+K Ctrl+F	Format selection
F12	Go to Definition
Ctrl+Shift+F10	Peek Definition
Ctrl+K F12	Open Definition to the side
Ctrl+.	Quick Fix
Shift+F12	Show References
F2	Rename Symbol
Ctrl+K Ctrl+X	Trim trailing whitespace
Ctrl+M	Change file language

Multi-cursor and selection

Alt+Click	Insert cursor*
Shift+Alt+ 1 / !	Insert cursor above/below
Ctrl+U	Undo last cursor operation
Shift+Alt+I	Insert cursor at end of each line selected
Ctrl+L	Select current line
Ctrl+Shift+L	Select all occurrences of current selection
Ctrl+F2	Select all occurrences of current word
Shift+Alt + --	Expand selection
Shift+Alt + --	Shrink selection
Shift+Alt + drag mouse	Column (box) selection

Display

F11	Toggle full screen
Shift+Alt+0	Toggle editor layout (horizontal/vertical)
Ctrl+ = / -	Zoom in/out
Ctrl+B	Toggle Sidebar visibility
Ctrl+Shift+E	Show Explorer / Toggle focus
Ctrl+Shift+F	Show Search
Ctrl+Shift+G	Show Source Control
Ctrl+Shift+D	Show Debug
Ctrl+Shift+X	Show Extensions
Ctrl+Shift+H	Replace in files
Ctrl+Shift+J	Toggle Search details
Ctrl+Shift+C	Open new command prompt/terminal
Ctrl+K Ctrl+H	Show Output panel
Ctrl+Shift+V	Open Markdown preview
Ctrl+K V	Open Markdown preview to the side
Ctrl+K Z	Zen Mode (Esc Esc to exit)

Search and replace

Ctrl+F	Find
Ctrl+H	Replace
F3 / Shift+F3	Find next/previous
Alt+Enter	Select all occurrences of Find match
Ctrl+D	Add selection to next Find match
Ctrl+K Ctrl+D	Move last selection to next Find match

Navigation

Ctrl+T	Show all Symbols
Ctrl+G	Go to Line...
Ctrl+P	Go to File...
Ctrl+Shift+O	Go to Symbol...
Ctrl+Shift+M	Show Problems panel
F8	Go to next error or warning
Shift+F8	Go to previous error or warning
Ctrl+Shift+Tab	Navigate editor group history
Ctrl+Alt+-	Go back
Ctrl+Shift+-	Go forward
Ctrl+M	Toggle Tab moves focus

Editor management

Ctrl+W	Close editor
Ctrl+K F	Close folder
Ctrl+ \	Split editor
Ctrl+ 1 / 2 / 3	Focus into 1 st , 2 nd , 3 rd editor group
Ctrl+K Ctrl + --	Focus into previous editor group
Ctrl+K Ctrl + --	Focus into next editor group
Ctrl+Shift+PgUp	Move editor left
Ctrl+Shift+PgDn	Move editor right
Ctrl+K --	Move active editor group left/up
Ctrl+K --	Move active editor group right/down

File management

Ctrl+N	New File
Ctrl+O	Open File...
Ctrl+S	Save
Ctrl+Shift+S	Save As...
Ctrl+W	Close
Ctrl+K Ctrl+W	Close All
Ctrl+Shift+T	Reopen closed editor
Ctrl+K Enter	Keep preview mode editor open
Ctrl+Tab	Open next
Ctrl+Shift+Tab	Open previous
Ctrl+K P	Copy path of active file
Ctrl+K R	Reveal active file in Explorer
Ctrl+K O	Show active file in new window/instance

Debug

F9	Toggle breakpoint
F5	Start / Continue
F11 / Shift+F11	Step into/out
F10	Step over
Shift+F5	Stop
Ctrl+K Ctrl+I	Show hover

Integrated terminal

Ctrl+`	Show integrated terminal
Ctrl+Shift+`	Create new terminal
Ctrl+Shift+C	Copy selection
Ctrl+Shift+V	Paste into active terminal
Ctrl+Shift+ 1 / !	Scroll up/down
Shift+ PgUp / PgDn	Scroll page up/down
Shift+ Home / End	Scroll to top/bottom

* The Alt+Click gesture may not work on some Linux distributions. You can change the modifier key for the Insert cursor command to Ctrl+Click with the "editor.multiCursorModifier" setting.

以上快捷键可以通过 **键绑定设置** 自定义。在 VS Code 中, 按下 **Ctrl+K Ctrl+S** (macOS 上 **Cmd+K Cmd+S**) 打开键绑定页面, 方便查询和修改快捷键。

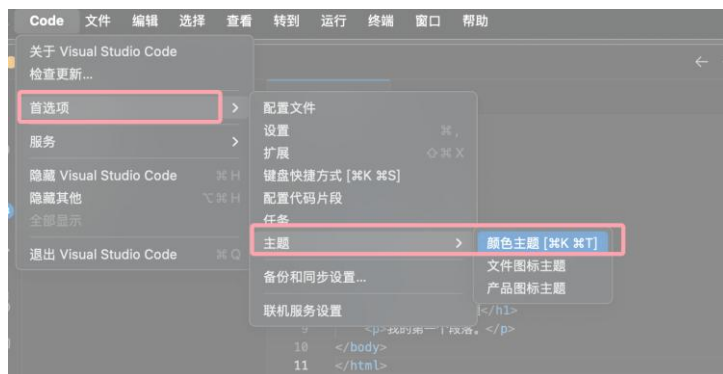
VSCode 主题设置

VSCode 主题设置允许我们根据个人偏好和工作环境自定义 Visual Studio Code 的用户界面配色。一个主题不仅会影响 VS Code 的用户界面元素, 还会影响代码编辑器的高亮配色。

从命令面板预览主题

从菜单栏选择 **Code > 设置(首选项) > 主题 > 颜色主题** 菜单项: 或使用命令 **Preferences: Color Theme** (⌘K ⌘T) 打开主题选择器。

VSCode 教程



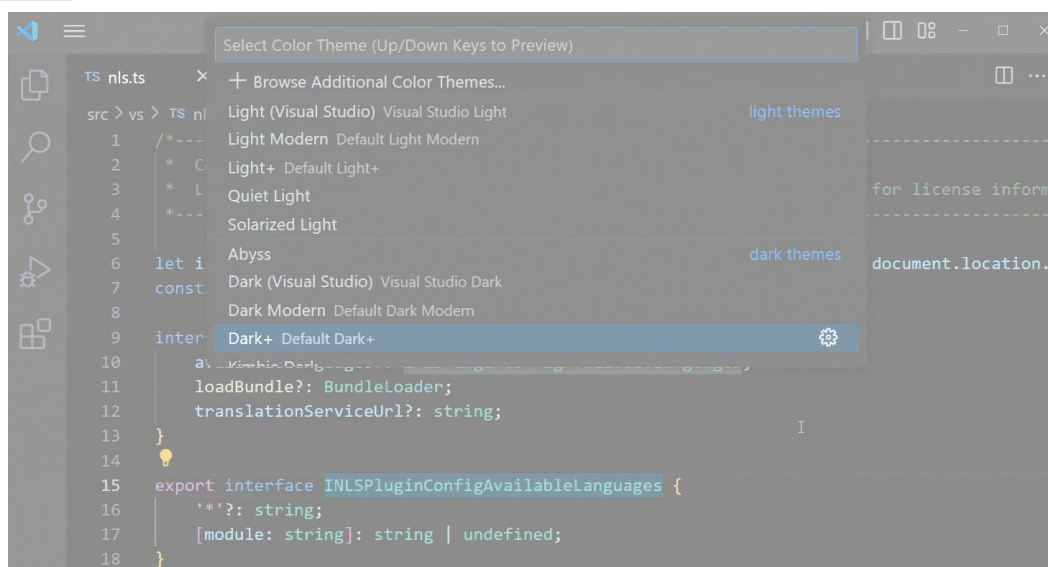
使用快捷键打开：

- Windows/Linux: **Ctrl + K Ctrl + T** 打开主题选择器。
- macOS: **⌘K ⌘T** 打开主题选择器

打开后，命令框如下所示：



使用 **上下箭头键** 在主题列表中浏览，并实时预览主题配色，选中您喜欢的主题后，按 **Enter** 确认。



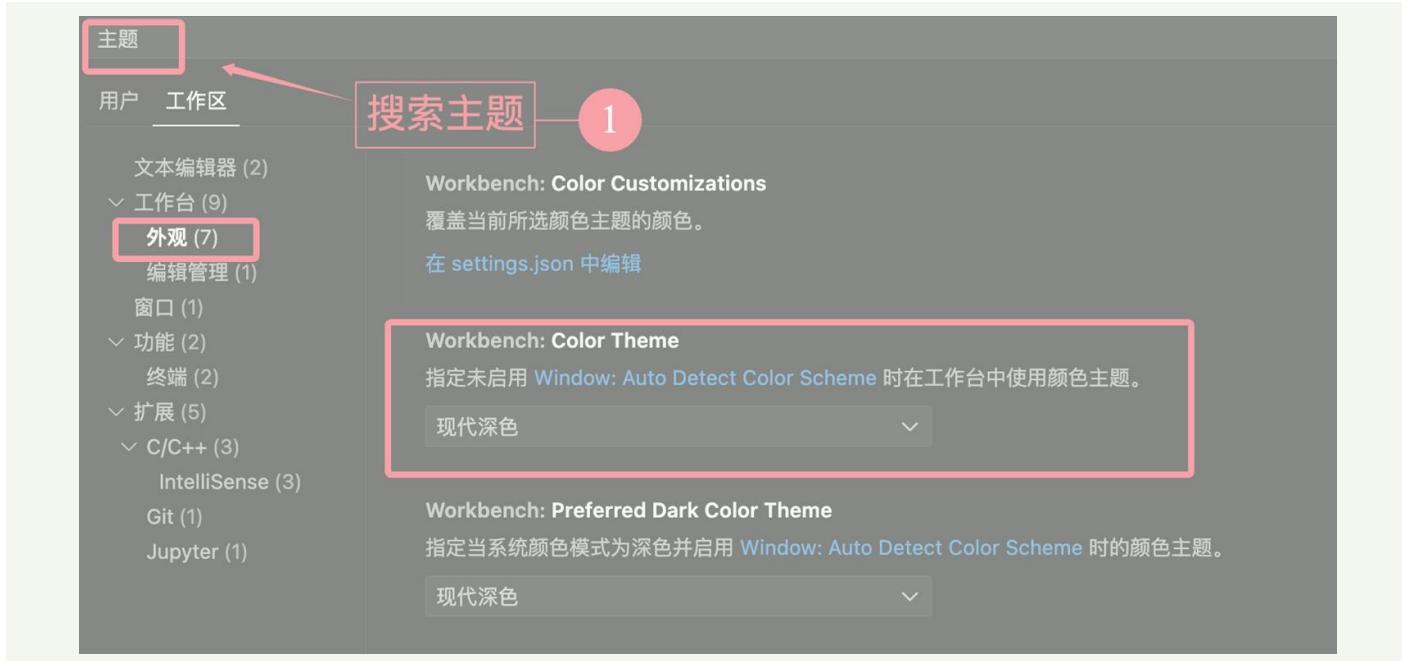
当前激活的主题会存储在用户设置中，关于设置菜单打开参考 [VS Code 设置](#)：

// 定义工作界面使用的颜色主题

"workbench.colorTheme": "Solarized Dark"

提示：默认情况下，主题存储在用户设置中，并会全局应用于所有工作区。

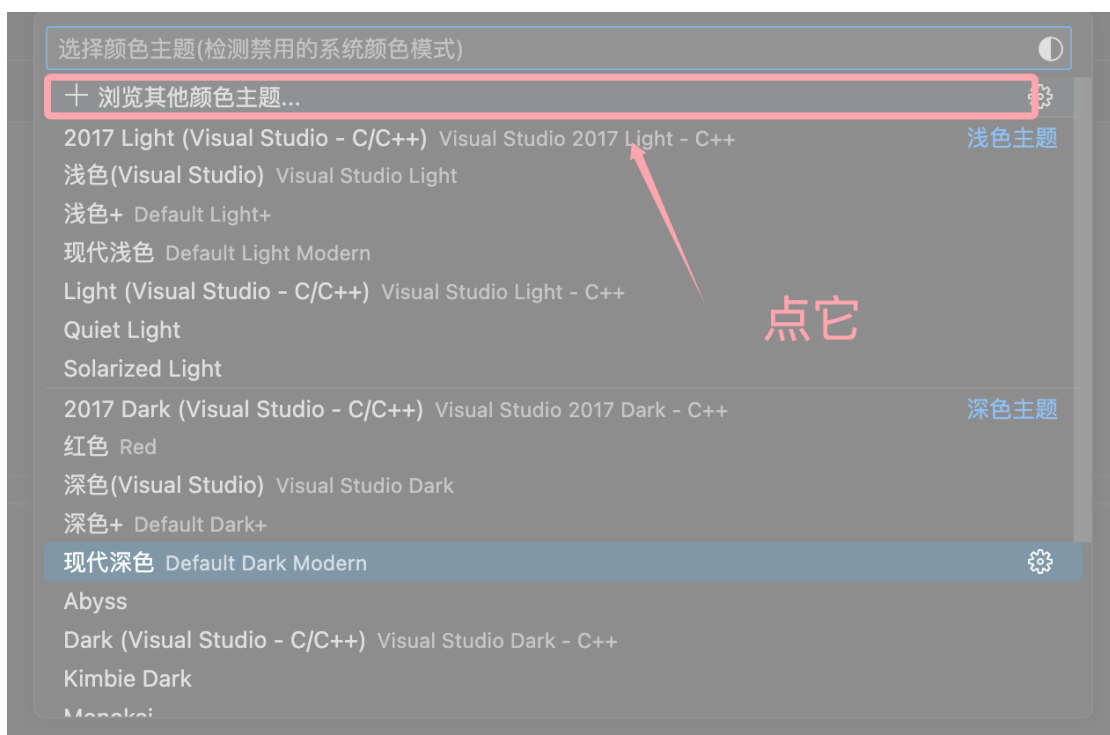
如果希望为某个特定工作区配置主题，可以在工作区设置中指定。



从市场中获取主题

从主题选择器浏览市场主题

在颜色主题选择器中点击 流量其他颜色主题（Browse Additional Color Themes...） 选项，跳转到市场主题浏览页面：



点击后会列出主题列表，使用 **上下箭头键** 在主题列表中浏览，并实时预览主题配色：

在颜色主题选择器中点击 流量其他颜色主题（Browse Additional Color Themes...） 选项，跳转到市场主题浏览页面：



从扩展视图中搜索主题

VS Code 除了自带了一些主题外，扩展市场也提供了非常多的主题：

比如搜索 **One Dark Pro** 这个主题：

也可以在搜索框中使用 **@category:"themes"** 过滤器，查找相关主题：



在 AI 技术深度融入编程领域的今天，VSCode 作为开发者最青睐的编辑器之一，其丰富的 AI 插件生态正成为提升生产力的关键。

本文将介绍 VS Code 中常用的 AI 扩展，我们可以选择合适的扩展来提高编程效率。

1、Cline

Cline 是一款强大的 VSCode 插件，旨在通过集成多种 AI 模型（如 OpenAI、DeepSeek、Claude 3.5 等）来提升开发效率。

Cline 可以在大型项目中轻松创建、修改和浏览文件，实现代码库的无缝导航。

Cline 支持代码生成、终端命令执行、Web 开发辅助等功能。

Cline 可以通过无头浏览器启动网站，进行交互操作并捕获日志，助力调试和优化。

扩展搜索关键词：**Cline**

插件链接地址：<https://marketplace.visualstudio.com/items?itemName=saoudrizwan.claude-dev>

Github 地址：<https://github.com/cline/cline>

2、Roo Code

Roo Code（前身为 Roo Cline）是一款集成于 VSCode 的强大 AI 编程助手，基于 Cline 进行二次开发，功能更为强大和灵活。

Roo Code 支持多种 AI 模型，包括 DeepSeek-V3、DeepSeek-R1、OpenAI、Google Gemini 等，并兼容自定义 API 和本地部署模型（如 Ollama）。

扩展搜索关键词：**Roo Code**

插件链接地址：<https://marketplace.visualstudio.com/items?itemName=RooVeterinaryInc.roo-cline>

Github 地址：<https://github.com/RooVetGit/Roo-Code>

3、Continue

Continue 是一款开源的 AI 代码助手插件，通过接入多种大语言模型（如 OpenAI、DeepSeek、Claude 等）来实现代码补全、生成、优化、错误修复等功能。

Continue 提供了一个聊天界面，开发者可以通过自然语言与 AI 交互，帮助理解代码逻辑、解决问题或生成代码片段。

扩展搜索关键词：**Continue**

插件链接地址：<https://marketplace.visualstudio.com/items?itemName=Continue.continue>

Github 地址：<https://github.com/continuedev/continue>

4、其他扩展

Codeium

Codeium 是一款免费的 AI 编程助手，支持多种 IDE 和超过 70 种编程语言。

扩展搜索关键词：Codeium

通义灵码

通义灵码是由阿里巴巴推出的一款 AI 编程工具，支持多种编程语言和主流 IDE。

VSCode 教程

扩展搜索关键词：**通义灵码**

Fitten Code

Fitten Code 是一款由非十科技（Fitten Tech）开发的 AI 编程助手，基于自研的大型代码模型，旨在通过智能代码生成、补全、调试等功能，显著提升开发效率。

扩展搜索关键词：**Fitten Code**

CodeGeeX

CodeGeeX 是一款由智谱 AI 开发的智能编程助手，基于强大的多语言代码生成模型。

扩展搜索关键词：**CodeGeeX**

GitHub Copilot

GitHub Copilot 是由 GitHub 和 OpenAI 合作开发的一款 AI 编程助手，通过智能代码补全、生成、调试等功能，显著提升开发效率。

扩展搜索关键词：**GitHub Copilot**

Tabnine

Tabnine 是以色列团队开发的 AI 补全工具，支持 80+ 语言，能预测整行甚至函数级代码。

扩展搜索关键词：**Tabnine**

Codeium

Codeium 扩展是一款免费的 AI 驱动代码补全工具，支持 70+ 编程语言，提供智能代码生成、搜索和交互式对话功能。

扩展搜索关键词：**Codeium**

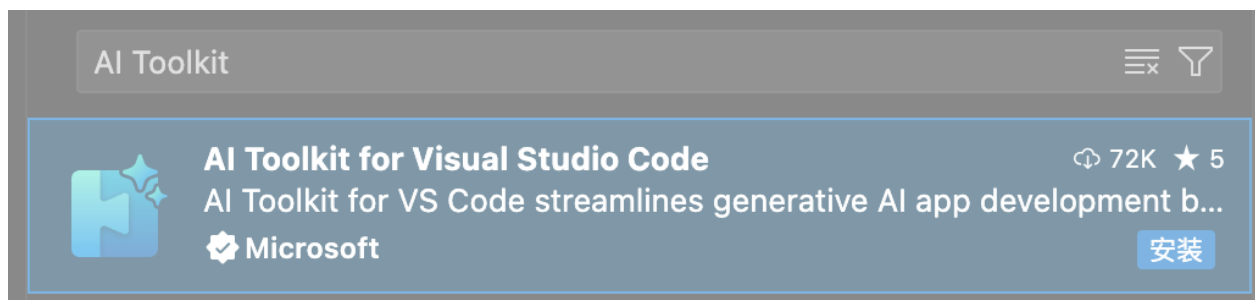
AI Toolkit

AI Toolkit 是一个用于开发、部署和维护人工智能应用的工具集合，旨在简化 AI 开发流程，帮助开发者快速构建和运行 AI 系统。

AI Toolkit 提供丰富的预训练模型，支持从公共目录（如 Azure AI Studio、Hugging Face、GitHub 等）下载和集成模型。

AI Toolkit 支持本地和云端的模型微调，允许开发者根据需求对模型进行定制化调整。

扩展搜索关键词：**AI Toolkit**



Blackbox AI

Blackbox AI 是一款基于深度学习的智能编码助手，支持代码生成、错误检测、代码翻译和实时搜索等功能。

扩展搜索关键词：**Blackbox AI**

Trae

Trae (/treɪ/) 是字节跳动推出的一款面向开发者的 AI 驱动的开发环境 (IDE)，类似 Cursor，支持中文，集成 Claude 和 GPT-4o 模型，目前免费。

Trae 与 AI 深度集成，提供智能问答、代码自动补全以及基于 Agent 的 AI 自动编程能力。

访问 **Trae 官网** <https://www.trae.com.cn/>，默认情况，下载按钮会自动匹配我们的电脑系统，我们也可以找到适合自己电脑操作系统的 Trae 安装包，进行下载。



Cursor

Cursor 不是 VS Code 的扩展，但是它是基于 VS Code 开发的一款编辑器，它在保留 VS Code 强大功能和熟悉操作体验的同时，专注于集成 AI 技术。

Cursor 操作界面类似 VSCode，是个付费开发工具，免费使用的有限制。

Cursor 教程：<https://www.runoob.com/cursor/cursor-intro.html>

VSCode 接入 DeepSeek

VSCode 拥有一个庞大的扩展市场，用户可以根据自己的需要安装各种扩展来增强编辑器的功能，包括语言支持、代码格式化工具、版本控制集成、主题和图标等。

VS Code 可以通过扩展很好的接入我们想要大模型，常用的扩展可以参考：[VS Code AI 扩展](#)。

接下来我们来学习下如何在 VS Code 上通过 Roo Code 扩展装上最近很火的 DeepSeek。

本文以 Roo Code 为例演示如何接入 DeepSeek。

安装 Roo Code 扩展

Roo Code (前身为 Roo Cline) 是一款基于 VS Code 的 AI 编程助手插件，通过集成多种 AI 模型和强大的自动化功能，为开发者提供高效、智能的编程体验。

Roo Code 可与 OpenAI、DeepSeek、Anthropic、Google Gemini 等主流 API 无缝对接，还支持通过 Ollama 使用本地模型，开发者能根据需求和预算灵活切换。

Roo Code 提供了多种预设模式，包括 Code (代码模式)、Architect (架构模式) 和 Ask (问答模式)，并支持用户自定。例如，用户可以创建一个专注于编写测试用例的 QA 工程师角色。

扩展搜索关键词：**Roo Code**

VSCode 教程

插件链接地址: <https://marketplace.visualstudio.com/items?itemName=RooVeterinaryInc.roo-cline>

Github 地址: <https://github.com/RooVetGit/Roo-Code>

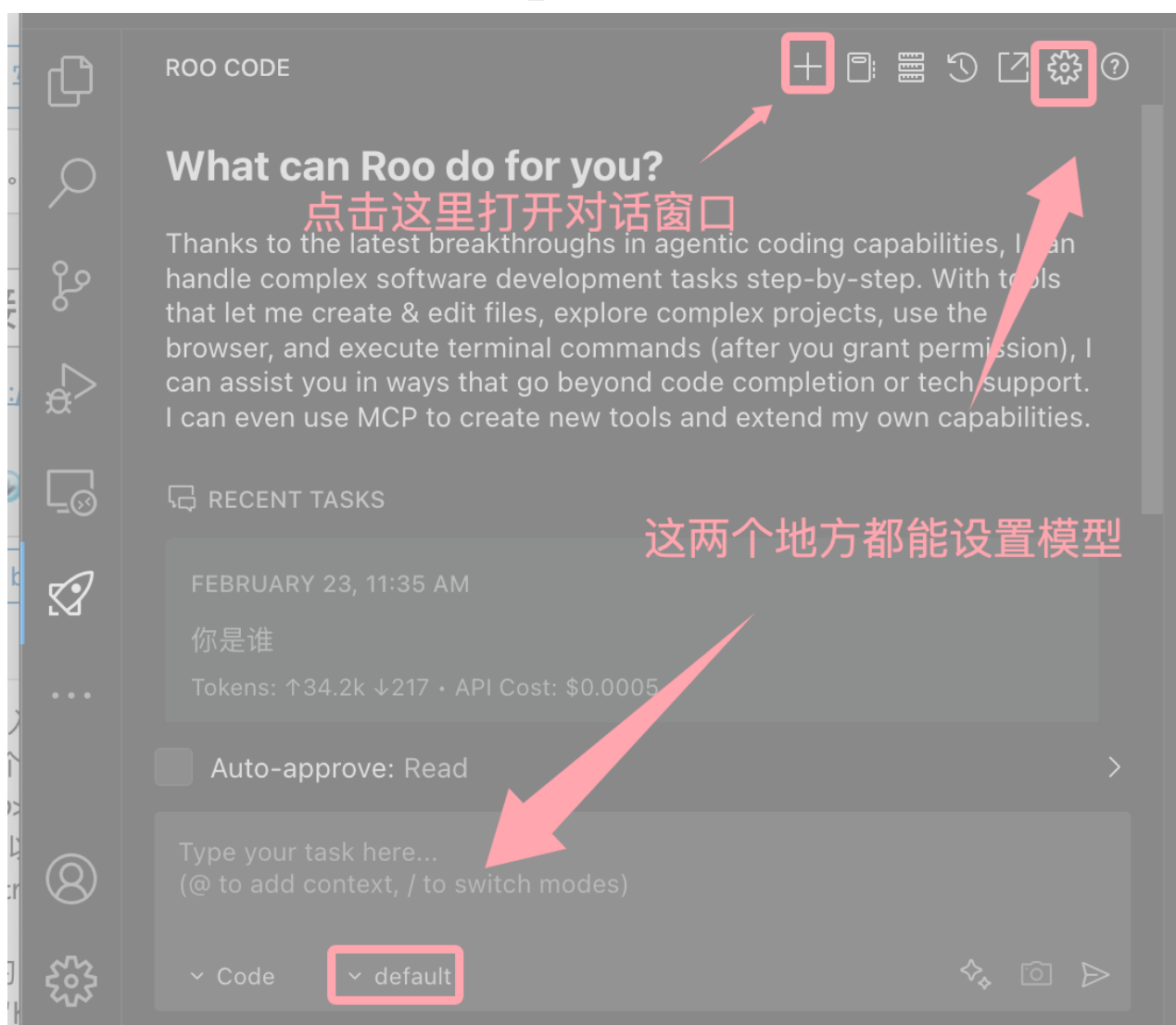
设置 API Key

安装扩展后在左侧活动栏会有个小火箭的图标, 打开就可以看到支持的大模型, 我们可以选择 DeepSeek:

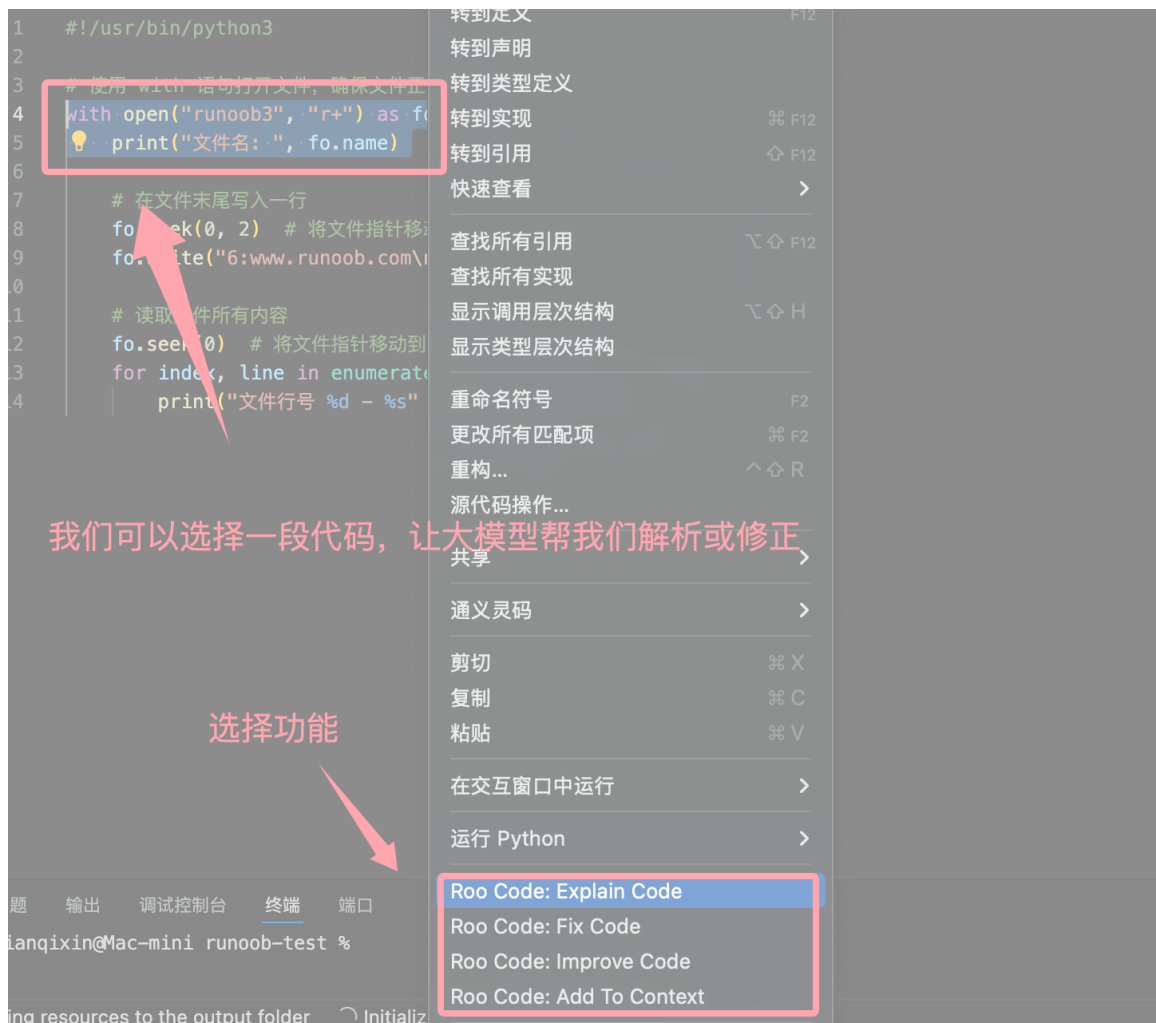
填写我们申请的 API Key:

我们可以在 DeepSeek 官网 API 开放平台 <https://platform.deepseek.com/usage> 申请 API Key:

安装成功后, 我们就可以点击扩展右上角的加号 **+**, 开启对话窗口:



我们也可以选择文件中的一小段代码, 让大模型帮我们解析或修正:



使用 @ 符号为 AI 提供上下文。输入 @ 即可查看文件夹中所有文件和代码符号的列表。

